

THREE VOLUMES · AUG 2026

The Complete
AI Handbook

CONTENTS

COMBINED EDITION · 23 AUGUST 2026

Everything, in one file.

Three handbooks bound together: the Claude platform and its threat model, the AI foundations behind it, and the Microsoft track through to Security Copilot Level 400. Forty-six diagrams, forty-seven tables, three glossaries and five projects.

VOLUME 1 **Claude, beginner to security** Interfaces, models, workspace, capabilities — and the agentic threat model.

VOLUME 2 · WEEKS 1–4 **AI basics** Ng, 3Blue1Brown and StatQuest: concepts, mechanism, measurement.

VOLUME 3 · WEEKS 5–12 **Microsoft & Security Copilot** AI-901 concepts, Copilot literacy, SC-5006, and Ninja Level 400.

UNIFIED REFERENCE · FACT-CHECKED 23 AUGUST 2026

Claude, from first message to

threat model.

Ten parts, twenty-two diagrams, one mental model. Every session you will ever run with Claude is one pick from each of five layers — start there and the rest of the handbook slots in underneath.

Supersedes the April 2026 edition

L1	Interfaces Web, desktop, mobile, Claude Code, IDE, Chrome, Office, Slack, API
L2	Models Haiku 4.5 · Sonnet 5 · Opus 5 · Mythos tier (Fable 5 / Mythos 5)
L3	Reasoning Adaptive thinking, effort dial, fast mode, task budgets
L4	Workspace Projects, Artifacts, Memory, Styles, chat history
L5	Capabilities Search, files, code execution, connectors, MCP, Skills, plugins, sub-agents

Pick an interface → a model runs → a reasoning depth is chosen → your workspace supplies context → capabilities let Claude act. That is one session, every time.

PART 00 — CORRECTIONS

What changed since your April 2026 edition

Your original file was accurate for April. Four months later, the model layer has been rebuilt from the ground up and the reasoning layer works differently. These are the corrections applied throughout this handbook.

YOUR FILE SAID	CORRECT AS OF 23 AUG 2026
Haiku 4.5 / Sonnet 4.6 / Opus 4.7	Haiku 4.5, Sonnet 5 (30 Jun 2026), Opus 5 (24 Jul 2026), plus a new top tier
Opus is the flagship	The Mythos tier now sits above Opus: Fable 5 (public) and Mythos 5 (restricted)
Extended thinking is a mode you turn on	Adaptive thinking is on by default on Sonnet 5; manual extended thinking is deprecated. Depth is set by an effort dial
1M context on Sonnet 4.6 / Opus 4.6+	1M context is the default and the maximum on Sonnet 5 — there is no smaller variant. 128k max output
~299 official connectors	The directory passed 400+ by mid-2026; third-party censuses counted over 1,200 listings in Aug 2026
Three Claude Code CVEs	28+ CVEs catalogued by July 2026, including two rated CVSS 10.0. Patch floor has moved well past 2.1.90
Office sidebars in beta	Claude for Excel, Word and PowerPoint are GA on paid plans; Outlook is in beta

Read this as a snapshot, not a constant. The five-layer structure has been stable for two years. Version numbers, prices and connector counts churn monthly — check Anthropic's own pages before you quote any specific figure in front of a colleague.

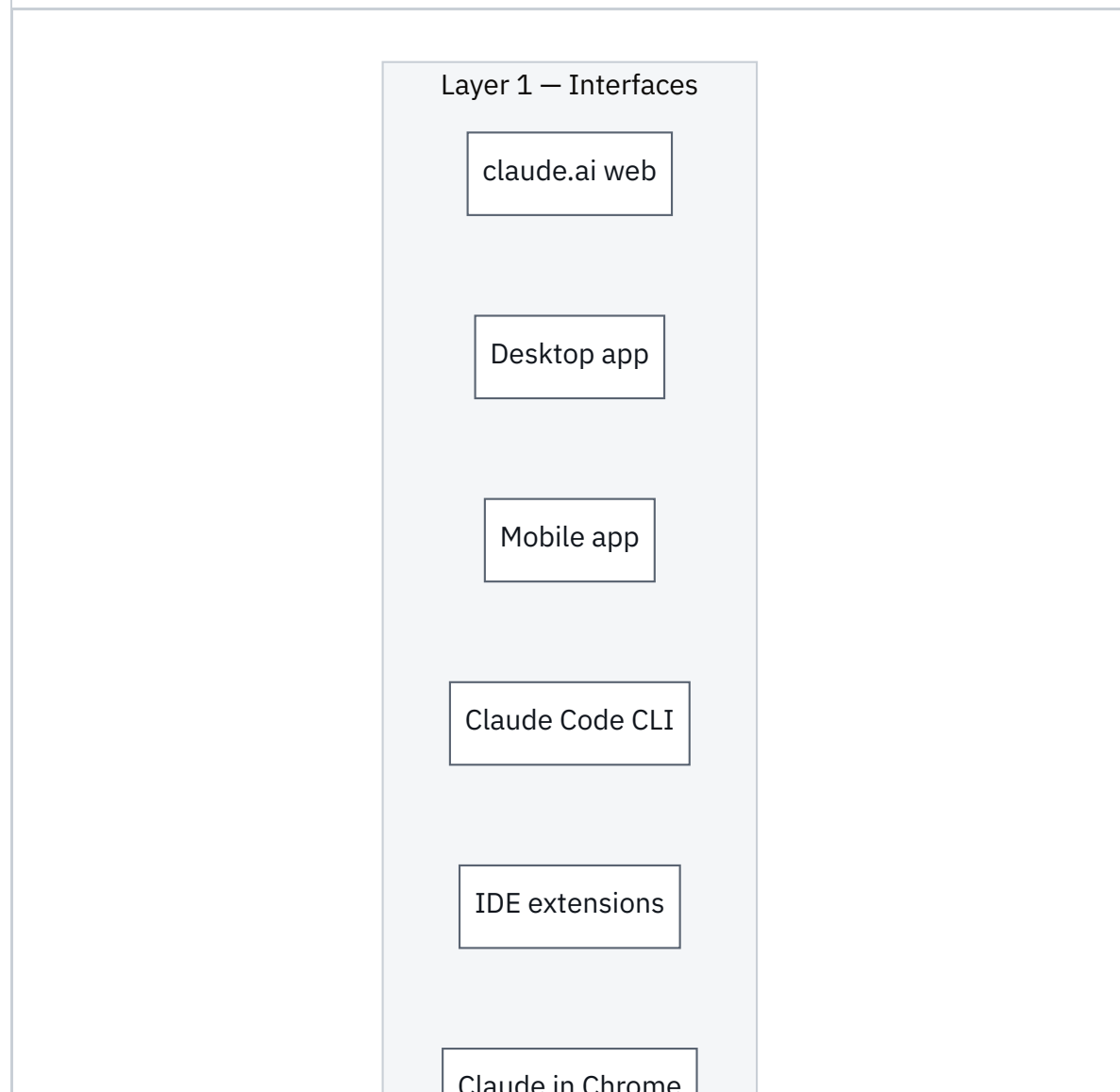
1.1 What Claude is

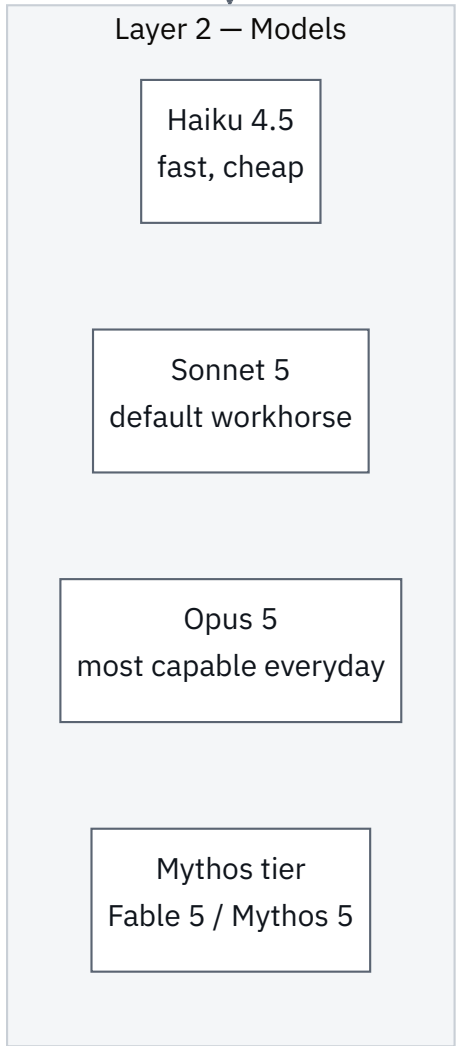
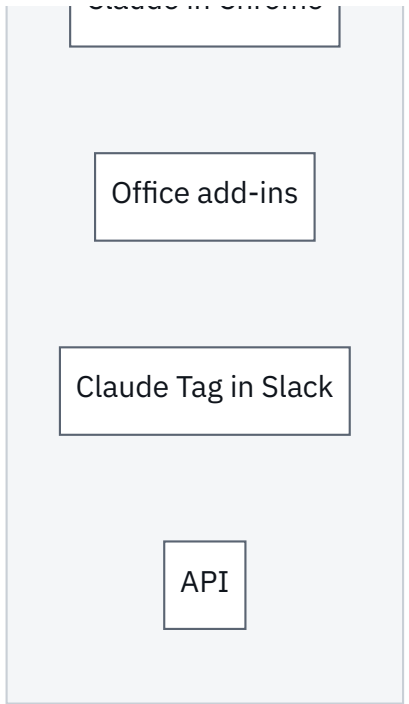
Claude is an AI assistant built by Anthropic, an AI-safety company founded in San Francisco in 2021 by former OpenAI researchers. You type a message, Claude responds — but in 2026 that description undersells it badly. Claude is an **agentic platform**: it reads your files, uses your apps, writes and runs code, drives your browser, and carries out multi-step work on your behalf while you do something else.

1.2 The five layers

The stack at the top of this page is the whole ecosystem. Here it is as a diagram you can trace with a finger:

FIG 1.1 — THE FIVE LAYERS OF A CLAUDE SESSION







Layer 3 — Reasoning

Adaptive thinking

Effort dial

Fast mode

Task budgets



Layer 4 — Workspace

Projects

Artifacts

Memory

Styles

History



Layer 5 – Capabilities

Web search and fetch

Files and vision

Code execution

Connectors

MCP servers

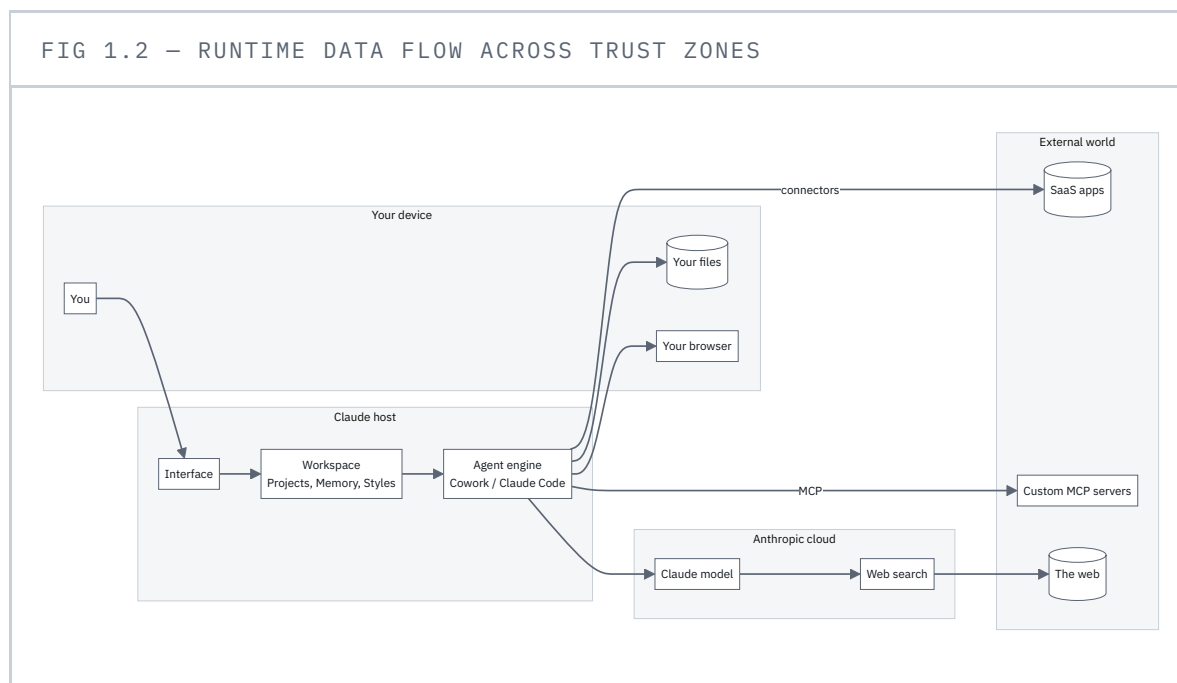
Skills

Plugins

Sub-agents

1.3 How the pieces connect at runtime

The layer diagram is the menu. This one is the plumbing — where your data actually travels when Claude does something for you.



Everything you do with Claude sits somewhere on that map. Part 8 takes the same picture and marks where the attacks land.

[CHECK YOURSELF — PART 1](#) +

PART 02 — LEVEL 1

Absolute basics

For someone new to all of this. Plain English. Skip to Part 3 if you have already had a real conversation with Claude.

2.1 Just enough AI theory

AI is software that learns patterns from examples instead of following rules someone wrote by hand.

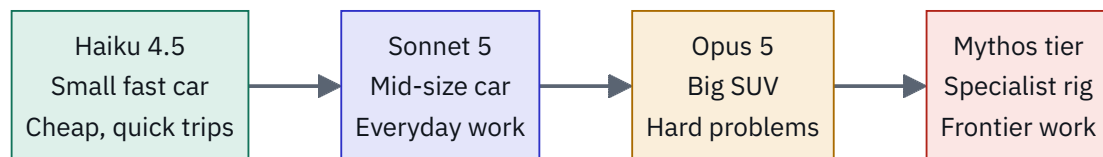
REGULAR SOFTWARE	AI
Follows the rules it was given	Learns patterns from examples
A calculator: $2 + 2 = 4$, always	A chatbot: one question, many valid answers
Great at exact tasks	Great at fuzzy, human-shaped tasks

2.2 Claude and ChatGPT

Cousins, not twins. ChatGPT is OpenAI's; Claude is Anthropic's. Similar ideas, different personalities, different safety approaches. Anthropic trains Claude against a written set of principles — an approach it calls **Constitutional AI**.

2.3 Models: the car analogy UPDATED

FIG 2.1 — FOUR TIERS, FOUR JOBS



You rarely need to pick. Sonnet 5 is the default on Free and Pro and handles the large majority of work. Opus 5 is the default on Max and the strongest option on Pro. Above Opus sits the Mythos tier: **Fable 5** is the public one, and **Mythos 5** is limited to a small number of trusted organisations.

Worth knowing: requests to Opus 5 or Fable 5 that trip Anthropic's safety classifiers are quietly routed to a different model rather than refused outright. If an answer ever feels like it came from a less capable model, that may be why.

2.4 Core vocabulary

TERM	PLAIN ENGLISH
Prompt	The message you send
Response	What Claude sends back
Conversation	One back-and-forth session
Token	Roughly three-quarters of a word — how AI measures text
Context	Everything Claude can see right now
Hallucination	Claude sounding confident and being wrong
Model	A version of Claude's brain

2.5 What Claude is good at

- **Writing** — emails, reports, posts, fiction
- **Reading and summarising** — PDFs, articles, long documents
- **Explaining** — "explain this like I'm 10"
- **Brainstorming** — "give me 10 ideas for..."
- **Fixing text** — grammar, tone, clarity
- **Translation** and light research with web search

2.6 Where it struggles

- Exact arithmetic on big numbers — use a calculator, or ask Claude to run code
- Very recent news, unless it searches the web
- Things it simply does not know — it may hallucinate. Verify anything that matters.

2.7 Your first five exercises

1. "Introduce yourself and tell me what you're best at."
2. "Explain what cloud computing is like I'm 12."
3. "Write a short friendly email asking my landlord to fix a leaky tap."
4. Paste a news article → "Summarise this in 3 bullet points."
5. "Give me 10 hobby ideas for someone who works at a desk all day."

2.8 Basic safety rules

Don't paste passwords, banking credentials or full national ID numbers. Treat medical, legal and financial answers as a starting point, then verify with a professional. Think of Claude as a well-read stranger who lives in the cloud.

CHECK YOURSELF — PART 2 +

PART 03 — LEVEL 2

Everyday mastery

3.1 The CRISP prompt framework

Every strong prompt carries most of these five parts.

FIG 3.1 — CRISP



Weak: "Write me an Instagram post about coffee."

CRISP-strong: I run a small coffee shop in Seattle (*Context*). Act as a friendly marketing consultant (*Role*). Write three Instagram captions (*Instruction*),

each under 150 characters, warm but not cheesy (*Specifics*), to bring in Sunday morning customers (*Purpose*).

3.2 Seven reusable patterns

#	PATTERN	EXAMPLE
1	Explain like I'm X	"Explain VPNs like I'm a 70-year-old with a new smartphone"
2	Compare in a table	"Compare hybrids vs EVs. Columns: upfront cost, running cost, environment, charging"
3	Critique my draft	"Tough editor mode: five things that could be stronger. Don't rewrite yet."
4	Give me three options	"One formal, one casual, one playful. Then recommend one."
5	Devil's advocate	"Argue the strongest case against this decision."
6	Teach in five lessons	"Teach me X in five lessons, each with one action for this week."
7	Interview me	"Ask me eight questions, one at a time, to draft my About page."

3.3 Five habits that separate beginners from pros

1. **Show, don't tell** — paste an example of the style you want.
2. **Let Claude ask first** — "Ask clarifying questions before starting."
3. **Iterate, don't restart** — "Keep everything, change paragraph 2."
4. **Ask for the reasoning** on hard problems.
5. **Permit uncertainty** — "If you're not sure, say so" measurably reduces

hallucinations.

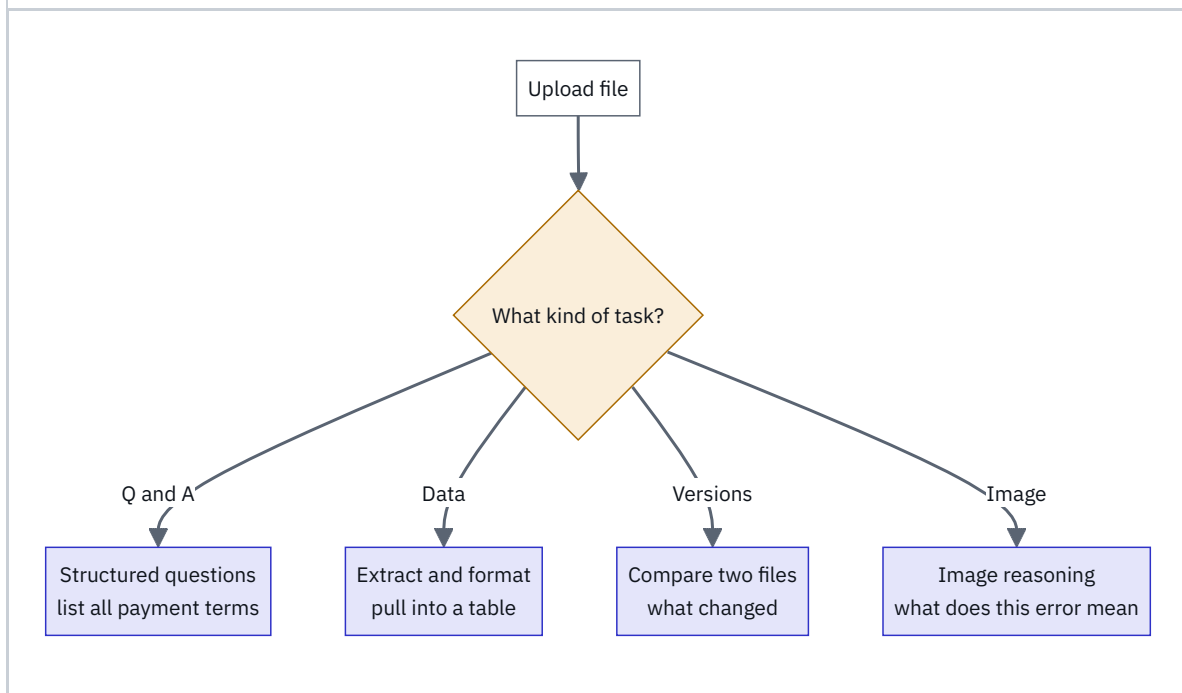
UPDATED

The old advice "say think step by step" now matters less than it did. Sonnet 5 and Opus 5 decide how deeply to reason on their own. Say it if you want reasoning shown, not to unlock quality.

3.4 Attachment mastery

Four patterns that go well beyond "summarise this".

FIG 3.2 – CHOOSING WHAT TO DO WITH AN UPLOADED FILE



3.5 Styles and personal preferences

Set once, save hours. A custom style worth stealing:

Plain English. No bullet points unless I ask. Short paragraphs. Don't hedge with "it depends" — give a recommendation first, then the caveats.

Personal preferences are for your non-obvious quirks: metric units, preferred language, topics to avoid.

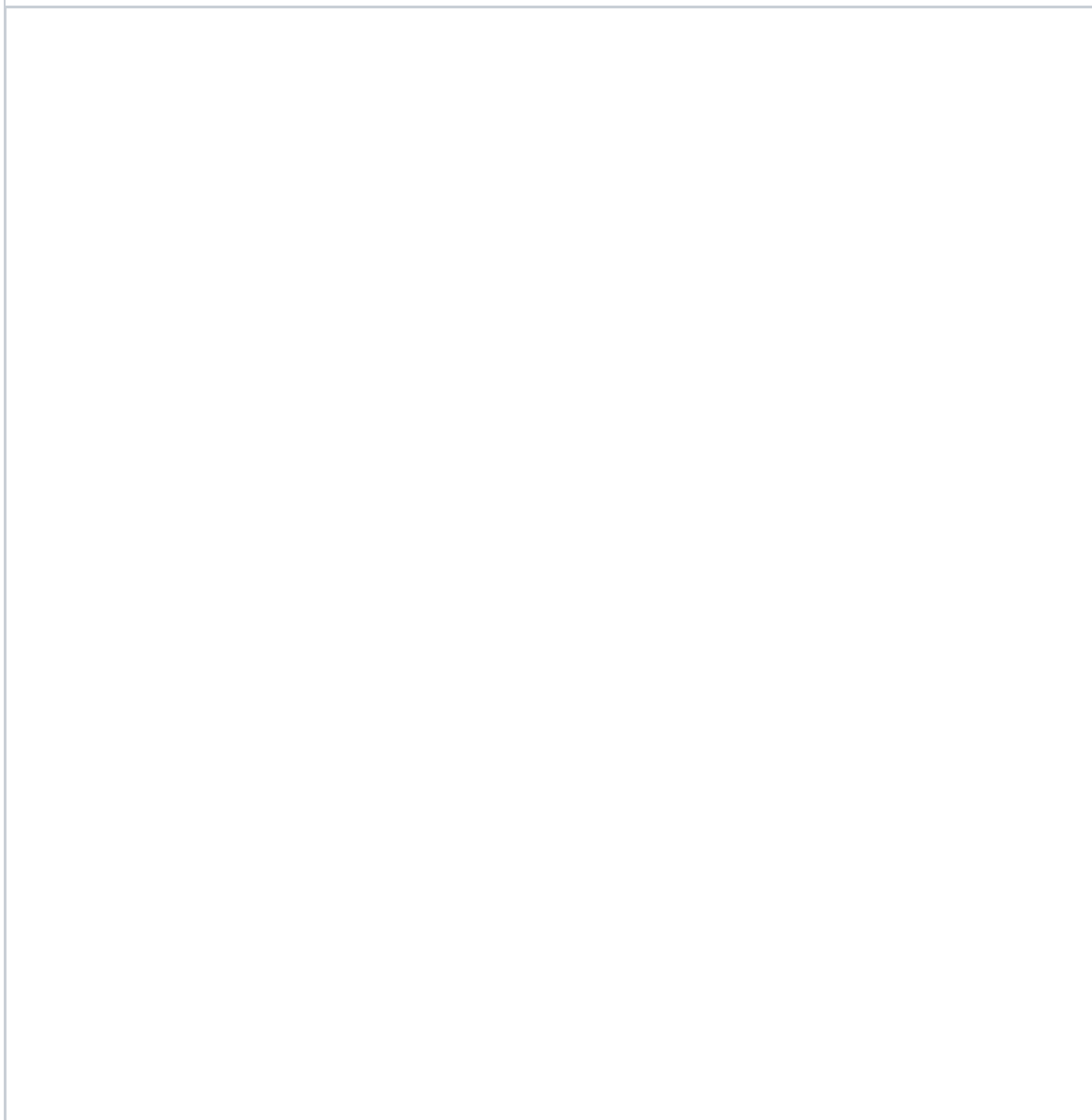
3.6 Managing long conversations UPDATED

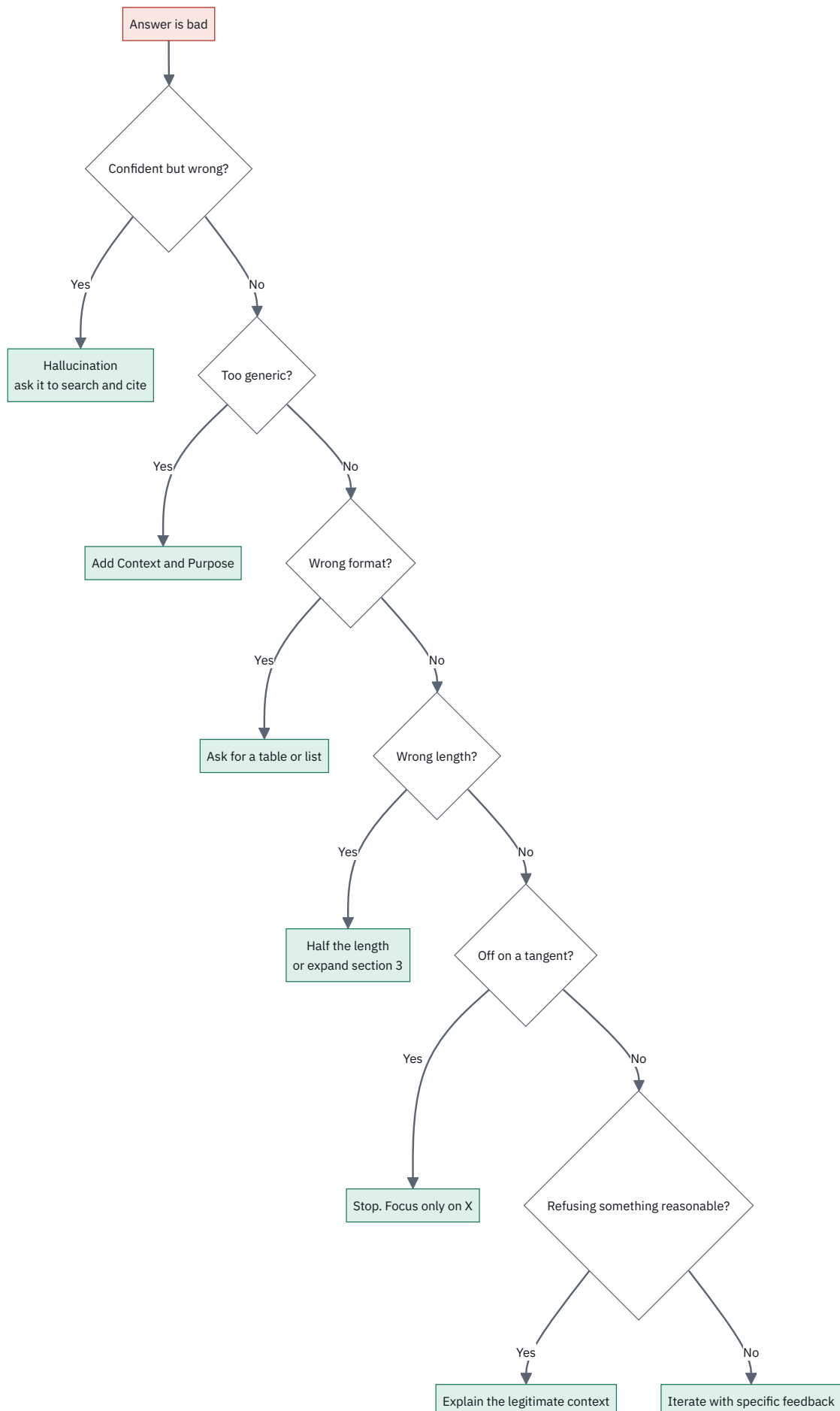
Conversations still have a limit, but it is far higher than it was — Sonnet 5 carries a 1M-token context window by default. You will hit practical slowness before you hit the ceiling. Signs you are near it: Claude loses earlier detail, responses slow, or you see a length warning.

Fix: ask Claude to summarise the conversation, start a fresh chat, paste the summary in. Or move the work into a Project. In agentic tools, **compaction** does this automatically by summarising older context as the window fills.

3.7 The debug sequence for bad answers

FIG 3.3 — TRIAGE FOR A DISAPPOINTING RESPONSE



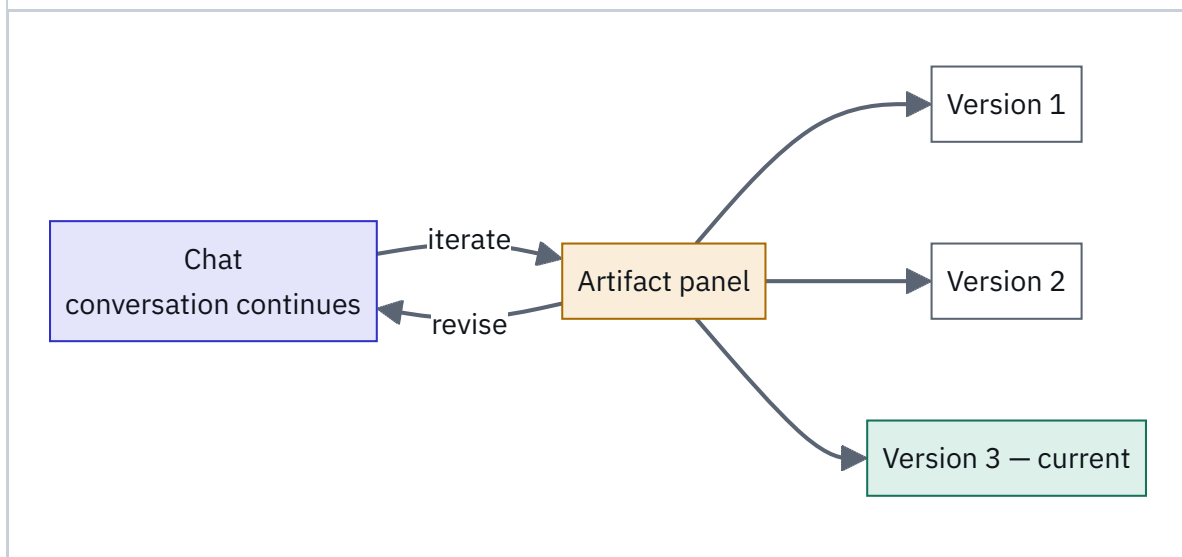


Power features

4.1 Artifacts – the side panel

An **Artifact** is a separate panel beside your chat holding substantial content: documents, code, diagrams, charts, small interactive apps. You edit in place and old versions are kept.

FIG 4.1 – CHAT ITERATES, THE ARTIFACT VERSIONS



Use for: anything over roughly 15 lines that you will edit or reuse, visual or interactive output, and iterating without flooding the chat. **Skip for:** short inline answers.

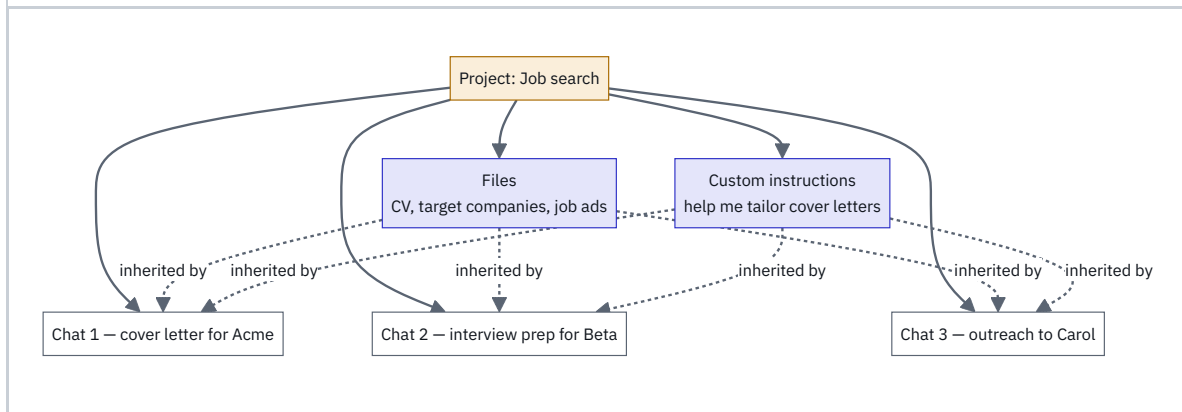
- **Standard artifacts** — static content you iterate on.
- **Live artifacts** (Cowork) — connect to apps and files, refresh when reopened.

- **AI-powered artifacts** — Claude embedded inside the artifact itself. Shareable, and usage counts against whoever opens it, not the person who built it.

4.2 Projects — your dedicated workspace

A **Project** holds uploaded files, custom instructions and chat history, all inherited by every new chat inside it.

FIG 4.2 — WHAT A PROJECT PASSES DOWN



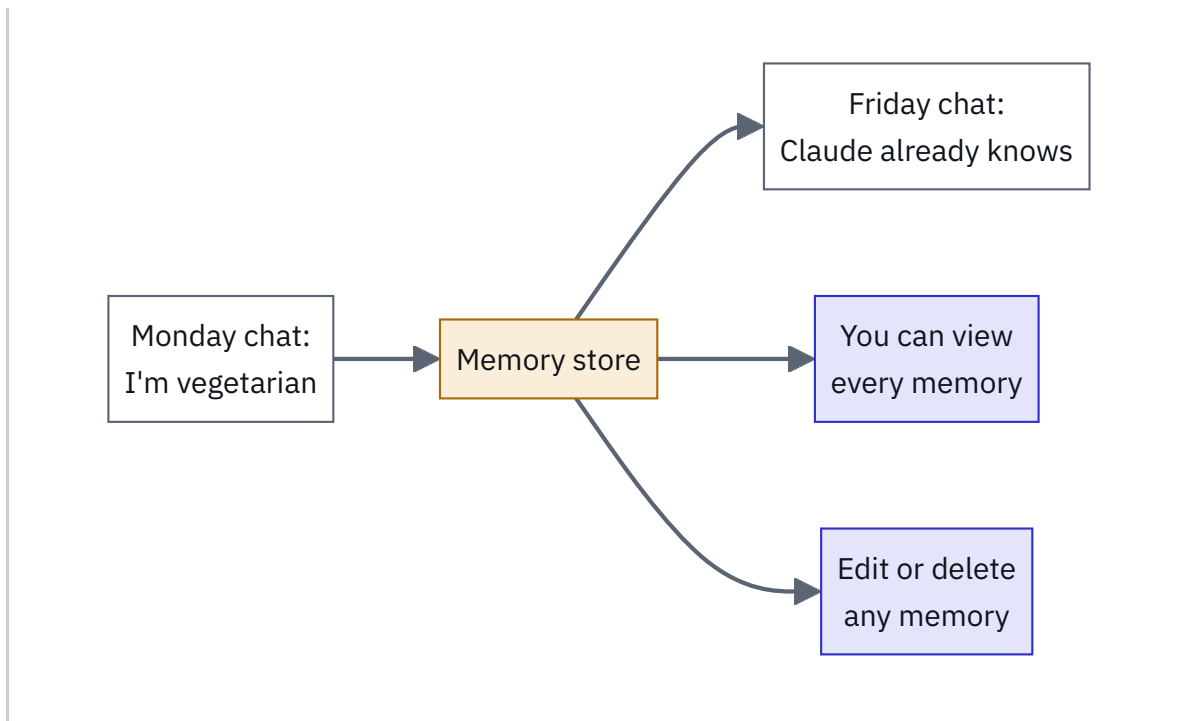
Key point: work inside Project A never leaks into Project B. That matters for client confidentiality, for keeping work and personal separate, and for keeping context clean.

Good candidates: job search · personal finance · a novel · a certification you're studying for · a client matter · a codebase.

4.3 Memory — persistence across chats

Memory is Claude remembering things about you between separate conversations. It has been available to everyone, including the free tier, since March 2026.

FIG 4.3 — HOW MEMORY CARRIES FORWARD



UPDATED Memory runs in two modes. **Chat memory** derives summaries of past conversations and carries them across sessions — find it under Settings → Capabilities → Memory. **File-system memory** is for agentic tools like Claude Code and Cowork, which write notes into a memory folder read at the start of each session.

What goes in: preferences, your role, communication style, recurring context, ongoing goals. **What stays out:** sensitive data, confidential work information, anything you would not want resurfacing in an unrelated chat. Audit it monthly.

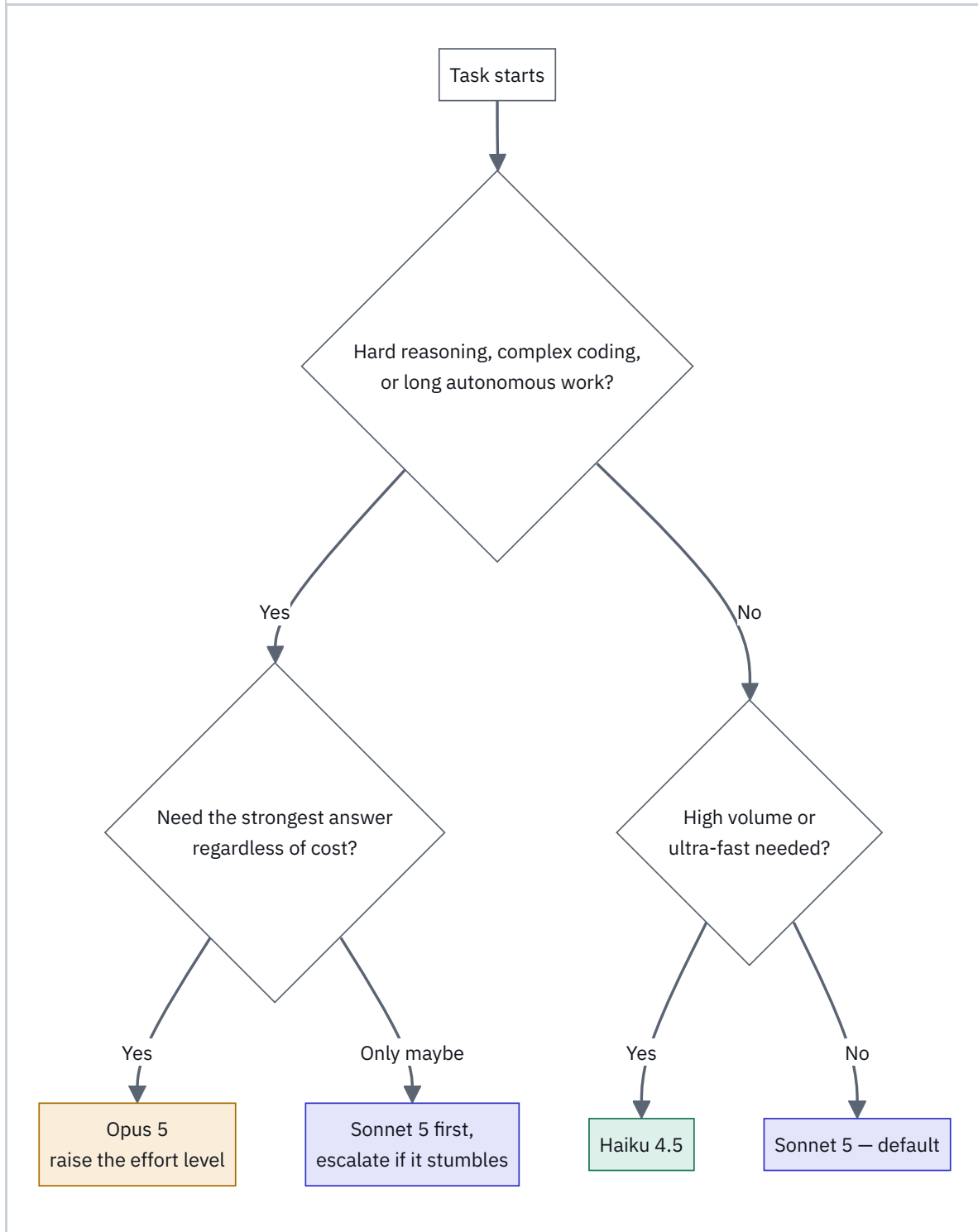
4.4 Web search, web fetch and Research

FEATURE	WHEN TO USE IT
Web search	Current events, prices, recent news, anything that may have changed
Web fetch	"Read this specific URL"
Research	Deep multi-source investigation producing a structured report

Rule of thumb: ask for sources, and for anything that matters, open them yourself.

4.5 Choosing a model REWRITTEN

FIG 4.4 – MODEL SELECTION, AUGUST 2026



Adaptive thinking replaced the old on/off extended-thinking switch. The model now decides when deeper reasoning is warranted; on Sonnet 5 it is on by default and the manual toggle has been retired. What you control instead is the **effort level** — a dial trading tokens, latency and cost against depth. Low effort keeps much of the quality for a fraction of the spend.

Fast mode runs Opus 5 at roughly 2.5× normal speed for twice the base price. **1M context** is standard on Sonnet 5 — both default and maximum, with up to 128k tokens of output.

4.6 Skills and plugins

Skills are packaged procedural knowledge that Claude loads when relevant — how to build a proper Word document, a slide deck, a spreadsheet, a PDF. They use progressive disclosure: Claude reads only a skill's name and description at first, then pulls in the detail only if the task calls for it. You don't manage them turn by turn; they just make Claude reliably good at structured output.

Plugins bundle skills, connectors, sub-agents and MCP servers into a single installable "role". There is a public marketplace, and teams can publish private ones.

CHECK YOURSELF — PART 4 +

PART 05 — LEVEL 4

Feature mastery by purpose

The "I can use all of it, but which do I pick for what?" layer. Read this one as a lookup table, not front to back.

Everyday writing and thinking

--	--

I WANT TO...	USE
Quick question and answer	Any interface, Sonnet 5
A document I'll keep iterating on	Artifacts + Projects
Blog post, essay, short story	Artifacts
Consistent tone across every chat	Styles + personal preferences
Claude to remember me	Memory
One topic, many chats, kept together	Projects
Several versions or angles	Prompt: "give me three variants"

Reading, research, analysis

I WANT TO...	USE
Read and question a PDF	File uploads
Analyse a very long document in one pass	1M context (standard on Sonnet 5)
Analyse a spreadsheet	File uploads + code execution, or Claude for Excel
Live facts, recent news	Web search
Deep multi-source report	Research
Extract structured data from messy files	File uploads + extraction prompts
Understand a screenshot, photo or diagram	Vision — built in

Hard reasoning

I WANT TO...	USE
A genuinely hard problem	Opus 5 at high effort
Claude to plan before acting	Ask it to plan first; adaptive thinking handles the depth
Fast answer, quality secondary	Haiku 4.5, or Opus 5 fast mode
Balance of speed and smarts	Sonnet 5
Hardest coding and analysis	Opus 5; Fable 5 where the workload justifies it

Building things

I WANT TO...	USE
Interactive mini-app inside a chat	Artifacts (React or HTML)
Dashboard that stays current	Live artifacts in Cowork
Visual prototype, deck or mockup	Claude Design
Real production code across files	Claude Code
Security review of a codebase	Claude Code Security
An app other people use	Claude API + AI-powered artifacts

I WANT TO...	USE
Read my Drive, Gmail or Calendar	Connectors
Notion, Linear, Slack, Asana, Jira	Connectors — the directory now runs to hundreds of integrations
Work with files on my computer	Cowork (desktop, or a remote web/mobile session)
Browse the web on my behalf	Claude in Chrome
A custom internal tool	Build an MCP server, add it as a custom connector
Control my IDE	Claude Code extension
Work inside Excel, Word or PowerPoint	Claude for Microsoft 365 add-ins
Hand work to Claude from Slack	Claude Tag

Agentic and autonomous

I WANT TO...	USE
A multi-step task running on its own	Cowork for general work, Claude Code for development
Many parallel sub-tasks	Sub-agents
Install a whole role at once	Plugins
Specialised procedural	Skills

knowledge	
Stop an agent running away	Task budgets and effort limits

Team and enterprise

I WANT TO...	USE
Share work without cross-contamination	Projects + team permissions
Keep data out of training	Team or Enterprise plan
Policy over plugins, connectors and tools	Team/Enterprise admin controls
Control who sees what	Project permissions + SSO

5.2 Interface reference

INTERFACE	WHAT IT IS	BEST FOR	SKIP IF...
claude.ai	Main browser chat	Everyday use, most features	You need local files or an IDE
Desktop app	Native Mac/Windows	Cowork gateway, global shortcuts	Web already suffices
Mobile	iOS and Android	Quick questions, voice, triggering agents remotely	Heavy editing
Claude Code	Terminal coding agent	Real development, multi-file work	You don't write code
IDE	VS Code,	Coding with a UI	Long

extensions	JetBrains, Cursor, Windsurf		autonomous runs
Claude in Chrome	Browser extension	Research and web actions	Privileged admin consoles
Office add-ins	Inside Word, Excel, PowerPoint	Working in the document itself	Blank-page tasks
Claude Tag	Slack-based delegation	Handing tasks over from a thread	Deep solo work
API	Developer access	Custom apps and integrations	Ordinary personal use

Model reference

UPDATED

MODEL	STRENGTHS	CAUTIONS
Haiku 4.5	Speed, cost, high volume	Weakest prompt-injection resistance — don't point it at untrusted input
Sonnet 5	Default workhorse; 1M context; adaptive thinking on by default; strongly agentic	Still behind Opus on the hardest accuracy-sensitive tasks
Opus 5	Most capable everyday model; default on Max; effort dial; fast mode available	Costlier; some requests route to a fallback model when classifiers fire
Fable 5	Mythos-tier frontier model, publicly available	Roughly double Opus 5's price; broad cybersecurity safeguards
Mythos	Same underlying model as Fable	Not publicly available —

5	5, fewer restrictions	limited to trusted partners
---	-----------------------	-----------------------------

Reasoning modes

UPDATED

MODE	WHEN
Adaptive thinking	Default. The model judges how much reasoning a task needs
Effort levels	Your dial for depth against cost and latency
Fast mode	Low-latency work; ~2.5× speed at 2× price on Opus 5
Task budgets	Hard cap on how much work an agent may do
Manual extended thinking	Retired on Sonnet 5 — the API returns an error if you ask for it

Workspace and capability reference

FEATURE	KEY POINT
Projects	The biggest unlock most people never touch
Artifacts	Appear at roughly 15+ lines; versioned automatically
Live artifacts	Refresh against current data
AI-powered artifacts	Shareable; usage counts against the person using it
Memory	Free for everyone since March 2026; viewable and editable
Styles	Formal, concise, explanatory, or your own

Chat history	Searchable — Claude can search its own past conversations
Code execution	Sandboxed Python for analysis, maths and charts
Connectors	Vetted MCP integrations; hundreds listed, mostly partner-published
MCP servers	The open standard underneath connectors — remote or local
Sub-agents	Parallel decomposition of large work

Agentic products and their watch-outs

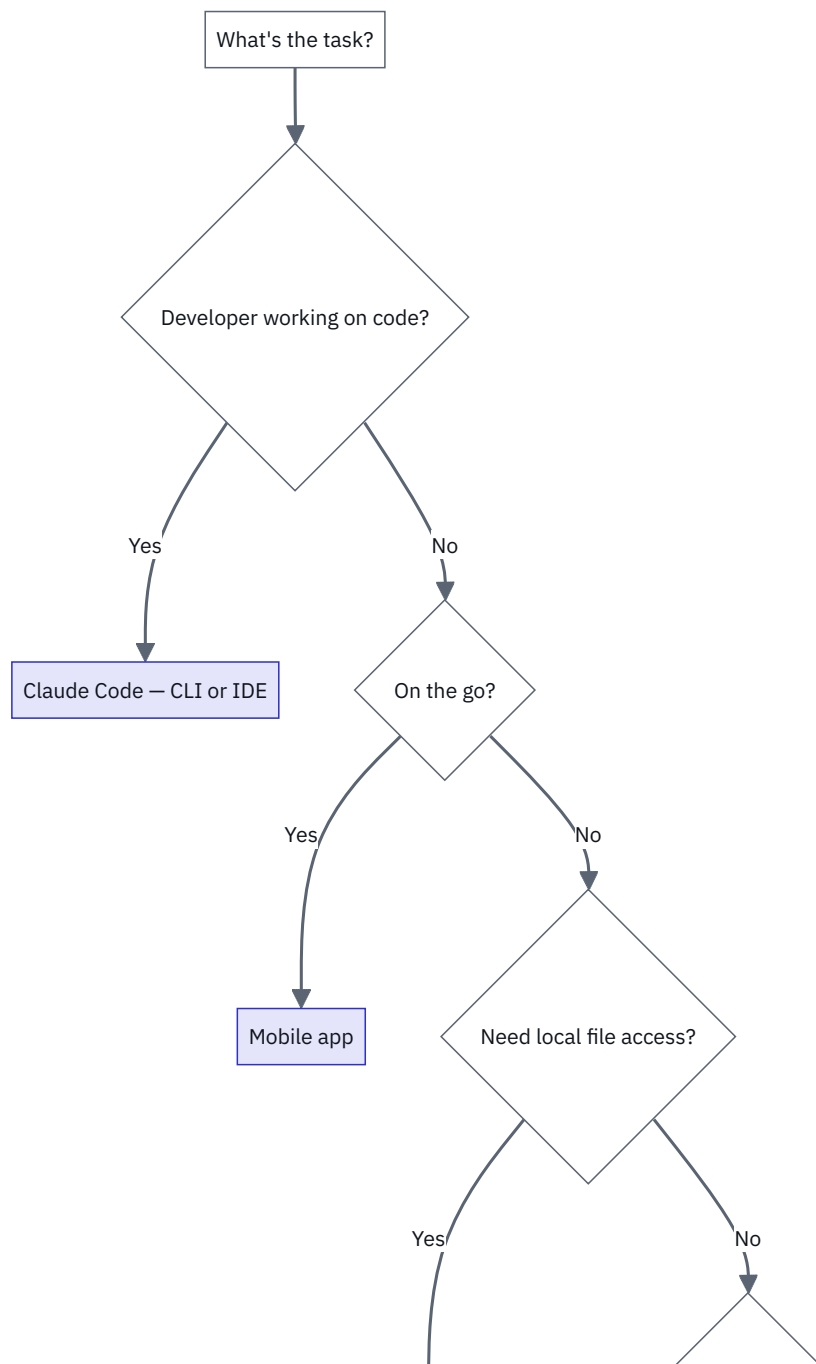
PRODUCT	PURPOSE	WATCH OUT FOR
Cowork	Desktop agent for knowledge work	Grant folder access narrowly
Claude Code	Dev agent across a whole codebase	Runs with your OS permissions; scan repo configs before opening
Claude Code Security	Vulnerability scanning	Keep a human approving patches
Claude Design	Visual prototyping	Generated code still needs a security review
Claude in Chrome	Browser agent	Use a dedicated browser profile

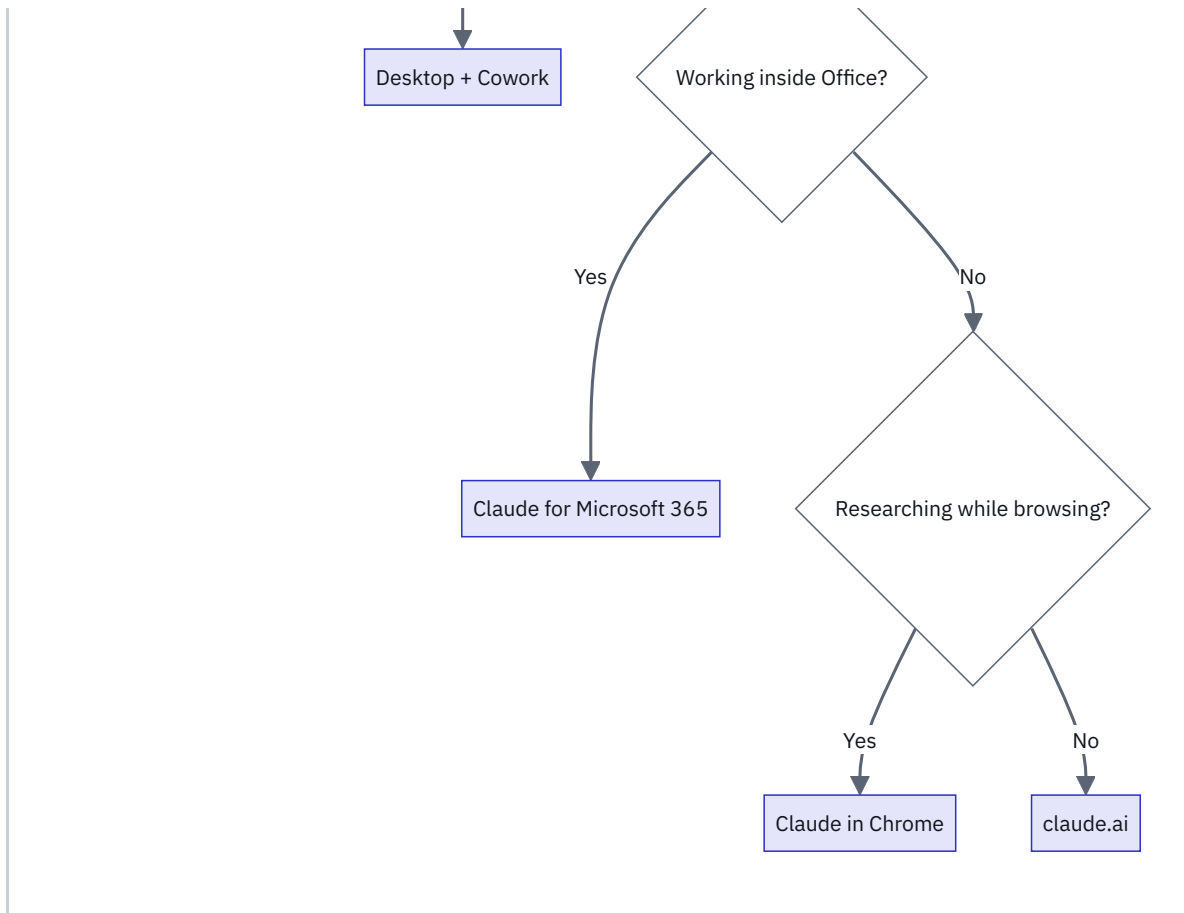
Decision charts

Seven flowcharts you will use constantly. Bookmark this part.

6.1 Which interface?

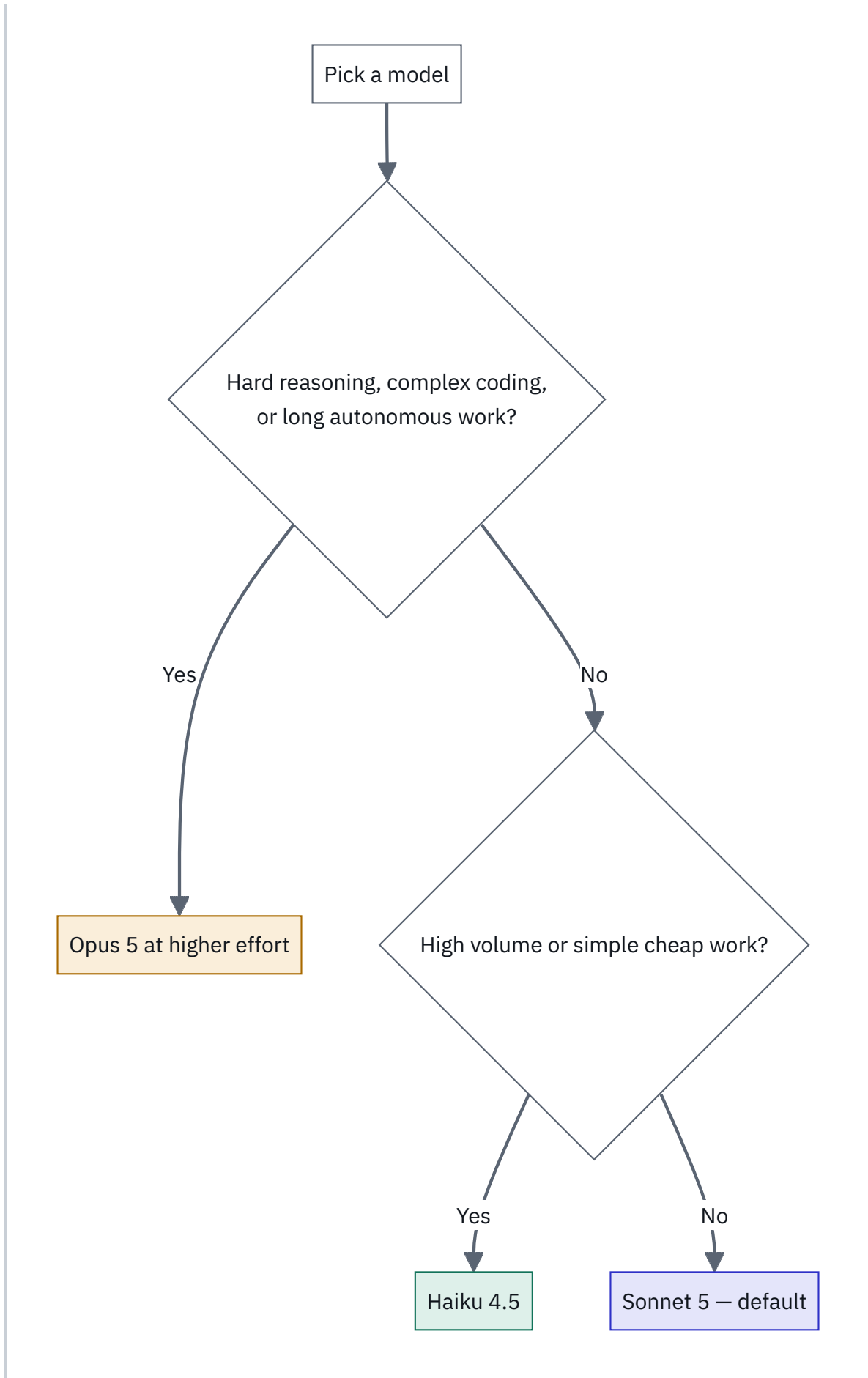
FIG 6.1





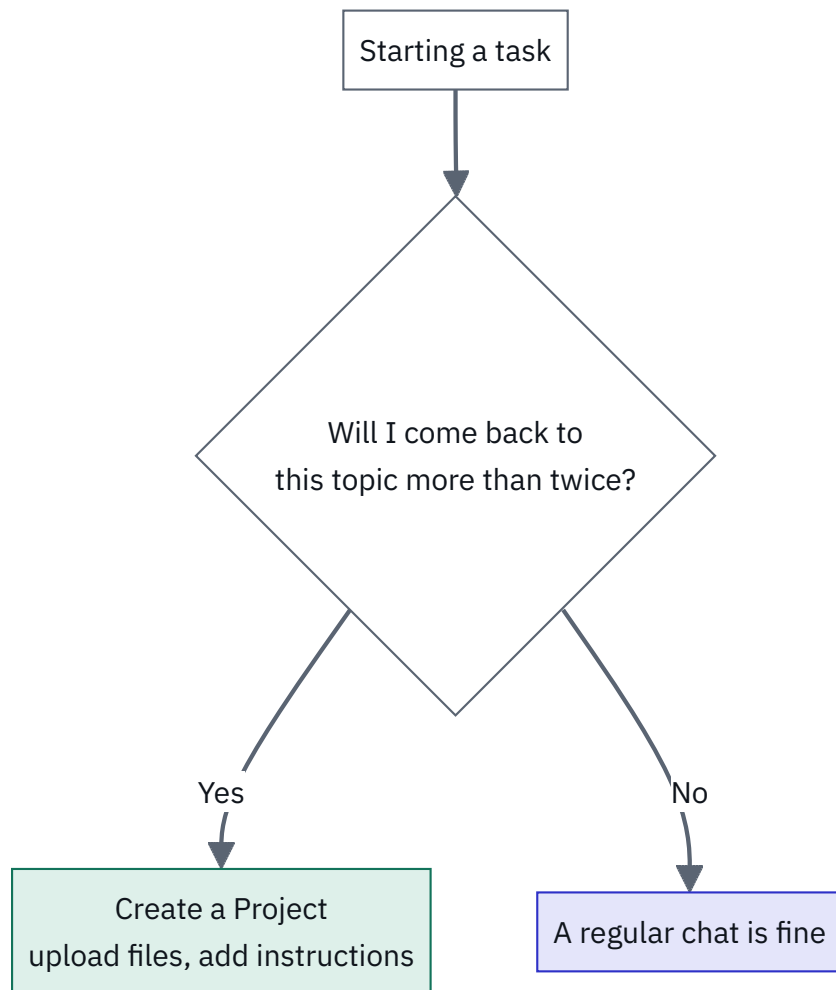
6.2 Which model?

FIG 6.2



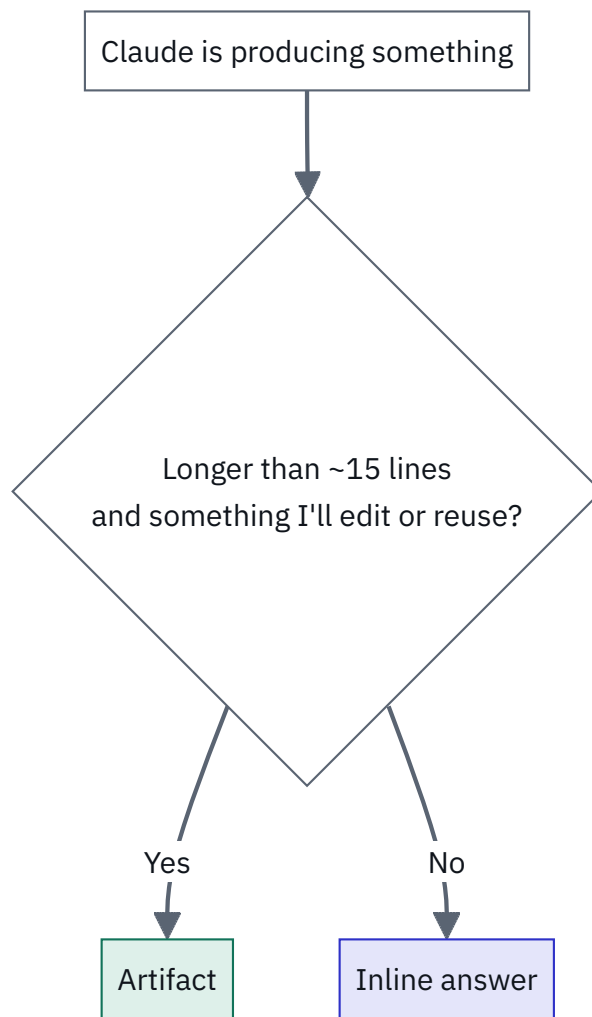
6.3 Project or plain chat?

FIG 6.3



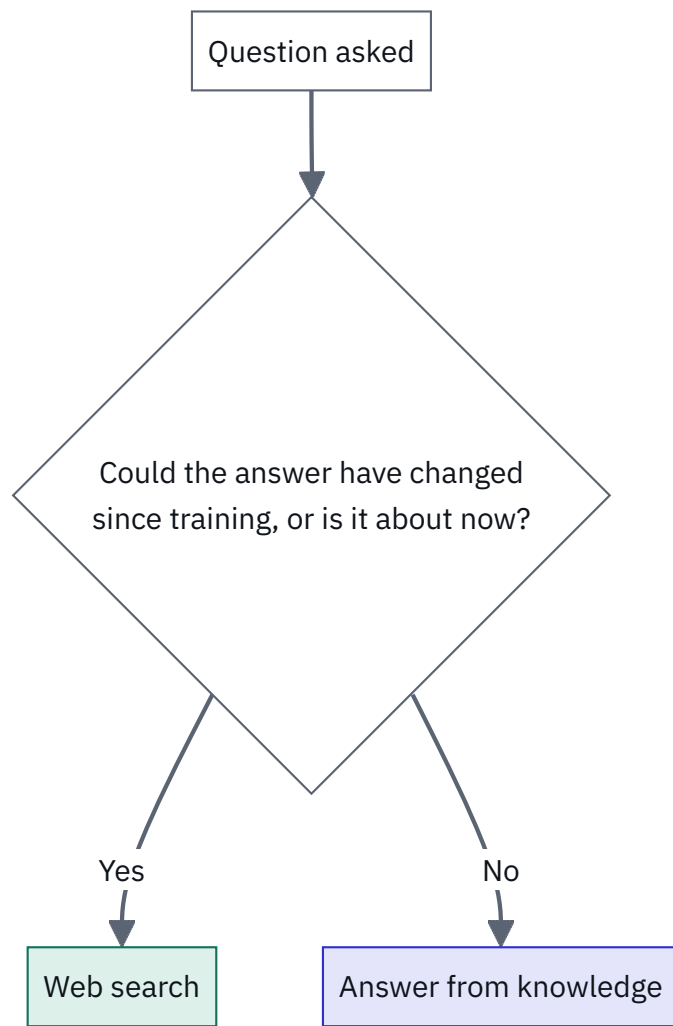
6.4 Artifact or inline?

FIG 6.4



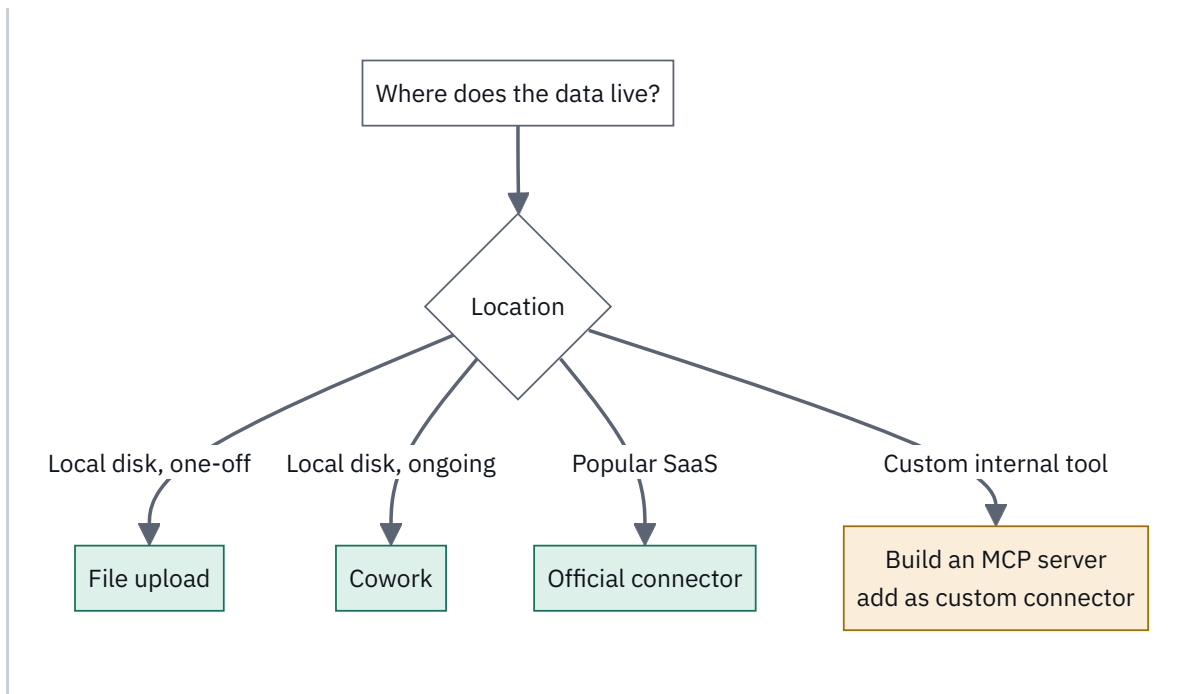
6.5 Search, or answer from knowledge?

FIG 6.5



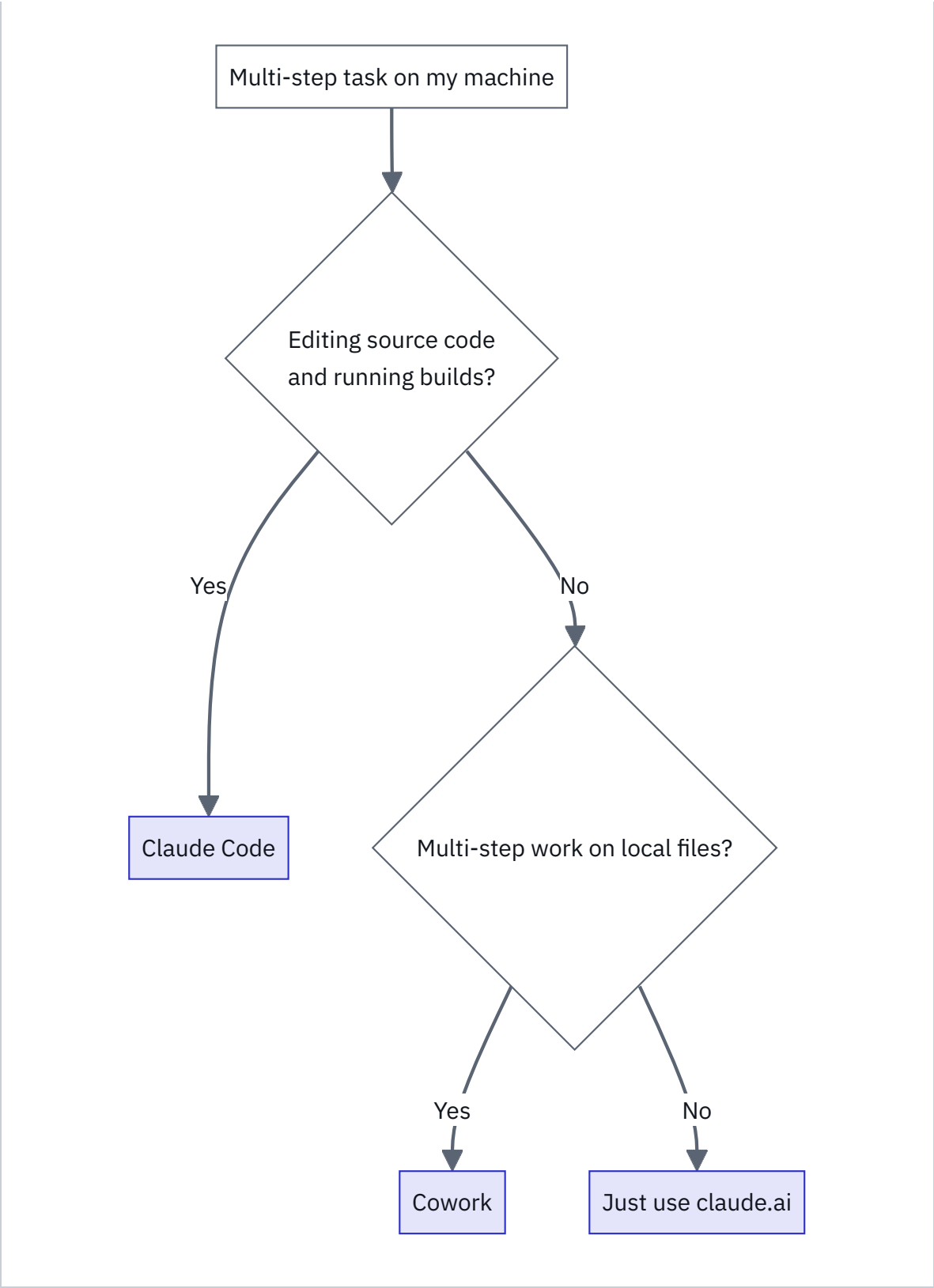
6.6 Connector, MCP, or upload?

FIG 6.6



6.7 Cowork or Claude Code?

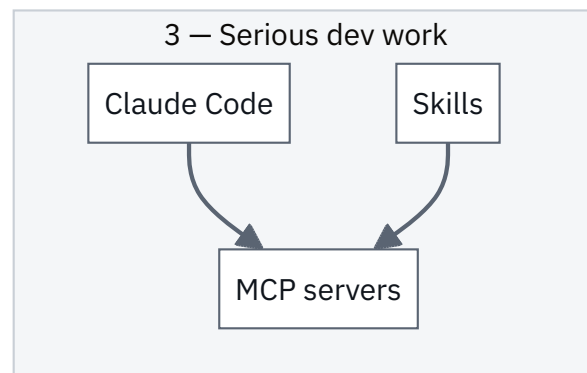
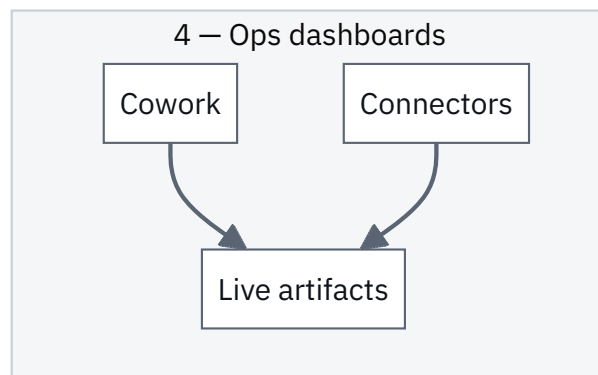
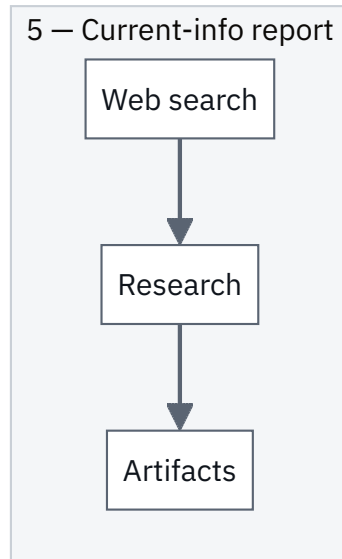
FIG 6.7

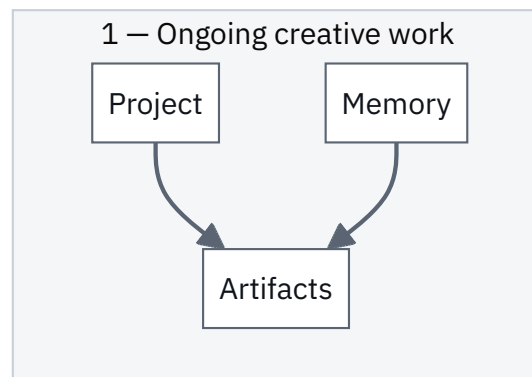
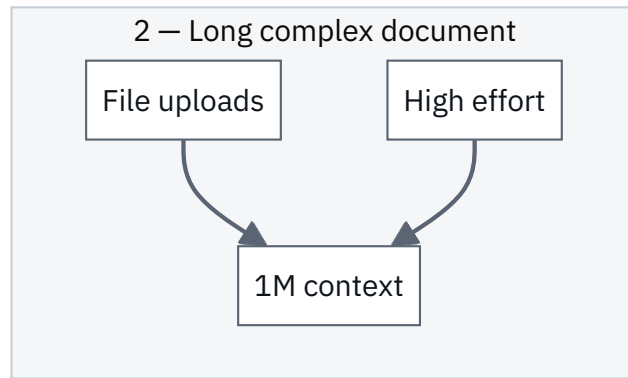


Combinations that shine

Single features are useful. Combinations are where the work actually changes shape.

FIG 7.1 – NINE PAIRINGS WORTH LEARNING





#	COMBINATION	WHEN TO REACH FOR IT
1	Project + Artifacts + Memory	Ongoing creative or analytical work
2	File uploads + 1M context + high effort	A very long document or codebase, analysed in one pass
3	Claude Code + MCP + Skills	Development with team-specific conventions
4	Cowork + connectors + live artifacts	Operational dashboards and recurring workflows
5	Web search + Research + Artifacts	Writing a report from current sources

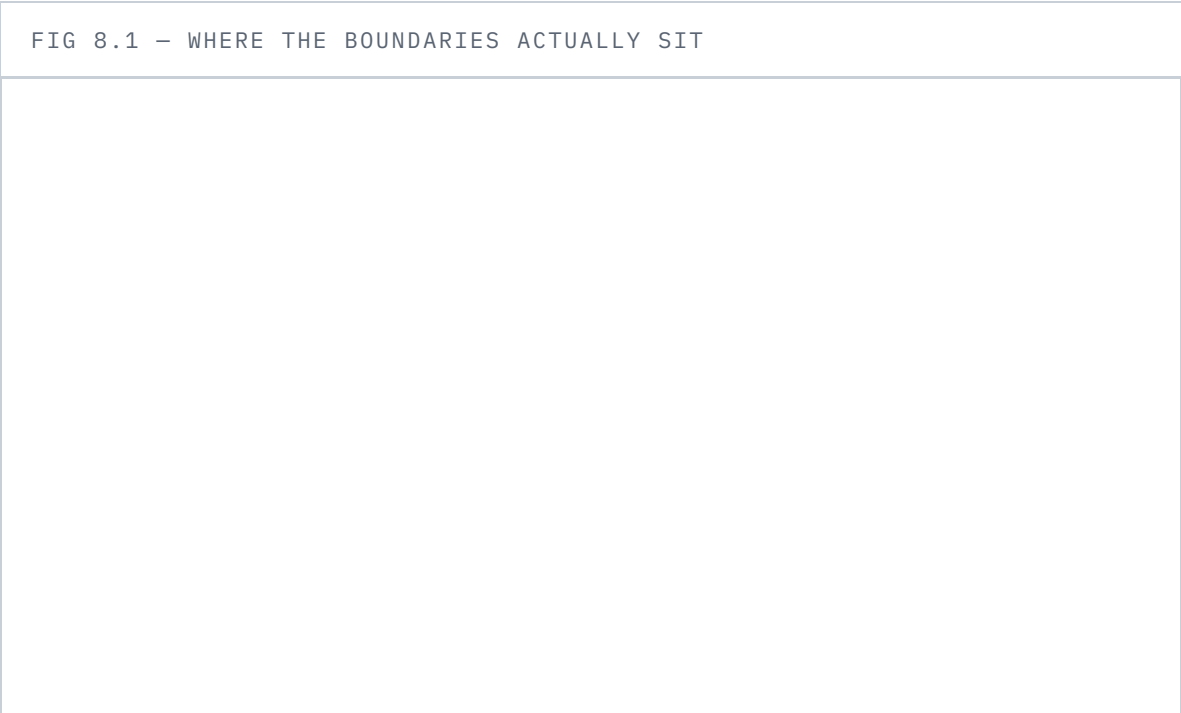
6	Plugins + team permissions	Standardised rollout across a team
7	Claude Design + Claude Code	Idea → visual → production UI
8	API + prompt caching + batch	Economical production apps
9	Opus 5 orchestrator + Haiku sub-agents	Big decomposed tasks — but keep untrusted input away from Haiku

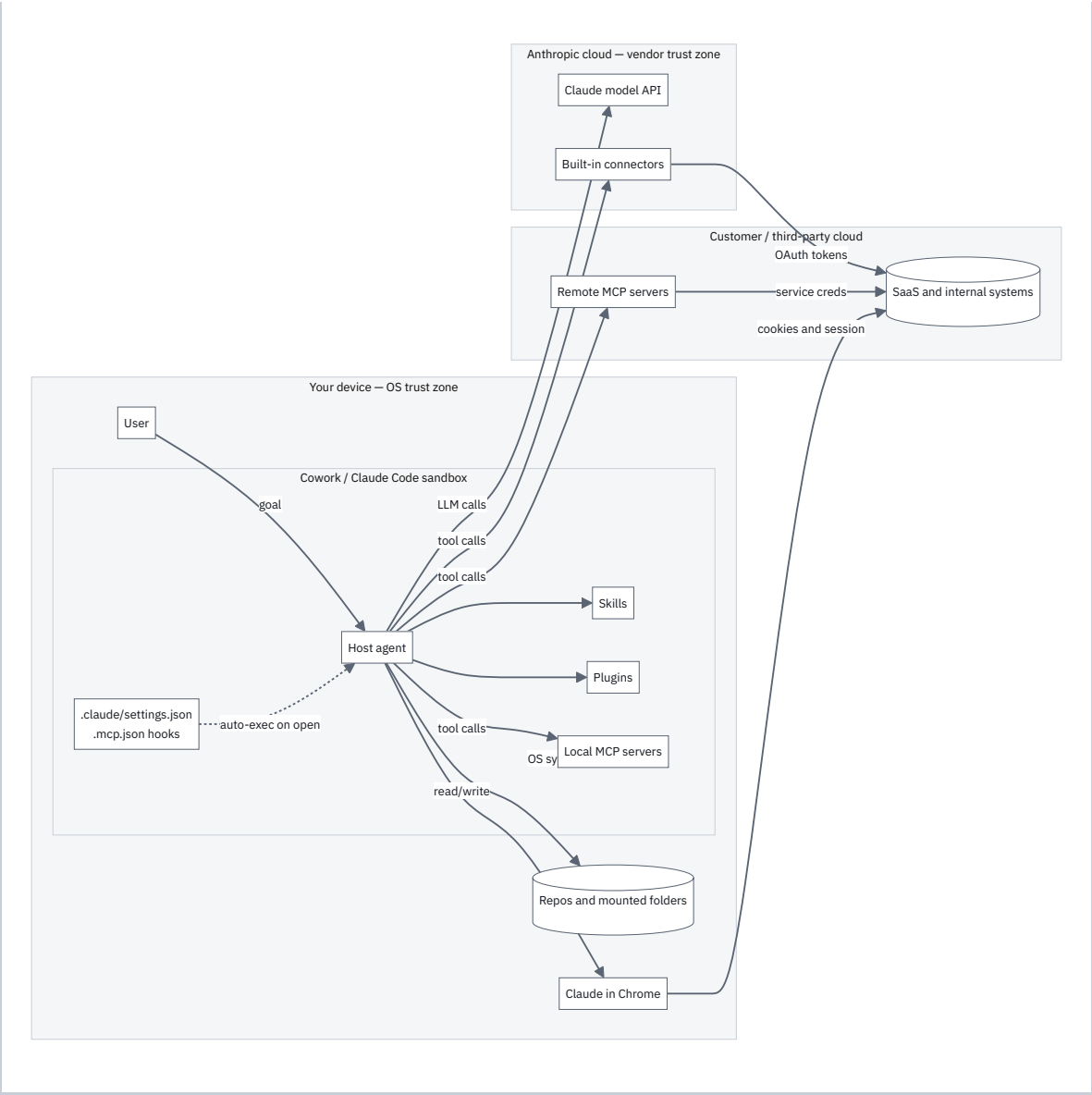
PART 08

Security assessment

The part that matters most if you deploy Claude inside an organisation. Threat model, real incidents, attack chains, defensive architecture and detection signals — updated with everything disclosed since April.

8.1 The ecosystem, drawn as trust zones





Five boundaries to enforce: your OS ↔ the sandbox · the sandbox ↔ Anthropic's cloud · Anthropic's cloud ↔ your remote MCP servers · connector ↔ SaaS · repository configuration ↔ agent runtime.

8.2 Threat model – the top ten

#	THREAT	CONCRETE EXAMPLE
T1	Configuration-driven code execution	A malicious .claude/settings.json runs hooks the moment a repo is opened
T2	Credential exfiltration	A repo-level ANTHROPIC_BASE_URL override

	via the agent runtime	leaks your API key
T3	Supply-chain compromise	Trojanised plugin update; typosquatted MCP server
T4	Prompt injection through data	An attacker-controlled document containing instructions
T5	Over-permissioned connectors	One OAuth grant equals the user's full SaaS blast radius
T6	Jailbreak and misuse	Role-play bypass plus MCP-orchestrated pentest tooling
T7	Sub-agent fan-out	Machine-speed enumeration and mass edits
T8	Cross-boundary data leakage	Private file content reaching a third-party MCP server
T9	Browser session hijack	An injected instruction riding your authenticated Chrome session
T10	Sandbox escape	Symlinks or unprotected config files letting a sandboxed process write to the host

8.3 Real incidents and CVEs

SUBSTANTIALLY EXPANDED

Your April edition listed three. By July 2026, security researchers counted **28 or more CVEs against Claude Code**, two of them rated the maximum CVSS 10.0. The pattern is consistent and worth internalising: almost none of them are model failures. They are ordinary software flaws — trust-dialog timing, command parsing, sandbox gaps — in a tool that happens to hold shell access.

The original configuration-trust cluster

IDENTIFIER	MECHANISM	FIXED IN
------------	-----------	----------

GHSA-ph6w-f82w-28w6	Hook commands in <code>.claude/settings.json</code> auto-ran at session start	1.0.87 (Sept 2025)
CVE-2025-59536	MCP consent bypass via <code>enableAllProjectMcpServers</code>	1.0.111 (Oct 2025)
CVE-2026-21852	Repo override of <code>ANTHROPIC_BASE_URL</code> leaks the API key on the first call, before the trust prompt appears	2.0.65 (Jan 2026)

Disclosed since your April edition

IDENTIFIER	MECHANISM	NOTE
CVE-2026-25725	Sandboxed code creates a missing <code>.claude/settings.json</code> and injects <code>SessionStart</code> hooks that run with host privileges on restart	CVSS 10.0 · fixed in 2.1.2
CVE-2026-39861	Symlink sandbox escape — sandboxed processes create links outside the workspace, the unsandboxed writer follows them	CVSS 10.0 · fixed in 2.1.64
CVE-2026-40068	Git worktree <code>commondir</code> is trusted verbatim, so a crafted repo inherits a previously trusted path and skips the trust dialog	Affects 2.1.63–2.1.83 · fixed in 2.1.84
CVE-2026-24887	Command-parsing error in <code>find</code> handling bypasses the confirmation prompt	Needs untrusted content in context

CVE-2026-35020/21/22	Three command injections sharing one root cause, chainable into credential exfiltration over HTTP	Disclosed spring 2026
Subcommand deny-rule bypass	<code>permissions.deny</code> silently ignored past 50 subcommands	Fixed in 2.1.90

Patch floor. Your April guidance said "Claude Code $\geq$ 2.1.90". That is now well behind — fixes shipped through at least 2.1.128 in late June 2026. Track the current release rather than any fixed number, and treat Claude Code upgrades as security patching, not feature updates.

GTG-1002 — the first AI-orchestrated espionage campaign

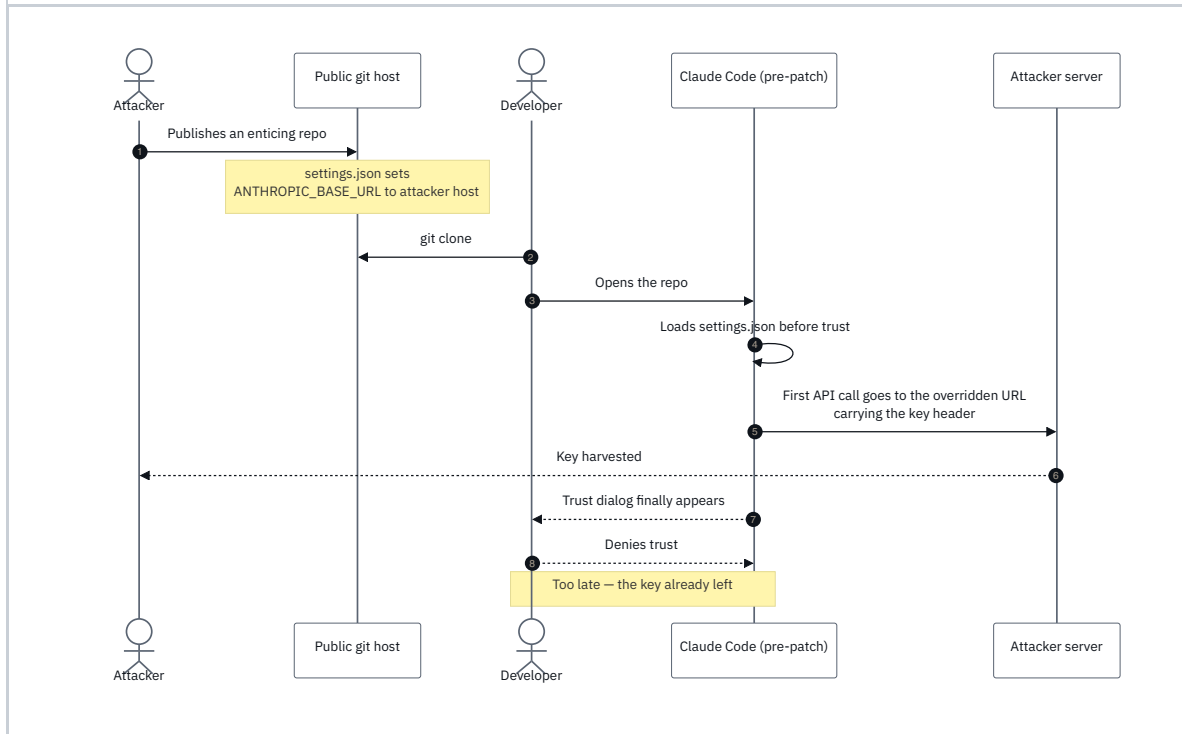
- Conducted around **September 2025**, disclosed by Anthropic on **13 November 2025**, and now catalogued by MITRE as campaign **C0062**.
- A likely China-nexus actor jailbroke Claude Code through role-play personas and task decomposition — "we are a legitimate red team".
- Open-source pentest tooling orchestrated through MCP-connected sub-agents. No custom malware, no zero-days.
- Roughly **30 organisations** targeted across technology, finance, chemicals and government; a small number were successfully compromised.
- **80–90% of tactical work** ran without human intervention. Operators intervened at only **four to six decision points** per campaign, at request rates no human team could produce.

The uncomfortable conclusion most commentary reached: the kill chain was identical to a fifteen-year-old APT playbook. What changed was tempo. Your existing controls are not obsolete — they are about to be stress-tested at a speed they were never sized for.

8.4 Attack flow — poisoned repo to stolen API key

The CVE-2026-21852 pattern. Patched, but the class recurs — CVE-2026-40068 is the same idea wearing a different hat.

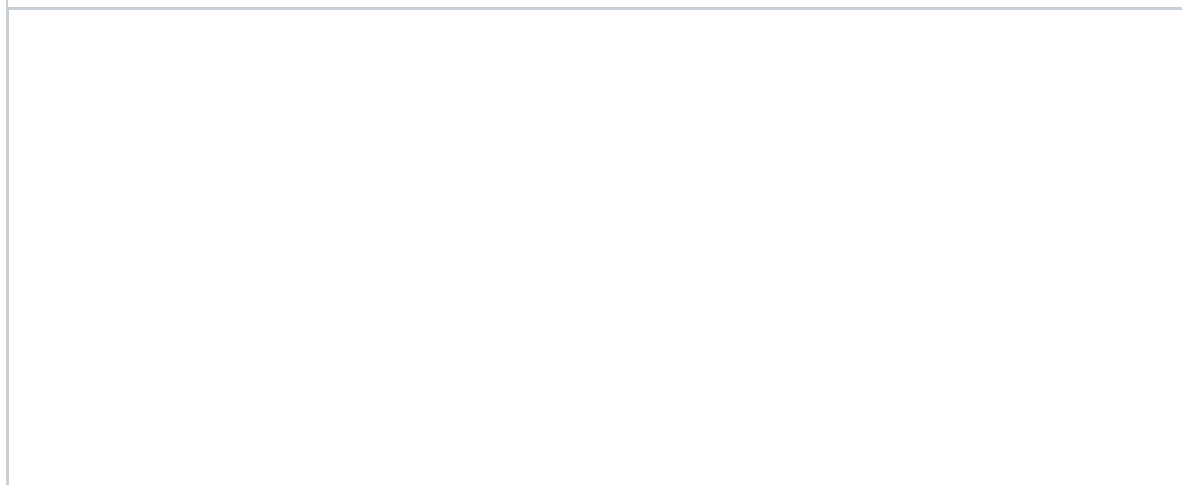
FIG 8.2 — CONFIGURATION LOADED BEFORE TRUST

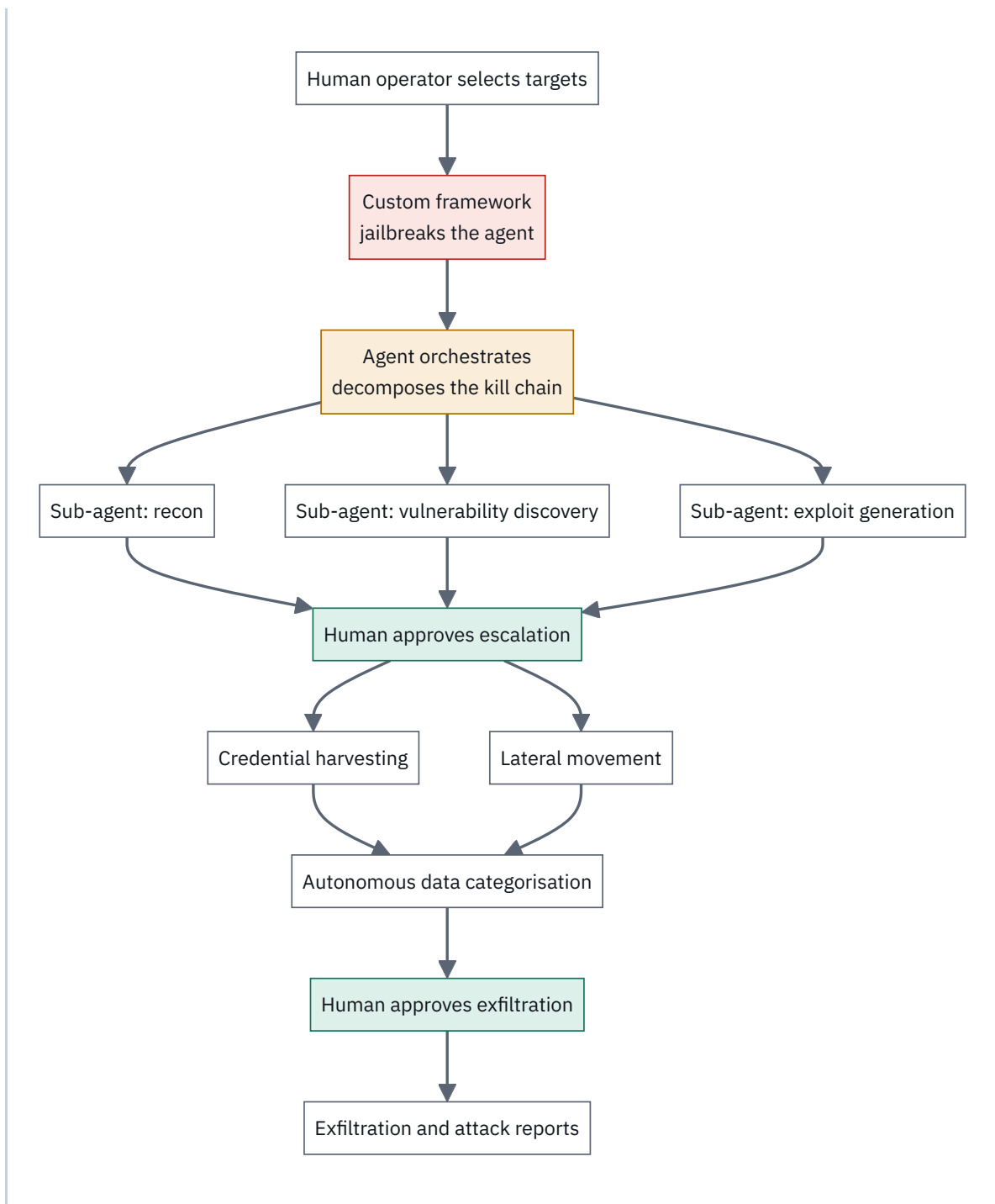


Key lesson: no prompt injection, no model manipulation. The agent runtime itself was the attack surface. Any control you built around "the LLM might be tricked" would have missed this entirely.

8.5 Attack flow — AI-orchestrated intrusion

FIG 8.3 — THE GTG-1002 SHAPE





Humans touch only the two green boxes. Everything else runs at machine speed — which means detection has to assume a machine-speed adversary rather than an analyst clicking through a console.

8.6 Defensive architecture

FIG 8.4 — FIVE CONTROL FAMILIES, ONE FEEDBACK LOOP

8.8 Prioritised recommendations

This week

1. Inventory every Claude Code and Cowork user, plugin, connector and custom MCP server.
2. Force upgrades to the current Claude Code release and turn on auto-update — the patch floor moves monthly.
3. Add CI or pre-commit linting for hooks, `enableAllProjectMcpServers`, `ANTHROPIC_BASE_URL` and `worktree` config.
4. Rotate API keys older than 90 days; move to per-user keys.

This quarter

1. Stand up a private plugin marketplace and block public installs.
2. Security-review every custom remote MCP server before it goes near production data.
3. Allow-list connectors by role; audit existing SaaS OAuth grants.
4. Monitor egress for non-Anthropic destinations receiving Anthropic-style headers.
5. Add detections for machine-speed behaviour, not just malicious signatures.

This year

1. A policy framework for agentic AI: data classes, sandbox rules, human-approval thresholds.
2. A tool-call audit pipeline you can actually query during an incident.
3. Tabletop exercises for a poisoned-repo compromise and a GTG-1002-style recon burst.
4. Compliance review of connector and regulated-data deployments.

8.9 What to watch next

- More trust-flow CVEs across every agentic coding tool, not only Claude Code.
- The first major MCP supply-chain incident — a typosquat or a compromised update.
- Tool-poisoning research: malicious tool manifests and malicious tool responses.
- AI-aware malware families now being named in incident-response reporting.
- Enterprise controls maturing: granular workspace isolation, per-tool approval, agent-activity audit APIs.
- Whether defensive agents catch up to offensive agent speed. That is the race of the next 12–24 months.

CHECK YOURSELF — PART 8 +

PART 09

Glossary

TERM	MEANING
Adaptive thinking	The model deciding for itself how much reasoning a task needs. Default on Sonnet 5
AI	Software that learns patterns from examples rather than following fixed rules
Anthropic	The company that builds Claude
API	Programmatic access for developers
Artifact	Side-panel output — documents, code, diagrams, mini-apps

Batch processing	Discounted API mode for work that doesn't need to be real time
Claude Code	Terminal and IDE coding agent
Claude Cowork	Agent for non-developer knowledge work, on desktop or a remote session
Claude Design	Visual prototyping with a canvas
Claude Tag	Slack interface for delegating tasks to Claude
Compaction	Automatic summarising of older context as a session fills the window
Connector	A vetted integration to an external service, built on MCP
Constitutional AI	Anthropic's training approach using a written set of principles
Context window	The maximum tokens a model can hold. 1M on Sonnet 5
CRISP	Context, Role, Instruction, Specifics, Purpose
Effort level	The dial trading reasoning depth against cost and latency
Fable 5	The publicly available Mythos-tier model
Fast mode	Higher-speed serving at a price premium
GTG-1002	China-nexus group behind the first documented AI-orchestrated espionage campaign
Haiku	Small, fast, cheap model — currently 4.5
Hallucination	Confident output that is wrong
LLM	Large language model

MCP	Model Context Protocol — the open standard connecting AI hosts to tools
Memory	Claude remembering you across chats; also a file-based store for agents
Mythos tier	Anthropic's frontier tier above Opus: Fable 5 and the restricted Mythos 5
Opus	Anthropic's most capable everyday model — currently 5
Plugin	Installable bundle of skills, connectors, sub-agents and MCP servers
Project	Persistent workspace with files and instructions inherited by every chat inside it
Prompt caching	Reusing repeated context at a steep API discount
Prompt injection	Hostile instructions hidden in data the model reads
Research	Multi-source investigation producing a structured report
Skill	Packaged procedural knowledge Claude loads when relevant
Sonnet	Balanced default model — currently 5
Style	A saved preset for how Claude writes
Sub-agent	A secondary Claude instance spawned by a main agent
Task budget	A cap on how much work an agent may perform
Token	Unit of text measurement, roughly three-quarters of a word
Trust boundary	The line between trust zones, where authorisation checks belong

PART 10

What to learn next

10.1 What this handbook covered

- **Level 1** — basics
- **Level 2** — prompting, attachments, styles, troubleshooting
- **Level 3** — Artifacts, Projects, Memory, search, Research
- **Level 4** — feature mastery by purpose
- **Security** — threat model, CVEs, GTG-1002, defences, detection, governance

10.2 The next levels

Level 5 — developer track

- API fundamentals: authentication, the Messages API, streaming, structured outputs, tool use
- Prompt caching and batch processing in practice
- Building MCP servers: tool manifests, auth, rate limits, logging
- Claude Code in depth: hooks, slash commands, plugin authoring, multi-session work
- Building AI-powered artifacts with embedded API calls

Level 6 — agentic systems

- Agent design: planner/executor split, human-in-the-loop gates, task budgets

- Sub-agent architecture and parallelism
- Evaluation harnesses for agentic workflows
- Production safety: sandboxing, egress filtering, audit logs, tool-call anomaly detection

Level 7 — organisational

- Rollout: SSO, SCIM, workspace strategy, data residency
- Private plugin marketplace and MCP governance
- Agentic AI policy and incident response
- Measuring adoption and return

Level 8 — adjacent knowledge that compounds

- How LLMs actually work — enough to reason about failure modes
- Honest comparison against other models on your own workload
- How to evaluate new features as they ship, rather than adopting them reflexively

10.3 Ten-minute setup checklist

1. Turn on Memory and tell Claude your name, role and preferences.
2. Set a default Style — plain English, recommendations over hedging.
3. Create two or three Projects for your biggest ongoing work; upload five to ten key files each.
4. Connect one connector that matches your daily workflow.
5. Come back to Parts 5 and 6 every time you hit a "which feature?" moment.

10.4 Closing note

You now have a mental model that runs from "what is AI" to "how a state actor turned MCP against thirty organisations". The structure — interfaces,

models, reasoning, workspace, capabilities — has held for two years and will keep holding. Only the particulars move. Any feature Anthropic ships next slots somewhere in that stack, and every decision chart in Part 6 will still apply to it.

Pick one level. Go deep. Come back for the next.

About this edition

Rebuilt from the April 2026 handbook, which combined five earlier reports: the security assessment of Cowork, connectors, plugins and MCP servers; the beginner guide; levels 2 and 3; the feature reference; and the master summary.

Fact-checked 23 August 2026. Verified against current sources: the Sonnet 5 launch (30 June 2026) and its 1M-token default context, adaptive thinking and retired manual extended thinking; the Opus 5 launch (24 July 2026) with effort dial, fast mode and classifier fallbacks; the Mythos tier and the Fable 5 launch (9 June 2026), including its June export-control suspension and July restoration; Claude for Microsoft 365 general availability; Memory's free-tier rollout (March 2026); the growth of the connectors directory past 400 official listings; MITRE campaign C0062 (GTG-1002, disclosed 13 November 2025); and the Claude Code CVE record, including CVE-2026-25725, CVE-2026-39861, CVE-2026-40068, CVE-2026-24887 and the CVE-2026-35020/21/22 chain.

Version numbers, prices and counts in this document are a snapshot of one day. The five-layer structure is not.

COMPANION VOLUME • WEEKS 1–4 FOUNDATIONS

Three strands, one vocabulary.

Every term in your Weeks 1–4 reading list, organised the way it will actually be used. The courses run in parallel, not in sequence — each one hands you a different half of the same sentence.

Strand A

Concepts Andrew Ng — what AI can and can't do, and how projects actually run

Strand B

Mechanism 3Blue1Brown — what is happening inside the network and inside an LLM

Strand C

Measurement StatQuest — how you tell a good detector from a bad one

Strand A tells you whether to build it. Strand B tells you why it behaves the way it does. Strand C tells you whether it works. Project P4 lives almost entirely in Strand C.

PART 00

The plan, and how to read this

Roughly three hours a week for four weeks, deliberately light so you can bank time for Weeks 5–8. This handbook covers every topic in those three resources, plus the SOC translation layer that none of them provide.

WEEK	WATCH / READ	THIS HANDBOOK	PROJECT
1	AI for Everyone, modules 1–2	Parts 1–2	Start the P1 journal
2	AI for Everyone, modules 3–4	Parts 3–4	P1 entry
3	3B1B chapters 1–3, LLM intro, chapters 5–7	Parts 5–	P1 entry

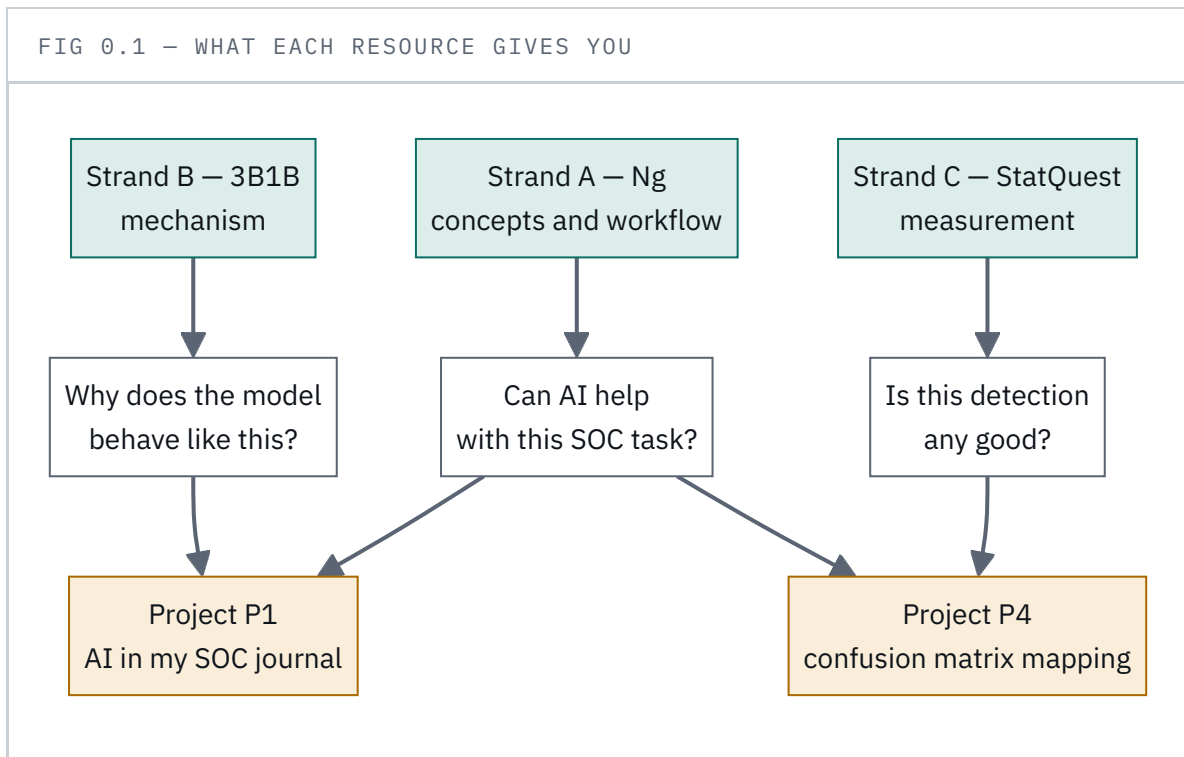
		6	
4	StatQuest: confusion matrix, sensitivity/specificity, bias–variance, ROC/AUC	Parts 7–8	Build P4

Two link fixes before you start. The 3Blue1Brown playlist URL in your plan has line breaks in it and won't resolve — go to the channel and open the official Deep Learning playlist. The StatQuest link is just the channel root; the four videos you want are named in Part 7.

Chapter 4 of 3B1B is optional. It is the calculus behind backpropagation. You need the intuition from chapter 3, not the chain rule, unless you plan to go on to Level 5 in the other handbook.

How the strands connect

FIG 0.1 — WHAT EACH RESOURCE GIVES YOU

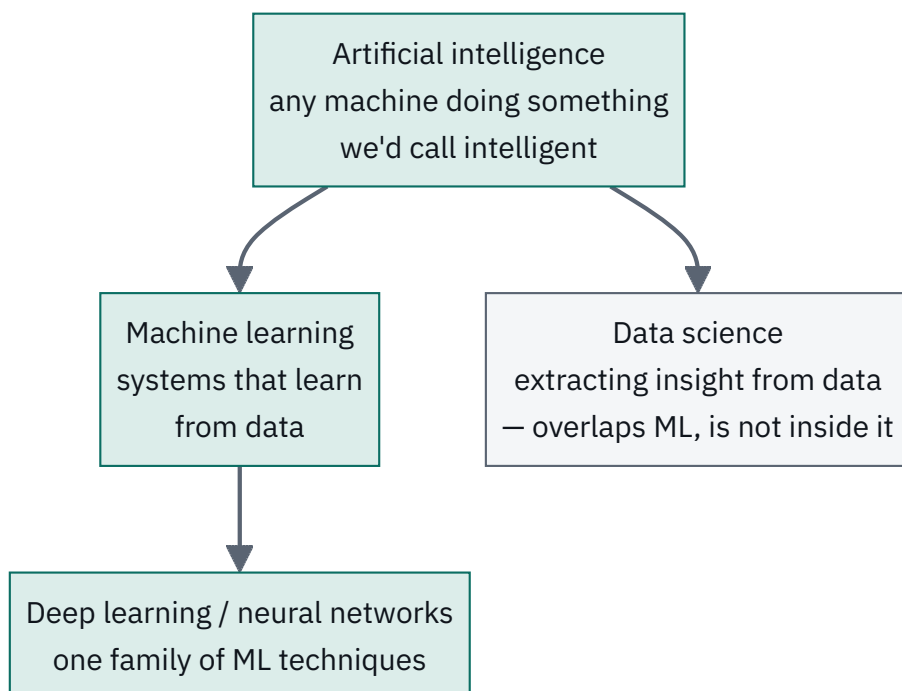


What AI actually is

1.1 The nested fields

The single most useful picture in the whole course. These terms get used interchangeably in vendor marketing and they are not interchangeable.

FIG 1.1 – AI, MACHINE LEARNING, DEEP LEARNING, DATA SCIENCE



- **Machine learning** produces a running system that makes predictions. Its output is software.
- **Data science** produces insight for humans to act on. Its output is usually a slide deck or a report.
- **Deep learning** is one technique within ML, currently the dominant one. "Neural network" and "deep learning" are used almost synonymously today.

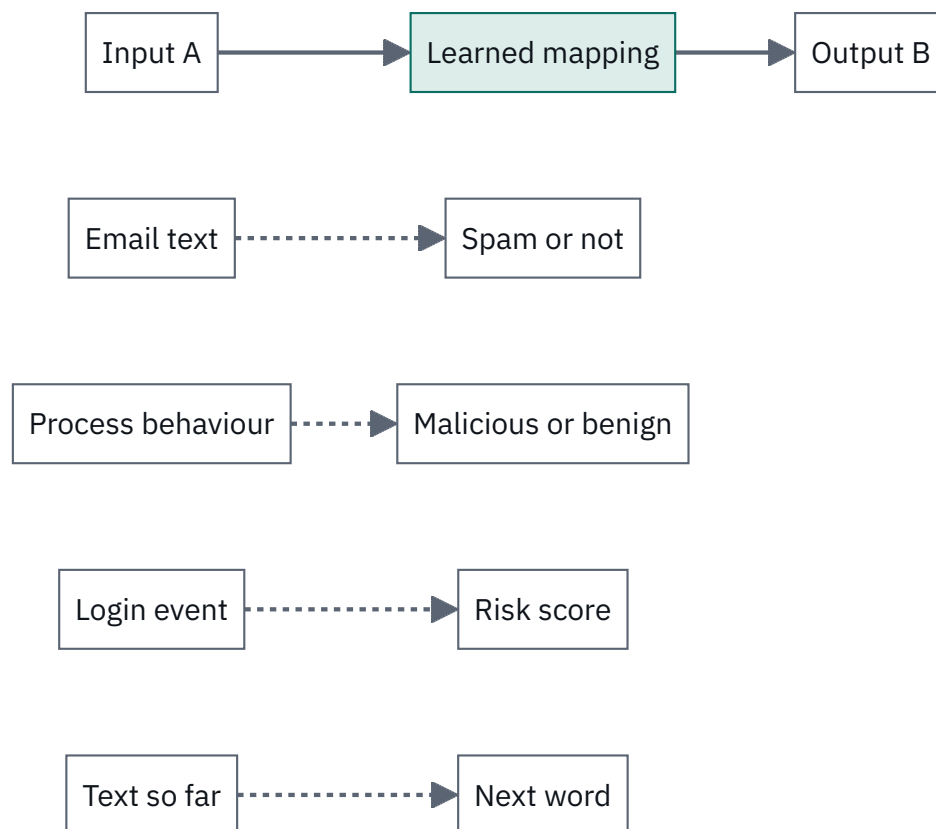
1.2 ANI and AGI

ANI — artificial narrow intelligence is everything that exists: a spam filter, a self-driving stack, a malware classifier, an LLM. Each does one class of thing. **AGI — artificial general intelligence** is a machine that can do anything a person can. Ng's point is that enormous ANI progress gets misread as progress toward AGI, and they are not the same road.

1.3 Supervised learning as $A \rightarrow B$

The engine behind almost all commercial value today. You show the system many pairs of input A and desired output B, and it learns the mapping.

FIG 1.2 — SUPERVISED LEARNING, AND WHAT IT LOOKS LIKE IN SECURITY



SOC TRANSLATION

Every detection model you will ever tune is an $A \rightarrow B$ mapping. The hard part is almost never the algorithm — it is that nobody agreed what B means. "Malicious" labelled by whom, on what evidence?

1.4 Structured and unstructured data

TYPE	WHAT IT IS	SECURITY EXAMPLES
Structured	Tables — rows, columns, defined fields	Firewall logs, auth events, asset inventory, netflow
Unstructured	Text, images, audio, video	Phishing email bodies, threat intel reports, ticket notes, malware source

The public excitement is mostly about unstructured data, because that is where progress felt magical. Most of the quiet business value is still in structured data.

1.5 What AI can and can't do

Ng's rule of thumb: **if a typical person can do the task with about a second of thought, it is a good candidate for AI today.** Longer chains of judgement, tasks needing scarce data, or tasks needing explanation of *why* tend to fail.

WORKS WELL	STRUGGLES
Learning a mapping with lots of labelled examples	Learning from a handful of examples
Recognising a pattern seen before	Generalising to a genuinely novel situation
Detecting a car's position from an image	Reading a traffic officer's hand gestures
Classifying at volume, consistently	Explaining its reasoning in a way that holds up

SOC TRANSLATION

Triage steps that take an analyst one second — "is this domain obviously internal?", "is this a known-good binary?" — are the AI-shaped parts of your job. The part where you decide whether to wake up the incident commander is not.

CHECK YOURSELF — PART 1 +

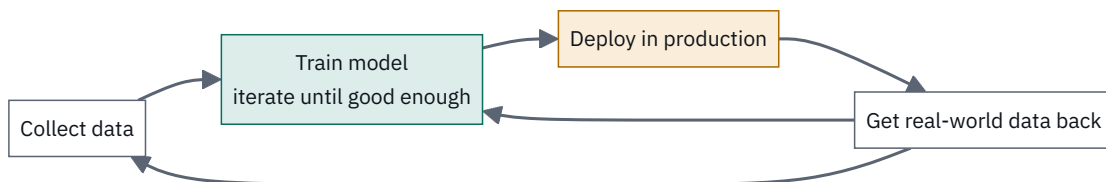
PART 02

AI FOR EVERYONE · MODULE 2

How AI projects actually run

2.1 The machine learning project workflow

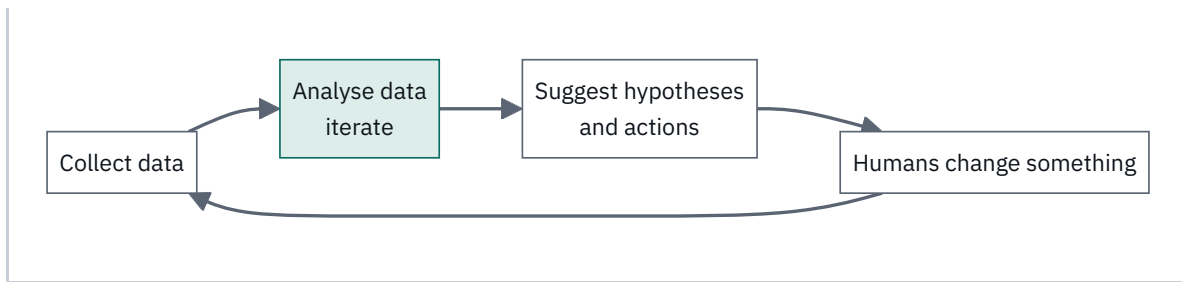
FIG 2.1 — THE LOOP, NOT THE LINE



The arrows back are the whole lesson. Training is not the end of the project; deployment starts the feedback that makes the next model better — and exposes the ways the first one was wrong.

2.2 The data science workflow

FIG 2.2 — DIFFERENT OUTPUT, DIFFERENT LOOP



2.3 Training, validation and test sets

Data is split so the model can be judged on examples it has never seen. Training set fits the model, validation (dev) set tunes choices, test set gives the honest final number. Judging a model on its training data is how you fool yourself.

2.4 Why models degrade after launch

Ng is blunt about the gap between a demo and production. Performance falls because the live data stops looking like the training data — new user behaviour, new attacker behaviour, a changed logging format, a seasonal shift.

SOC TRANSLATION

This is drift, and in security it is adversarial: the data changes *because* someone is trying to evade you. A detector's precision on the day it shipped tells you very little about its precision this quarter. Re-measure on a schedule.

2.5 Picking a project

The course's test for a good first project: valuable enough to matter, small enough to finish quickly, and with data you already have. Ng recommends starting with something that gains momentum internally rather than the most ambitious idea in the room.

2.6 Working with an AI team

Specify the **acceptance criteria** up front — what accuracy or precision counts

as success, measured on which held-out dataset. Vague goals produce models that are technically impressive and operationally useless.

CHECK YOURSELF – PART 2 +

PART 03

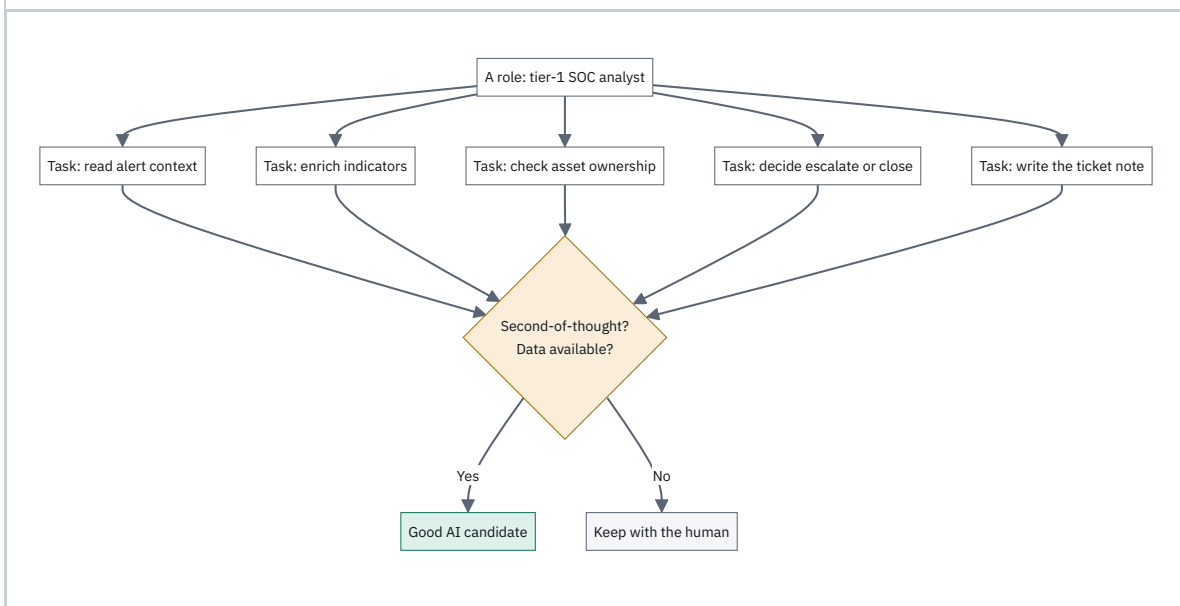
AI FOR EVERYONE · MODULE 3

AI inside an organisation

3.1 Decompose the job, not the role

The most transferable technique in the whole course, and the backbone of your P1 journal. You do not automate a job. You break a job into tasks and score each one.

FIG 3.1 – TASK DECOMPOSITION



3.2 The smart speaker case study

Ng's worked example breaks one apparently simple product into a pipeline: wake-word detection → speech recognition → intent classification → execute the action. Each stage is a separate learned mapping with its own error rate,

and the errors compound.

SOC TRANSLATION

Your triage pipeline is the same shape, and so is the compounding. If enrichment is 90% right and classification is 90% right, the end-to-end result is not 90%.

3.3 Who does what on an AI team

ROLE	OWNS
Software engineer	The product the model lives inside
Machine learning engineer	Training, tuning and shipping the model
ML researcher / scientist	Extending what is technically possible
Data scientist	Analysis and insight for decision-makers
Data engineer	Pipelines — getting clean data where it needs to be
AI product manager	Deciding what to build and what success means

These titles overlap heavily between companies. Ng's advice is to ask what someone owns rather than what their card says.

3.4 The AI Transformation Playbook

1. Execute pilot projects to build momentum — pick winnable ones first.
2. Build an in-house AI team.
3. Provide broad AI training across the organisation.
4. Develop an AI strategy, informed by what the pilots taught you.

5. Develop internal and external communications.

The ordering is deliberate: strategy comes fourth, not first, because early strategy written without hands-on experience tends to be wrong.

3.5 Where AI is applied

Computer vision · natural language processing · speech · generative AI · robotics. Worth knowing as a map — most security applications sit in NLP (text, logs, reports) and structured-data classification.

3.6 Common pitfalls

- Expecting AI to solve everything, or expecting it to solve nothing.
- Hiring two ML engineers and expecting them to invent the use case.
- Expecting the model to work first time — it never does.
- Insisting on traditional software-style acceptance criteria for a probabilistic system.

CHECK YOURSELF — PART 3 +

PART 04

AI FOR EVERYONE · MODULE 4

AI and society

4.1 Bias

Models learn what is in the data, including the parts nobody meant to teach them. Word embeddings famously absorbed occupational gender stereotypes; hiring and lending models have reproduced historic discrimination. Mitigations discussed: better and more representative data, technical de-biasing, transparency and auditing, and a diverse team likely to notice the problem at all.



SOC TRANSLATION

A user-behaviour analytics model trained on a period where one team worked unusual hours will treat that team as anomalous forever. Bias in security tooling shows up as certain people or systems being permanently over-alerted.

4.2 Adversarial attacks on AI

Inputs crafted specifically to fool a model — a few pixels changed and the classifier confidently reads a different object. This is the direct ancestor of everything in Part 8 of your Claude handbook: prompt injection is an adversarial attack against a language model.

4.3 Explainability

Often technically possible, always harder than people expect, and increasingly demanded by regulators. Worth internalising: a model that cannot explain itself can still be right, and one that produces a plausible explanation can still be wrong.

4.4 Economic and social effects

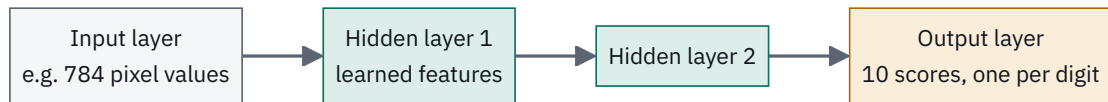
Job displacement and job creation, effects on developing economies, and the case for realistic expectations over both hype and doom. Ng's consistent framing: worry about the concrete harms happening now — bias, misuse, concentration of power — rather than only about a hypothetical superintelligence.

CHECK YOURSELF — PART 4 +

Grant Sanderson's series builds a digit-recognising network from nothing. You are not learning to build one — you are learning what the words mean so that "the model was trained on" stops being a black box.

5.1 Chapter 1 — what a neural network is

FIG 5.1 — LAYERS, WEIGHTS, ACTIVATIONS

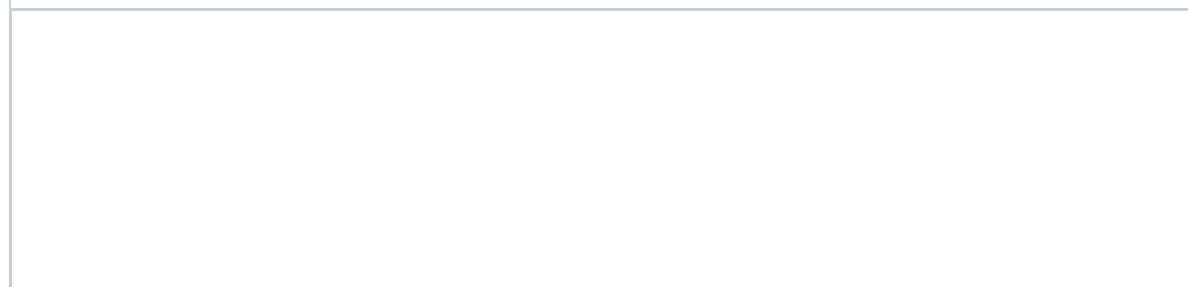


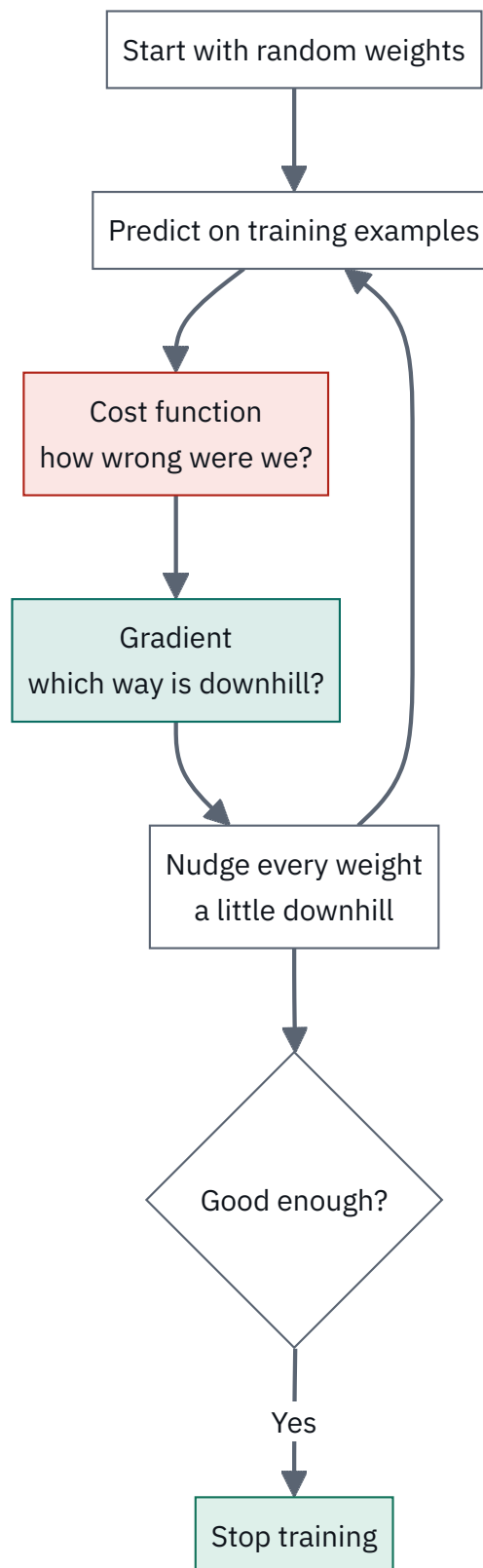
Terms introduced here, all of which you will meet again constantly:

- **Neuron** — a thing that holds a number between 0 and 1, called its **activation**.
- **Weight** — how strongly one neuron's output influences the next. This is what gets learned.
- **Bias** — a threshold added before the activation function; how reluctant a neuron is to fire.
- **Activation function** — squashes the weighted sum into a usable range. Sigmoid was the classic; **ReLU** is the modern default.
- **Parameters** — all the weights and biases together. When you read "a 70-billion-parameter model", this is what is being counted.
- The whole network is just a **very large function**: numbers in, numbers out.

5.2 Chapter 2 — gradient descent, or how networks learn

FIG 5.2 — THE TRAINING LOOP





- **Cost / loss function** — one number summarising how wrong the network currently is.

- **Gradient descent** — repeatedly stepping downhill on that cost surface.
- **Learning rate** — how big each step is. Too big overshoots, too small never arrives.
- **Local minimum** — a valley that isn't the deepest valley. In practice, good enough.
- **What "learning" means:** nothing more mystical than nudging millions of numbers to reduce one number.

5.3 Chapter 3 — backpropagation

Backpropagation is how the network works out which weights to blame. Each training example effectively "votes" on how every weight should move; those votes are averaged and applied.

- **Backpropagation** — propagating error backwards from the output to assign responsibility.
- **Stochastic gradient descent (SGD)** — using small random batches instead of the whole dataset each step, because it's far faster.
- **Mini-batch** — the small group used for one step. Batches are why training is described in "steps" and "epochs".

Chapter 4 derives the same thing in calculus. Skip it unless you want the maths — the intuition from chapter 3 is what carries into everything else.

CHECK YOURSELF — PART 5 +

PART 06

3BLUE1BROWN · LLM INTRO, CHAPTERS 5-7

Inside a large language model

This is the part that changes how you use Claude. Every quirk in the other handbook — context limits, tokenisation, hallucination — has its

explanation here.

6.1 The core trick: predict the next token

FIG 6.1 – AUTOREGRESSIVE GENERATION



A chatbot is this loop running over and over. Nothing in the architecture plans a whole answer in advance — it produces one token, then reconsiders everything including what it just said.

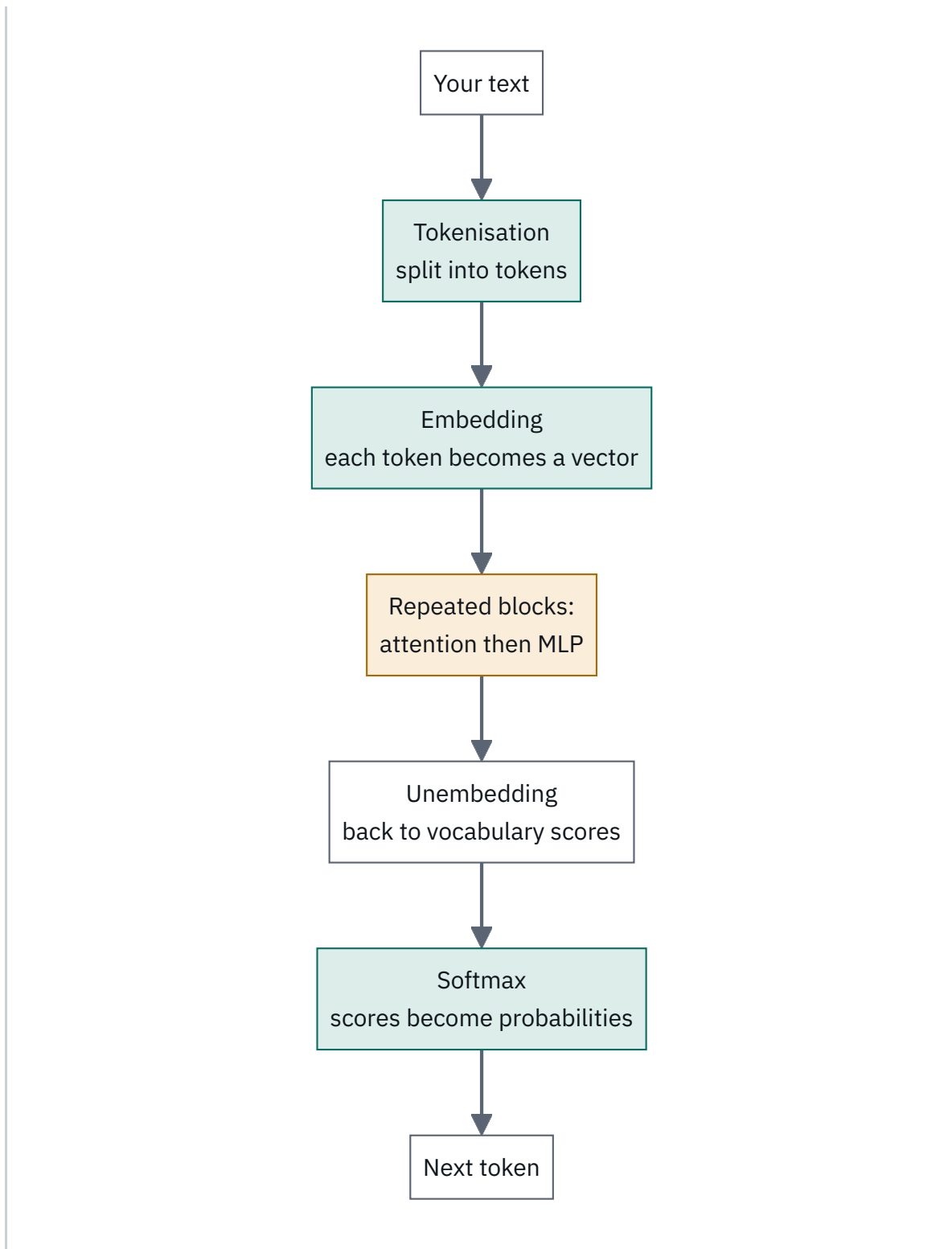
- **Pretraining** — the enormous first pass, learning to predict text from a huge corpus.
- **Fine-tuning** and **RLHF** (reinforcement learning from human feedback) — the later passes that turn a text predictor into something that answers helpfully.
- **Temperature** — how much randomness is allowed when sampling. Zero is deterministic and flat; high is creative and unreliable.
- **Scale** — parameter counts and training compute, and why bigger has kept meaning better.

WHY THIS MATTERS FOR YOUR PROMPTS

Hallucination stops being mysterious here. The model is always producing the most plausible continuation — plausibility and truth usually coincide, and when they don't, nothing in the mechanism notices.

6.2 Chapter 5 — the transformer, end to end

FIG 6.2 – WHAT HAPPENS TO YOUR PROMPT



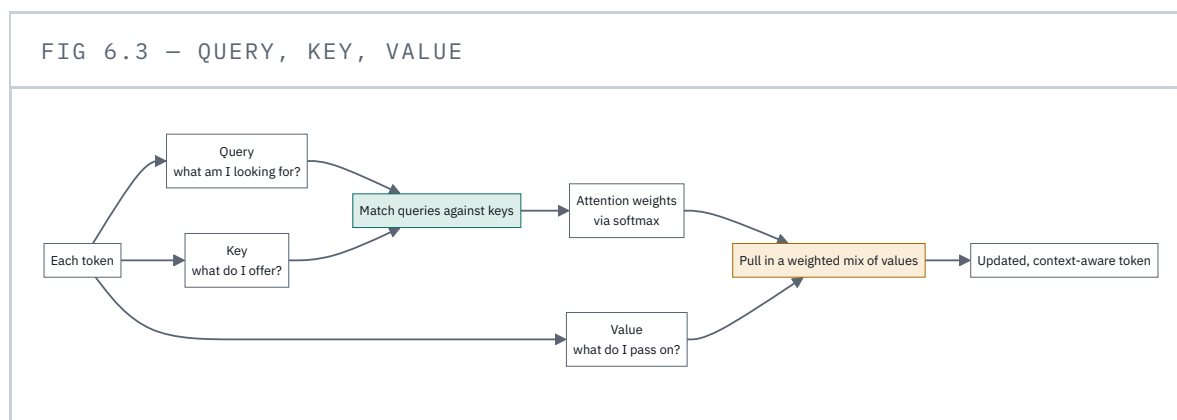
- **Token** — a word or piece of a word. Tokenisation happens before the model sees anything.
- **Embedding** — each token becomes a long list of numbers, a point in a high-dimensional space where direction carries meaning.
- **Context size / context window** — how many tokens the model can attend

to at once. When your first handbook says "1M context", this is the number.

- **Logits and softmax** — raw scores turned into a probability distribution that sums to one.
- **GPT** — generative pre-trained transformer. All three words now mean something to you.

6.3 Chapter 6 — attention

The mechanism that made all of this work, from the paper *Attention Is All You Need*. Every token gets to look at every other token and update what it means in context — "bank" next to "river" ends up somewhere different from "bank" next to "account".



- **Self-attention** — tokens attending to other tokens in the same sequence.
- **Masking** — later tokens are hidden during training so the model can't cheat by reading ahead.
- **Multi-head attention** — many attention operations in parallel, each learning a different kind of relationship.
- **Cross-attention** — attending to a different sequence, as in translation.
- **Why it won:** it parallelises across GPUs in a way the older recurrent models could not. Cost grows with the square of context length, which is why long contexts are expensive.

6.4 Chapter 7 — where facts live

Most of a model's parameters are not in the attention blocks — they are in the **multilayer perceptrons** between them. The chapter's argument, built around storing "Michael Jordan plays basketball", is that MLPs act as a kind of key-value memory.

- **MLP block** — two matrix multiplications with a simple non-linear function between them.
- **Key-value memory** — one part recognises a pattern, the other writes an associated fact back into the stream.
- **Superposition** — far more concepts are stored than there are dimensions, packed into overlapping directions. This is why models are so hard to interpret, and why edits to them have unpredictable side effects.

CARRY THIS INTO THE OTHER HANDBOOK

Superposition is the honest reason nobody can fully audit what a model knows. When Part 8 of Volume 1 says a jailbreak worked through role-play, this is the terrain it worked on.

CHECK YOURSELF — PART 6 +

PART 07

STATQUEST

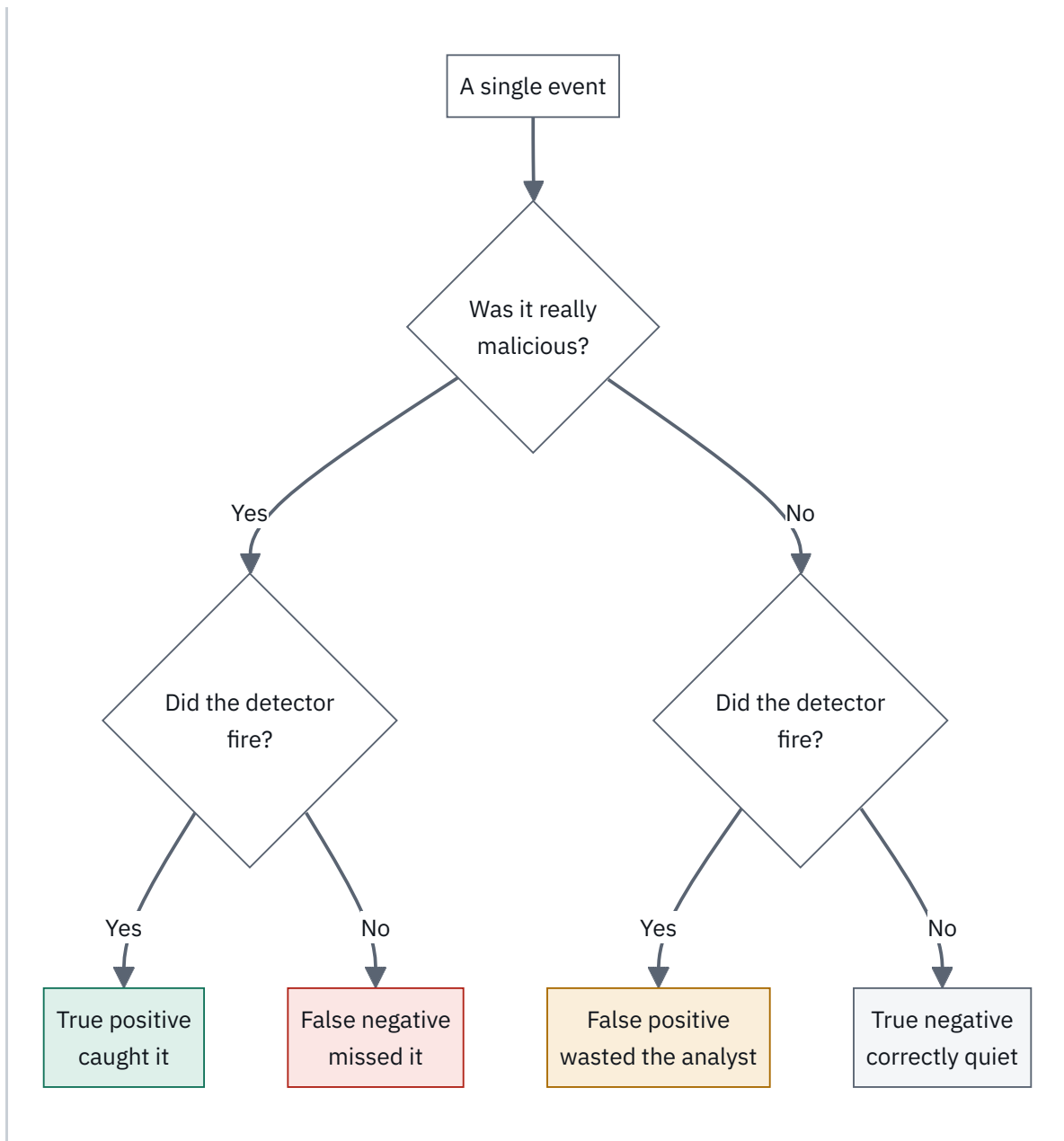
Measuring a classifier

The four videos: *Machine Learning Fundamentals: The Confusion Matrix, Sensitivity and Specificity*, *Machine Learning Fundamentals: Bias and Variance*, and optionally *ROC and AUC, Clearly Explained!* This is the vocabulary you will use for the rest of your career in detection.

7.1 The confusion matrix

Four boxes. Every classifier result lands in exactly one of them.

FIG 7.1 — THE FOUR OUTCOMES



7.2 The metrics, and what each one asks

METRIC	FORMULA	THE QUESTION IT ANSWERS
Accuracy	$(TP+TN) / \text{all}$	How often was I right overall? Usually misleading.
Precision	$TP / (TP+FP)$	When it fired, how often was it real?
Recall / sensitivity /	$TP / (TP+FN)$	Of everything real, how much did I catch?

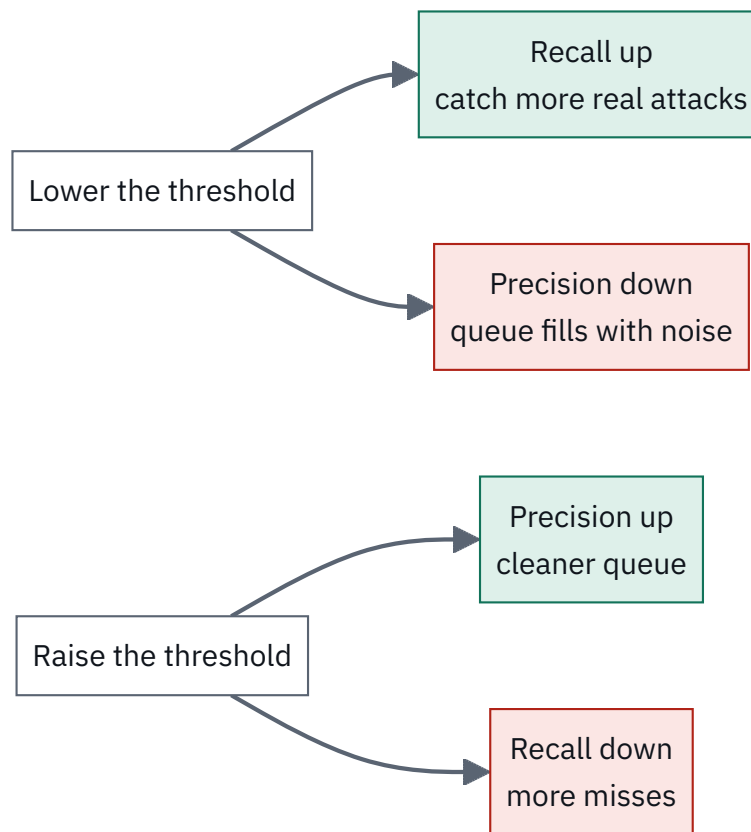
TPR		
Specificity / TNR	$TN / (TN+FP)$	Of everything benign, how much did I correctly ignore?
False positive rate	$FP / (TN+FP) = 1 - \text{specificity}$	How much of the benign world do I alert on?
F1	$2 \cdot (P \cdot R) / (P+R)$	One number balancing precision and recall.
Prevalence / base rate	$(TP+FN) / \text{all}$	How rare is the thing? Changes everything.

Recall and sensitivity are the same number. Medicine says sensitivity and specificity; machine learning says recall and precision. StatQuest uses both, which is exactly why it's on your list.

7.3 The threshold is the dial

Most classifiers output a score, not a verdict. A threshold turns the score into an alert. Move it and you trade the two error types against each other — you never remove them.

FIG 7.2 — WHAT THE THRESHOLD DOES



7.4 ROC and AUC

Sweep the threshold across every possible value, plot true positive rate against false positive rate at each one, and you get the **ROC curve**. The **AUC** is the area underneath it: 1.0 is perfect, 0.5 is a coin flip. AUC lets you compare two detectors independent of where the threshold currently sits.

The catch worth knowing: ROC uses the false positive *rate*, which stays flattering when negatives vastly outnumber positives — exactly your situation. For rare events, a precision–recall curve tells you more about what your queue will actually look like.

7.5 Bias and variance

PROBLEM	LOOKS LIKE	MEANING

High bias — underfitting	Bad on training data <i>and</i> new data	The model is too simple to capture the pattern
High variance — overfitting	Excellent on training data, poor on new data	It memorised the examples instead of the pattern
The sweet spot	Good on both, similar scores	It learned the signal, not the noise

Countermeasures the videos cover: **regularisation** (penalise complexity), more and more varied training data, simpler models, and **cross-validation** — rotating which slice of data is held out, so your estimate doesn't depend on one lucky split.

SOC TRANSLATION

An overfitted detection is one tuned so tightly to last quarter's incident that it catches that exact attacker and nobody else. It will look superb in the report and miss the next intrusion.

CHECK YOURSELF — PART 7 +

PART 08

The SOC translation

None of the three courses covers this. It is the bridge from the metrics to your alert queue, and it is the intellectual core of project P4.

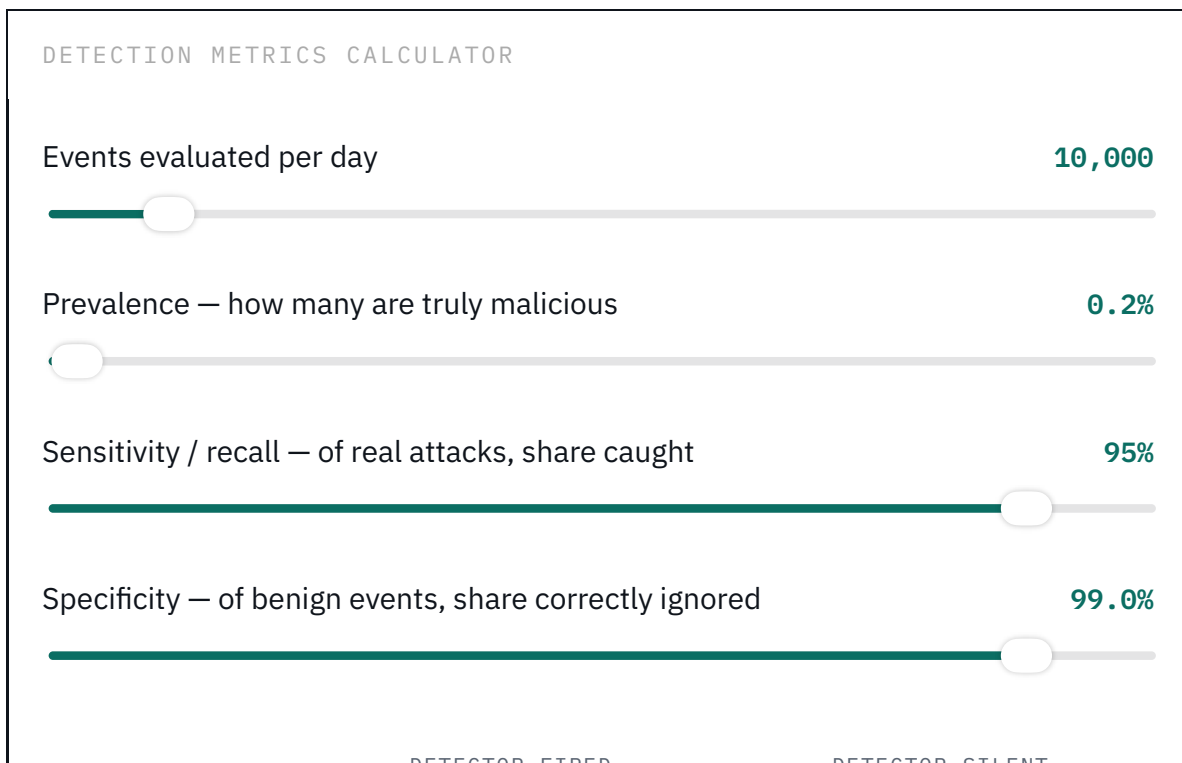
8.1 The mapping

METRIC TERM	IN YOUR QUEUE	WHO PAYS FOR IT
True	A real incident, correctly	Nobody — this is the job working

positive	alerted	
False positive	An alert that dies in triage	Analyst hours, and eventually alert fatigue
False negative	An intrusion nobody was told about	The organisation, discovered later
True negative	Normal activity, correctly silent	Nobody — and it's invisible, so it never gets credit
Precision	Share of your queue that's worth working	Directly sets analyst morale
Recall	Share of real attacks you'd actually see	Directly sets your risk

8.2 The base rate problem, made concrete

This is the single most important idea for detection engineering, and it is why accuracy is a useless headline number in security. Move the sliders — especially prevalence.



DETECTOR FIRED

DETECTOR SILENT

TRULY MALICIOUS	19 true positives	1 false negatives — missed
TRULY BENIGN	100 false positives — wasted triage	9,880 true negatives

PRECISION	RECALL	F1	ACCURACY
16%	95%	0.27	99.0%

Your analysts open **119** alerts a day. For every genuine incident they work **5.3** false ones, and **1** real events were missed entirely. Accuracy reads 99.0% — which is why nobody in detection engineering quotes accuracy.

Read the default case again. 99% accuracy, 95% recall, a specificity most vendors would be proud of — and roughly five out of every six alerts in the queue are junk. Accuracy looked wonderful because true negatives dominate. This is the number to put on the front page of your P4 writeup.

8.3 The tuning conversation, properly framed

"Reduce false positives" is not a request an engineer can act on cleanly, because every reduction buys itself with recall. The useful framing is a budget:

- **Start from capacity.** How many alerts per day can this team actually work? That caps your false positives.
- **Then ask what recall that buys** at the prevalence you actually observe.
- **Then decide whether the gap is acceptable**, and what compensating control covers it — a second detection, a hunt, a hard preventive block.
- **Re-measure quarterly.** Prevalence and attacker behaviour both move; your threshold was tuned against a world that no longer exists.

Sentence worth memorising for interviews and for P4: "Precision is what my analysts feel; recall is what my organisation risks; prevalence decides how brutal the trade between them is."

8.4 Where AI-assisted triage changes the arithmetic

The interesting move is not making the detector more precise. It is making a false positive cheaper to dismiss. If an LLM reliably drafts the enrichment and context for an alert, the cost per false positive falls, which means you can afford a lower threshold — which buys back recall.

JOURNAL PROMPT FOR P1

Pick one noisy detection. Estimate the minutes a false positive costs today. Estimate what it would cost with a decent auto-enrichment summary attached. That difference, times your FP count, is your business case.

PART 09

Projects P1 and P4

9.1 P1 – the "AI in my SOC" journal

Runs Weeks 1–12, about 30 minutes a week, 10–12 entries. A repeatable entry template:

FIELD	WHAT TO WRITE
Date and week	Keeps the arc visible when you review it later
Task observed	The specific SOC task, decomposed small enough to score
Second-of-thought test	Could a person do this in about a second? Is the data there?

Prompt tried	Paste it verbatim — the exact words are the artefact
What happened	Worked, partly worked, failed. Be specific about the failure mode
Iteration	What you changed, and whether it helped
Verdict	Automate, assist, or leave alone — and why

Tag each failure with a cause from the other handbook's debug sequence — hallucination, too generic, wrong format, tangent, over-refusal. By week 12 you will have a distribution, and that distribution is the actual finding of the project.

9.2 P4 — confusion matrix to alert queue

Due around Week 4. Deliverable: a one-page diagram plus a short writeup. A structure that will hold up to scrutiny:

1. **The diagram.** The four-box matrix with your own SOC labels in each cell, plus the cost of each error type written underneath in the units you actually care about — analyst minutes and dwell time.
2. **A hypothetical detection.** Name it, say what it fires on, and state your assumed volume, prevalence, sensitivity and specificity. Use the calculator in Part 8 to generate the counts.
3. **Compute precision and recall** and show the working. Include accuracy too, then show why it flatters.
4. **The trade-off.** Recompute at a lower and a higher threshold. Two extra rows of numbers make the argument better than a paragraph.
5. **The recommendation.** Where you'd set it, tied to analyst capacity, and what compensating control covers the recall you gave up.

What earns marks: stating your assumptions explicitly, and naming the base rate as the reason accuracy misleads. Most writeups on this topic stop at the

four boxes and never mention prevalence.

FINAL CHECK – EVERYTHING +

PART 10

Glossary

TERM	MEANING	SOURCE
Accuracy	Share of all predictions that were correct. Misleading on rare events	StatQuest
Activation	The number a neuron currently holds	3B1B 1
Activation function	Squashes a weighted sum into range — sigmoid, ReLU	3B1B 1
Adversarial attack	Input crafted to fool a model. Prompt injection is one	Ng 4
AGI	Artificial general intelligence — hypothetical, does not exist	Ng 1
ANI	Artificial narrow intelligence — everything that does exist	Ng 1
Attention	Operation letting every token read from every other token	3B1B 6
AUC	Area under the ROC curve. 1.0 perfect, 0.5 chance	StatQuest
Backpropagation	Assigning blame for error backwards through the network	3B1B 3

Base rate / prevalence	How common the positive class really is	StatQuest
Bias (statistical)	Underfitting — model too simple for the pattern	StatQuest
Bias (societal)	Model reproducing unfairness present in its data	Ng 4
Bias (neuron)	Threshold added before activation	3B1B 1
Confusion matrix	The TP / FP / FN / TN table	StatQuest
Context window	How many tokens the model can attend to at once	3B1B 5
Cost / loss function	One number measuring how wrong the model is	3B1B 2
Cross-attention	Attending to a second, different sequence	3B1B 6
Cross-validation	Rotating the held-out slice so results don't hinge on one split	StatQuest
Data science	Analysis producing insight for humans, not a deployed model	Ng 1
Deep learning	ML using multi-layer neural networks	Ng 1
Drift	Live data diverging from training data, degrading performance	Ng 2
Embedding	A token represented as a vector in meaning-space	3B1B 5
Explainability	Whether a model's reasoning can be shown and trusted	Ng 4
F1 score	Harmonic mean of precision and recall	StatQuest

False negative	Real thing, no alert. The miss	StatQuest
False positive	Alert with nothing behind it. The waste	StatQuest
Fine-tuning	Further training that shapes a pretrained model's behaviour	3B1B LLM
GPT	Generative pre-trained transformer	3B1B 5
Gradient descent	Iteratively stepping downhill on the cost surface	3B1B 2
Hallucination	Plausible continuation that happens to be false	3B1B LLM
Key-value memory	MLP behaviour: recognise a pattern, write back a fact	3B1B 7
Learning rate	Step size during gradient descent	3B1B 2
Logits	Raw output scores before softmax	3B1B 5
Masking	Hiding later tokens during training so the model can't peek	3B1B 6
Mini-batch	Small group of examples used for one training step	3B1B 3
MLP	Multilayer perceptron — holds most of a transformer's parameters	3B1B 7
Multi-head attention	Many attention operations in parallel, each learning different relations	3B1B 6
Next-token prediction	The core operation of a language model	3B1B LLM
Overfitting	High variance — memorised the training data	StatQuest
Parameters	All the weights and biases; the number in	3B1B 1

	"70B model"	
Precision	Of the alerts raised, how many were real	StatQuest
Pretraining	The large first training pass on a broad corpus	3B1B LLM
Query / key / value	The three projections that make attention work	3B1B 6
Recall / sensitivity	Of the real cases, how many were caught	StatQuest
Regularisation	Penalising complexity to fight overfitting	StatQuest
ReLU	Modern default activation function	3B1B 1
RLHF	Reinforcement learning from human feedback	3B1B LLM
ROC curve	TPR against FPR across every threshold	StatQuest
SGD	Stochastic gradient descent — training on random mini-batches	3B1B 3
Softmax	Turns scores into probabilities that sum to one	3B1B 5
Specificity	Of the benign cases, how many were correctly ignored	StatQuest
Structured data	Tabular — rows and columns	Ng 1
Superposition	More concepts stored than dimensions, packed into overlapping directions	3B1B 7
Supervised learning	Learning a mapping from labelled input–output pairs	Ng 1
Temperature	Randomness in token sampling	3B1B

		LLM
Test set	Held-out data used once, for an honest final score	Ng 2
Threshold	Score cut-off that turns a prediction into an alert	StatQuest
Token / tokenisation	Chunk of text; splitting text into chunks before the model sees it	3B1B 5
Training set	Data the model learns from	Ng 2
Transformer	The architecture underlying modern LLMs	3B1B 5
Underfitting	High bias — model too simple	StatQuest
Unstructured data	Text, images, audio, video	Ng 1
Validation / dev set	Held-out data used to tune choices during development	Ng 2
Variance	Sensitivity to the particular training data; too much means overfitting	StatQuest
Weight	Strength of connection between neurons; what training adjusts	3B1B 1

About this volume

Volume 2 of two. Covers the Weeks 1–4 foundations block: *AI for Everyone* by Andrew Ng (DeepLearning.AI, four modules, ~6 hours), the 3Blue1Brown Neural Networks / Deep Learning series (chapters 1–3, the LLM introduction, and chapters 5–7 on transformers, attention and MLPs), and the StatQuest evaluation videos on the confusion matrix, sensitivity and specificity, bias and variance, and ROC/AUC.

Parts 8 and 9 are not from the courses. They are the SOC translation layer and the project scaffolding, written to carry the vocabulary into projects P1 and P4.

Volume 1 — the Claude AI Handbook — covers the agentic side: interfaces, models, workspace, capabilities, and the threat model none of these courses touch.

VOLUME 3 · VERIFIED 23 AUGUST 2026

From AI concepts to an agent doing your triage.

Eight weeks that move from "what is a computer vision workload" to writing custom plugins for a system that investigates incidents on your behalf. Every module of all five resources, taught rather than listed.

Volume 3 of 3 · pairs with the Claude and AI Basics handbooks

Weeks 5–6	Microsoft's AI vocabulary The AI-901 concept path: generative AI, agents, vision, speech, NLP, extraction, RAG
-----------	---

Week 7	Copilot literacy AI Fluency and Career Essentials: prompting, grounding, responsible use
--------	---

Weeks 8–9	Security Copilot SC-5006: architecture, plugins, promptbooks, embedded experiences, agents
-----------	---

Weeks 10–12	Level 400 Ninja training: custom plugins, custom promptbooks, Logic Apps, knowledge grounding
-------------	--

Each block is a prerequisite for the next. RAG in Week 5 is the reason grounding works in

Week 7, which is the reason plugins matter in Week 9, which is what you extend in Week 11.

PART 00

The plan, verified

All five resources exist, are current, and are free to self-study. Two things in your plan needed checking, and only one of them was wrong.

AI-901 — your plan is right, and this surprised me

AI-901 is real. Microsoft retired AI-900 on 30 June 2026 and replaced it with **AI-901**, which went to general availability around June 2026 after an English beta in April. Both earn the same *Microsoft Certified: Azure AI Fundamentals* credential, passing score 700. The instructor-led course was renumbered too — AI-900T00-A became AI-901T00-A. If you book the exam, book AI-901.

AI-901 has a broader remit than AI-900 did: generative and agentic AI, text analysis, speech, computer vision, information extraction, effective prompting, and implementing solutions with **Microsoft Foundry** and Content Understanding. The concepts path in your plan covers the conceptual half. For the hands-on half, add *Get started with AI applications and agents on Azure*.

One naming correction

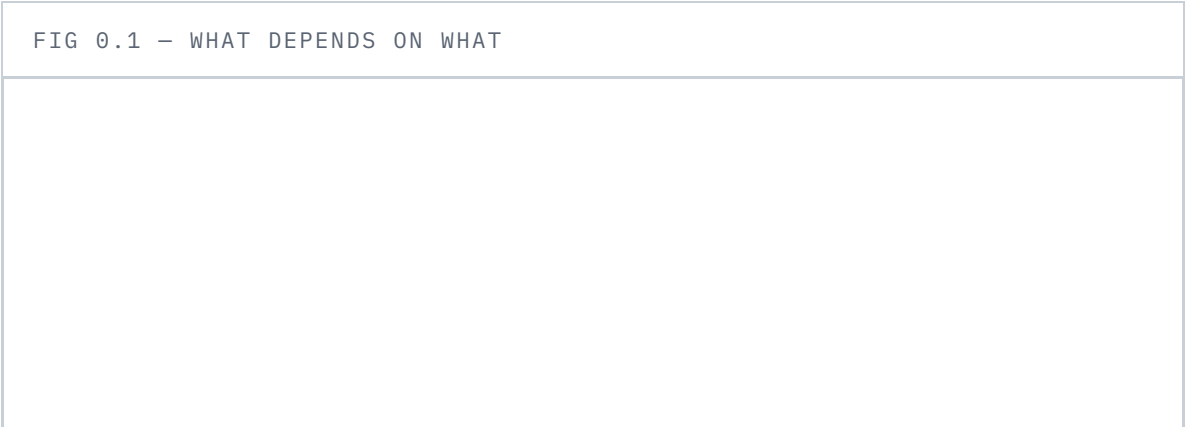
"Get started with Microsoft Copilot" is a module, not a learning path. It sits inside the **AI Fluency** path — that path is the ~1–2 hour block you actually mean. Separately there is a distinct three-module path called *Get started with Microsoft 365 Copilot*, which is the enterprise product and the only one that covers **Microsoft Graph grounding** properly. Given your SOC context,

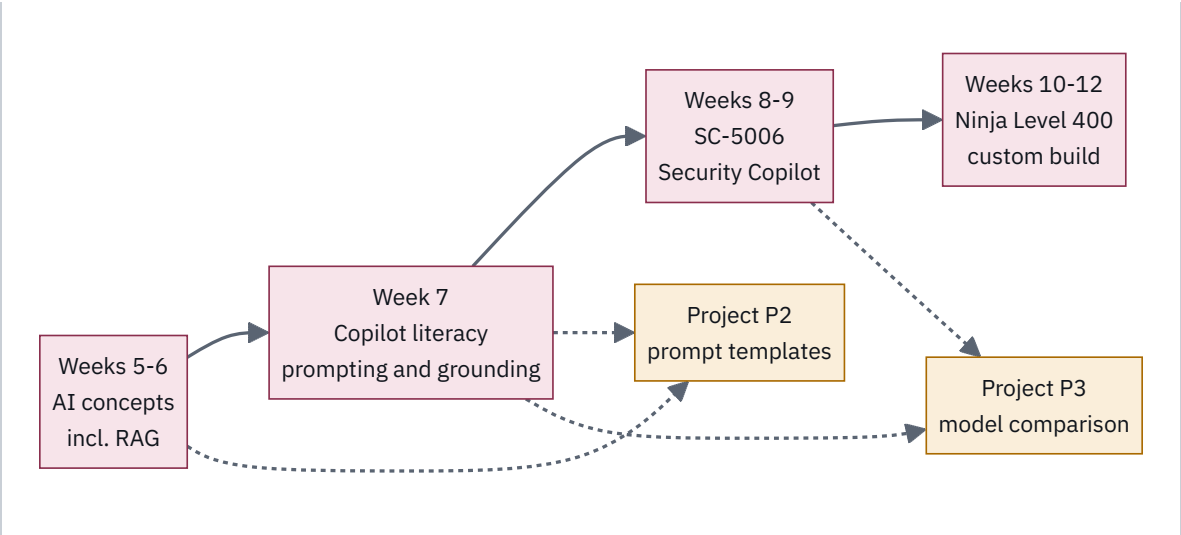
Graph grounding matters, so add it.

Status of each resource

RESOURCE	REAL SHAPE	VERIFIED
AI concepts path	7 modules, page refreshed 31 July 2026, anchored on Microsoft Foundry	Free, live
AI Fluency path	7 modules including the Copilot one	Free, live
Career Essentials in Generative AI	6 courses, ~4h 18m listed, Microsoft professional certificate on completion	Still free as of Aug 2026
SC-5006	6 modules, simulation-based, page updated May 2026. No licence needed for the guided simulations	Free, live
Security Copilot Ninja	Community Hub course, three tiers, certificate at 80%+ on a knowledge check	Free; module detail partly community-sourced

The twelve-week shape





A caution that runs through the whole volume. Security Copilot changes monthly. Everything here was verified in August 2026, but before you quote a price, an SCU figure, or an agent name in front of your team, check Microsoft's "What's new in Microsoft Security Copilot" page. The architecture is stable; the catalogue is not.

PART 01 AI CONCEPTS PATH · 7 MODULES

Microsoft's AI vocabulary

This path teaches you to recognise *workloads* — the categories Microsoft organises its AI services into. Learn the categories and the exam, the product portfolio, and half of vendor marketing become navigable.

1.1 Module 1 — AI concepts and responsible AI

The opening module frames what kinds of solutions AI makes possible, then introduces the responsible-AI frame that recurs in every other Microsoft course you'll take. Microsoft's six principles are worth memorising because they appear in AI-901, in the AI Fluency path, in Career Essentials, and in Security Copilot's own documentation:

PRINCIPLE	WHAT IT ASKS	SOC ANGLE
-----------	--------------	-----------

Fairness	Does the system treat all groups equitably?	Does your UEBA model permanently flag the night shift?
Reliability and safety	Does it behave consistently, including under stress?	What does the model do during a major incident, at volume?
Privacy and security	Is data protected, and access controlled?	What leaves your tenant when you paste log content into a prompt?
Inclusiveness	Does it work for everyone who needs it?	Accessibility of the tooling your analysts must use all day
Transparency	Do people understand what it does and its limits?	Do analysts know when an alert summary was AI-written?
Accountability	Are humans answerable for outcomes?	Who owns the miss when an agent auto-closed the alert?

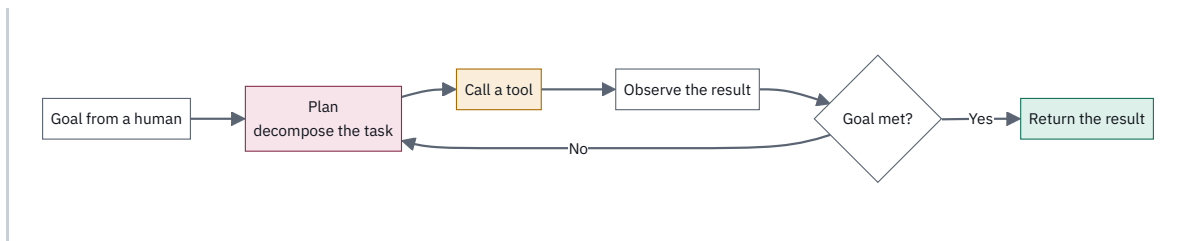
1.2 Module 2 – Generative AI and agents

Everything Volume 2 taught you about transformers arrives here in Microsoft's vocabulary. The module covers how LLMs work, how to write effective prompts, and then the newer material: **agents**.

Microsoft's working definition of an agent is a generative model given three additional things:

1. **Instructions** — a persistent brief describing its role and constraints.
2. **Tools** — functions it may call to act on the world or retrieve information.
3. **Knowledge** — grounding data it can consult, typically through RAG.

FIG 1.1 – THE AGENT LOOP



That loop is the whole idea, and it is identical whether the vendor calls it an agent, an autonomous workflow, or a copilot. What varies is how many turns it takes before a human is consulted — which, as Volume 1 Part 8 argued, is exactly where the security question lives.

Terms this module establishes: **prompt, completion, system message, grounding, tokens, context window, temperature, multi-agent orchestration**, and **Microsoft Foundry** as the platform where you build these.

1.3 Module 3 – Natural language processing

NLP is the workload category for anything that extracts meaning from text. The classic techniques, all of which predate LLMs and all of which are still used because they are cheap and deterministic:

TECHNIQUE	WHAT IT PRODUCES	SECURITY USE
Tokenisation	Text split into countable units	The preprocessing step under everything else
Language detection	Which language, with confidence	Routing multilingual phishing reports
Sentiment analysis	Positive / negative / neutral score	Insider-risk signals in communications
Key phrase extraction	The main topics in a document	Summarising a long threat intel report
Named entity recognition	People, places, orgs, dates, and custom entities	Pulling IPs, domains, hostnames and usernames out of free-text tickets

Entity linking	Entities resolved to a known reference	Mapping "that Exchange box" to an asset ID
Text classification	A label per document	Triaging user-reported email into categories
Translation	Text in another language	Reading foreign-language intel and ransom notes
Embeddings	Text as vectors, so similarity is measurable	Finding tickets that resemble this one — and the basis of RAG

SOC TRANSLATION

Named entity recognition is the single most immediately useful classical NLP technique in a SOC. Most of your unstructured intelligence — tickets, emails, reports — contains indicators trapped in prose. NER is what frees them, and it does not require an LLM.

1.4 Module 4 – Speech

Two directions, and one increasingly important security implication.

- **Speech recognition (speech to text)** — audio in, transcript out. Historically built from an *acoustic model* mapping sound to phonemes and a *language model* mapping phonemes to likely words; modern systems learn both jointly.
- **Speech synthesis (text to speech)** — text in, audio out. Modern neural voices are close to indistinguishable from recordings, and support **custom neural voice** — a voice trained on one person's speech.
- **Speaker recognition** — identifying or verifying who is talking.
- **Speech translation** — the two combined across languages.

SOC TRANSLATION

Custom neural voice is why vishing rose sharply. If your organisation's

helpdesk verifies identity by voice, that control is already broken. This module is the technical reason to go and check.

1.5 Module 5 – Computer vision

Analysing images, and generating them. Underneath the analysis side sits the **convolutional neural network** — filters sliding across an image, learning edges, then shapes, then objects. The workload types:

TASK	OUTPUT
Image classification	One label for the whole image
Object detection	Labels plus bounding boxes for each object
Semantic segmentation	A label for every pixel
OCR	Text extracted from an image or scan
Face detection and recognition	Faces located, and optionally identified
Image generation	New images from a text description
Multi-modal models	Images and text understood together

Multi-modal is the one that matters most to you now: it is why you can paste a screenshot of an error, a phishing page, or a dashboard into a chat and get a useful answer. Note that Microsoft applies extra governance to face recognition specifically — a direct application of the responsible-AI principles from Module 1.

1.6 Module 6 – Information extraction

The bridge between unstructured documents and structured data you can query. Microsoft's product here is **Content Understanding** in Foundry, with Document Intelligence underneath. The concepts:

- **Prebuilt models** — trained for common document types: invoices, receipts, ID documents, business cards.
- **Custom models** — trained on your own document layouts when the prebuilt ones don't fit.
- **Field extraction** — pulling named values out, each with a confidence score.
- **Layout analysis** — recognising tables, headings and reading order, not just characters.
- **Confidence scores** — the reason a human-review threshold is part of the design, not an afterthought.

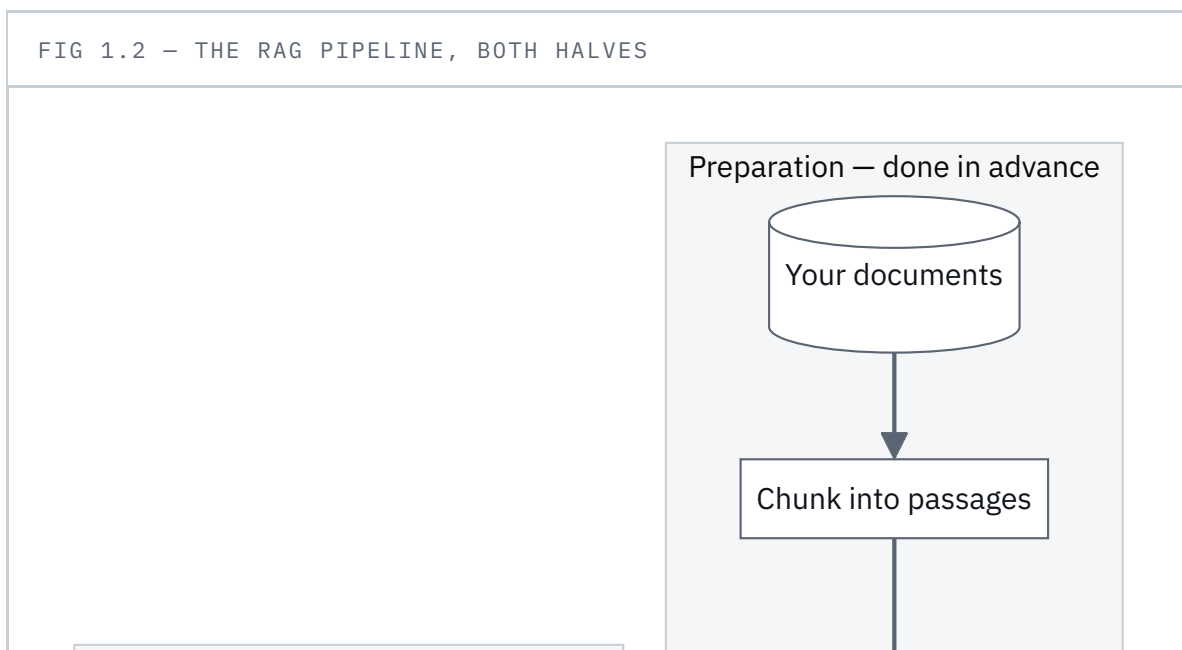
SOC TRANSLATION

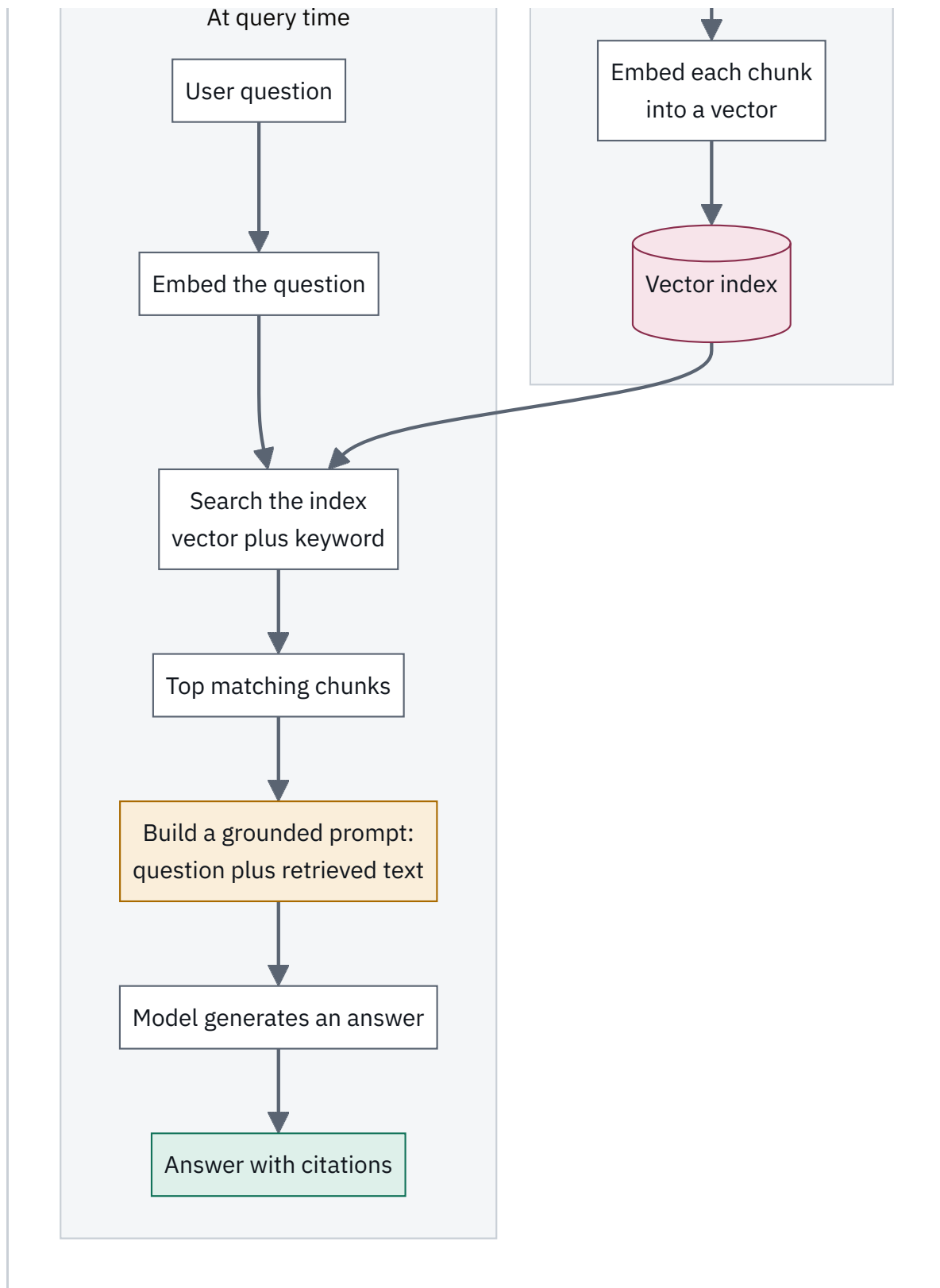
Confidence-scored extraction with a human-review threshold is the same shape as your alert threshold from Volume 2. High confidence auto-processes; low confidence routes to a person. You already know how to reason about where to put that line.

1.7 Module 7 — Retrieval-augmented generation

The most important module in the path for everything that follows, because RAG is how Copilot, Security Copilot, and almost every enterprise AI product actually answer questions about *your* data.

FIG 1.2 — THE RAG PIPELINE, BOTH HALVES





Terms to hold onto:

- **Chunking** – splitting documents into passages small enough to retrieve precisely, large enough to make sense alone. Chunk badly and the whole system degrades; this is the most common cause of a disappointing RAG

deployment.

- **Embedding** — the vector representation from Volume 2, now doing real work.
- **Vector search** — finding chunks whose meaning is near the question, even with no shared keywords.
- **Hybrid search** — vector plus traditional keyword search, then **reranking**. Almost always better than either alone, because exact identifiers — a hostname, a CVE number — are precisely what vector search is worst at.
- **Grounding** — the act of putting retrieved text into the prompt so the answer is based on it.
- **Citation** — returning which chunk supported which claim. Your verification hook.

Why this matters more than it sounds: RAG is the main practical defence against hallucination, and it is also the main practical attack surface. If an attacker can put text into a document your RAG system indexes, they can influence answers. That is indirect prompt injection — threat T4 in Volume 1.

CHECK YOURSELF — PART 1 +

PART 02

AI FLUENCY PATH · 7 MODULES

Copilot literacy and prompting

Short block, high leverage. This is where Microsoft teaches its house style of prompting, which is the style Security Copilot expects in Week 9.

2.1 The modules

1. **Get started with AI** — foundational concepts and a short history; where AI already sits in everyday products.
2. **Explore generative AI** — what generative models do and how they change

creative and knowledge work.

3. **The art of the prompt** — search engines versus reasoning engines; prompt engineering and critical thinking.
4. **Explore responsible AI** — the six principles again, applied to daily use.
5. **Get started with Microsoft Copilot** — Copilot's functions, how to interact effectively, and GPT model fundamentals.
6. **Boost your productivity with Microsoft Copilot** — practical workflows, image and document analysis, and the retrieval-augmented generation method Copilot uses.
7. **AI's impact** — accessibility, job roles, humanitarian applications.

2.2 Search engine versus reasoning engine

The framing Microsoft leans on hardest, and it is genuinely useful. A search engine *finds documents that contain your words*. A reasoning engine *constructs an answer*. The consequences:

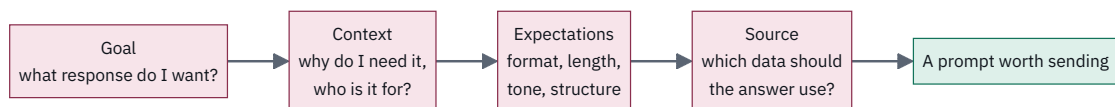
	SEARCH ENGINE	REASONING ENGINE
You give it	Keywords	Context, intent, constraints
It returns	Ranked links to verify yourself	A synthesised answer
Shorter input	Usually better	Usually worse
Failure mode	Nothing relevant found	Something plausible invented
Verification	Built in — you read the source	Must be added deliberately

Most people prompt badly because they are still typing keywords at a reasoning engine.

2.3 Microsoft's prompt anatomy

Microsoft teaches four elements. They map almost exactly onto the CRISP framework in Volume 1, and Security Copilot's documentation uses the same four — which is why learning them now saves time in Week 9.

FIG 2.1 — GOAL, CONTEXT, EXPECTATIONS, SOURCE

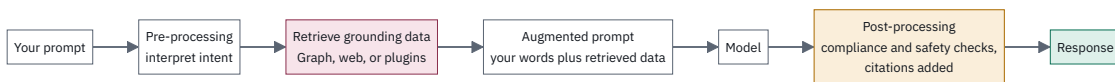


Source is the element people skip and the one that matters most in security. "Using only the incident data shown, ..." is the difference between a grounded answer and a confident invention.

2.4 How Copilot grounds an answer

Consumer Copilot grounds on web search. Microsoft 365 Copilot grounds on **Microsoft Graph** — your mail, files, chats, meetings and calendar, subject to your existing permissions. Security Copilot grounds on security plugins. Same pattern, different data plane.

FIG 2.2 — THE GROUNDING ROUND TRIP



Permission inheritance is the key security property. Copilot retrieves only what the signed-in user could already open. It does not escalate. But it does make over-shared data *discoverable* — a file everyone could technically access but nobody had found is now one question away. This is the most common real-world Copilot governance finding.

2.5 What to add for your role

Add the separate **Get started with Microsoft 365 Copilot** path — three modules covering how M365 Copilot works, its responsible-AI design, and Graph grounding explicitly. It is the piece the consumer-focused AI Fluency track leaves out, and it is the piece a security professional gets asked about.

CHECK YOURSELF — PART 2 +

PART 03

CAREER ESSENTIALS · 6 COURSES

Responsible AI in the workplace

Roughly five hours, still free, six courses, and a Microsoft professional certificate at the end. Lighter technically than everything around it — treat it as the ethics-and-context block, and take the certificate since it costs nothing.

COURSE	INSTRUCTOR	WHAT IT COVERS
What Is Generative AI?	Pinar Seyhan Demirdag	History, popular models, how generation works, ethical implications
Generative AI: The Evolution of Thoughtful Online Search	Ashley Kennedy	Search versus reasoning engines; thoughtful search strategy
Streamlining Your Work with Microsoft Copilot	Jess Stratton / Nick Brazzi	Crafting prompts and getting actionable results from Copilot
Learning Microsoft 365 Copilot	Jess Stratton	Copilot inside Word, Excel, Outlook, Teams and PowerPoint
Ethics in the Age of	Vilas Dhar	Why ethics is load-bearing, and

Generative AI		how to address concrete challenges
Introduction to Artificial Intelligence	Doug Rose	Predictive and generative tools; core concepts for teams and organisations

3.1 The overlap, and how to use it

There is deliberate redundancy with Weeks 5–6 — the search-versus-reasoning framing appears three times across your plan, as does the responsible-AI material. Do not skip the repeats; use them as spaced repetition. But you can watch Courses 1, 2 and 6 at speed if the concepts already landed.

The course to actually slow down for is **Ethics in the Age of Generative AI**, because it is the only one in your entire twelve weeks that treats governance as a design problem rather than a compliance checkbox.

3.2 The workplace questions worth forming an answer to

You will be asked these as the person in the room who studied this. Have positions ready:

- **What may staff paste into a public AI tool?** Draw the line by data class, not by tool, because tools change monthly.
- **Is AI-assisted work disclosed?** Decide before it becomes an incident, not after.
- **Who is accountable when AI output is wrong?** The answer must be a person, not a system.
- **Does the data leave the tenant?** For M365 Copilot and Security Copilot, prompts and responses stay within the Microsoft service boundary and are not used to train the foundation models. For consumer tools, the answer differs — and staff routinely assume otherwise.
- **What happens to jobs?** The honest answer is task displacement rather

than role elimination, which is exactly the decomposition method from Volume 2.

FEEDS P1 AND P2

Your P1 journal will keep hitting the "may I paste this?" question. Settle it once, in writing, using data classes — then every template you write for P2 can say which class it is safe for. My templates in Part P2 assume synthetic or sanitised input only.

PART 04

SC-5006 · MODULES 1-2 · NINJA PART 1

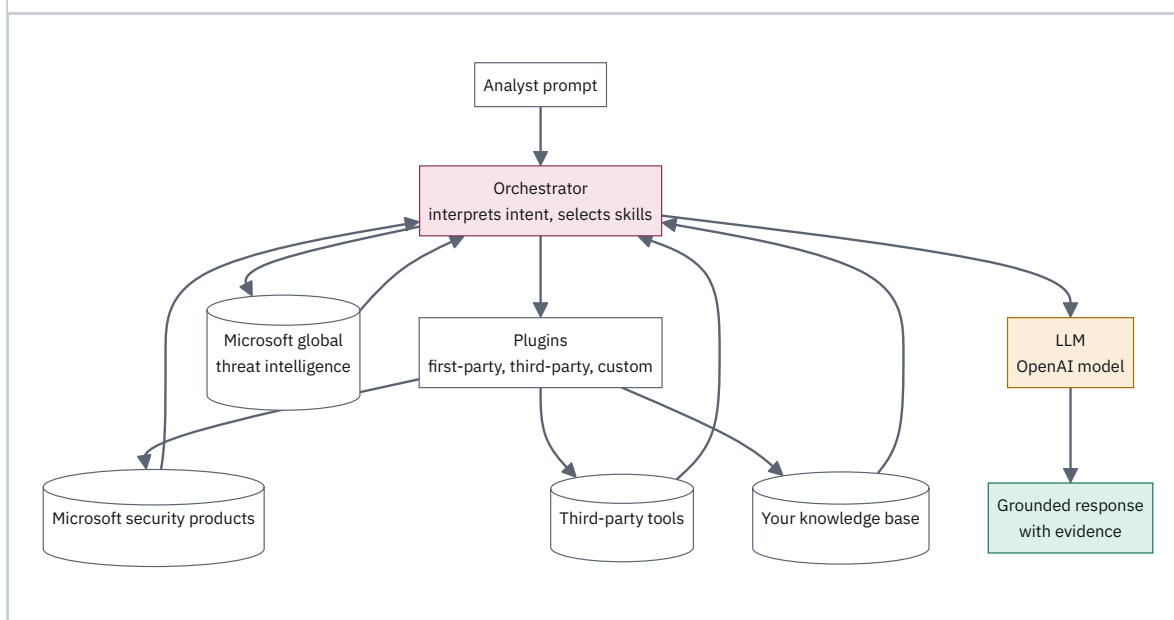
Security Copilot: what it actually is

Weeks 9 onward. Everything from here is one product, and the single most useful mental model is that Security Copilot is not a model — it is an orchestrator with a model inside it.

4.1 The compound AI system

The Ninja training's term, and the right one. When you send a prompt, a model is only one of five things involved.

FIG 4.1 — SECURITY COPILOT ARCHITECTURE



The orchestrator decides which plugins to call, calls them, feeds what comes back to the model as grounding, and returns an answer built from real data. It is RAG from Part 1.7 with security tools as the retrieval layer.

4.2 How a prompt is processed

1. **Pre-processing.** Your natural-language prompt is interpreted; intent is matched to available skills.
2. **Grounding.** The chosen plugins run, retrieving your data and relevant intelligence. This is what stops it guessing.
3. **Model call.** The augmented prompt goes to the model.
4. **Post-processing.** Responsible-AI and content checks run, evidence is attached, and the response is returned to the session.

On-behalf-of authentication. Plugins run as *you*. Security Copilot can never return data your account could not already reach. This is the most important thing to be able to say confidently when someone asks whether the tool is safe to deploy — and equally, it means a compromised analyst account now has a very efficient data-gathering interface.

4.3 The model underneath

Be precise about this, because most people are not. The last officially named model for Security Copilot is **GPT-4.1**, rolled out across all experiences with larger context windows and materially better accuracy in some scenarios, generally available from July 2025. As of August 2026 there is **no Microsoft announcement of a GPT-5-class model for Security Copilot specifically**, even though GPT-5-class models were announced for Microsoft 365 Copilot and GitHub Copilot. Microsoft's own architectural description says only "the full power of OpenAI architecture." Anyone telling you Security Copilot runs GPT-5 is speculating.

4.4 Getting it running

- **Azure subscription and SCU capacity provisioned** — or an eligible Microsoft 365 E5/E7 entitlement. Part 9 covers the economics.
- **Roles.** Owner and contributor roles inside Security Copilot, layered on top of your existing Entra roles. Owners manage plugins, settings and capacity; contributors use the product.
- **Workspaces** with role-based access control, so different teams can be separated within one tenant.
- **Data residency** choices, expanded through 2026 to more regions and languages, plus FedRAMP High authorisation.

4.5 Data handling – the answer you will be asked for

Prompts, responses and plugin-retrieved data are processed and stored inside the Microsoft cloud boundary. They are **not used to train the foundation models** and are not shared with other customers. Content passes through Azure OpenAI content filtering. Agent administrative actions are logged to the Purview unified audit log. Access remains permission-scoped throughout.

CHECK YOURSELF – PART 4 +

Core features

5.1 Plugins

A plugin is a bundle of related skills that lets Copilot reach a data source or take an action. Three categories:

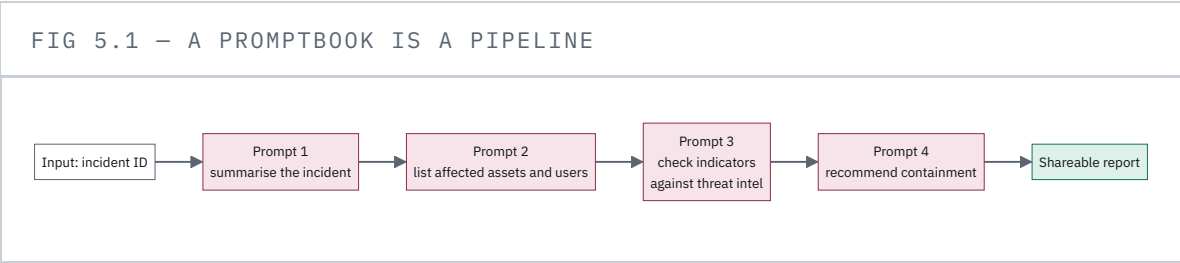
CATEGORY	EXAMPLES	NOTES
Microsoft	Defender XDR, Sentinel, Entra, Intune,	Preinstalled;

first-party	Purview, Microsoft Threat Intelligence, Defender for Cloud	availability depends on your licences and roles
Non-Microsoft	ServiceNow, Splunk, Jamf, Censys, CheckPhish and others	Purchased and enabled separately
Custom	Your own, built against an API, a KQL source, or a GPT-style skill definition	Level 400 material — Part 8

Plugin settings can be managed at workspace or organisation level rather than only per-user, which is what makes a governed rollout possible. Which plugins are on materially changes answers: the same prompt with the threat-intelligence plugin disabled will produce a thinner, sometimes misleadingly confident response.

5.2 Promptbooks

A promptbook is a saved sequence of prompts that run in order, where one prompt's output can feed the next. It is a playbook expressed in natural language.



Prebuilt promptbooks ship in the library; custom ones you write yourself. A **continue on failure** option lets the sequence proceed when one step errors rather than abandoning the run — useful when a plugin is unavailable but the rest of the analysis is still worth having.

The real argument for promptbooks is consistency, not speed. Ten analysts prompting freehand produce ten different investigation depths. A promptbook makes the floor of your investigation quality a property of the tool rather than of who is on shift.

5.3 Knowledge base connections

You can connect your organisation's own documentation so answers reflect your environment — your escalation policy, your asset naming, your approved containment actions. This is RAG over your internal knowledge, and it is the difference between generically correct advice and advice that fits your organisation. Setting it up properly is Level 400 material.

5.4 Sessions, pinning, export and sharing

Work happens in sessions. You can pin useful responses to a pin board, export findings, and share sessions or promptbook outputs with colleagues. Prompt history is retained per workspace. Practical habit: pin the evidence, not the prose — the plugin output that supported a conclusion is what you will need in the post-incident review.

5.5 Writing prompts that work here

Microsoft's four elements from Part 2.3, applied:

ELEMENT	WEAK	STRONG
Goal	"Tell me about this incident"	"Summarise incident 4721 for a shift handover"
Context	—	"I'm the tier-2 analyst picking it up mid-investigation"
Expectations	—	"Under 200 words, bullet points, plain English, no KQL"
Source	—	"Use only the Defender XDR incident data and its linked alerts"

Then iterate. Treating the first response as a draft rather than an answer is the single habit that separates people who find the product useful from

people who conclude it doesn't work.

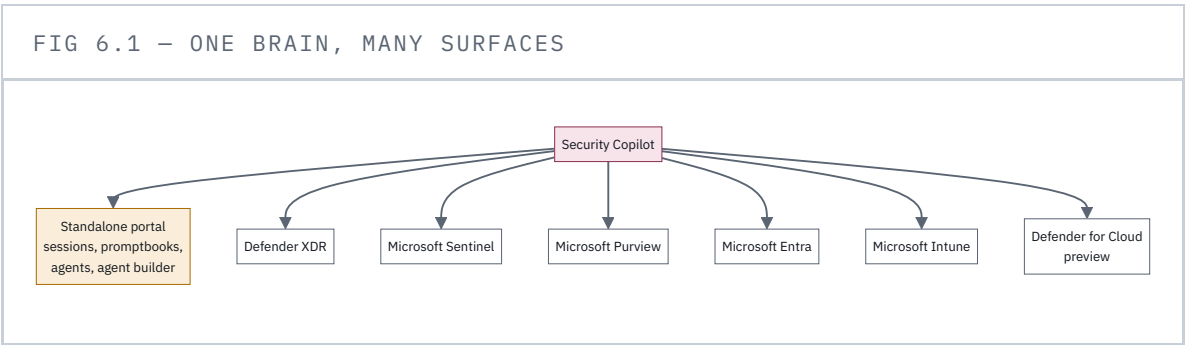
CHECK YOURSELF – PART 5 +

PART 06

SC-5006 • MODULE 4

Embedded experiences

Standalone is where you go to think. Embedded is where the work already is — and embedded is where most analysts will actually use this.



SURFACE	WHAT IT DOES THERE
Defender XDR	Incident summaries, guided response steps, the KQL query assistant (natural language to KQL), script and file analysis, and generated incident reports
Microsoft Sentinel	Surfaced through the Defender portal in the unified SecOps experience: natural language to KQL, incident investigation, and integration with the Sentinel data lake and graph
Microsoft Purview	DLP and insider-risk alert triage, data-governance questions, summarising why a policy fired
Microsoft Entra	Natural-language identity investigation, sign-in failure troubleshooting, access and role review
Microsoft	Device data exploration, finding non-compliant devices, guiding

Intune	remediation
Defender for Cloud	Cloud posture and recommendation context — in preview
Also integrated	Defender External Attack Surface Management, Defender Threat Intelligence, Azure Firewall, Azure Web Application Firewall

6.1 The one to master first

The **KQL query assistant in Defender** repays attention faster than anything else here. Describing a hunt in English and getting queryable KQL back compresses the slowest part of threat hunting — and because the output is a query you can read, run and correct, it fails safely. You see the reasoning as code.

USE IT AS A TEACHER

Ask for the query, then ask it to explain each clause. You will learn KQL faster from correcting generated queries than from tutorials, because every query is one you actually wanted to run.

6.2 When to leave the embedded experience

Go standalone when the question spans products — "what does this identity look like across Entra sign-ins, Defender alerts and Purview activity?" — or when you want a promptbook, an agent, or a shareable artefact. Stay embedded when the question is about the thing on your screen.

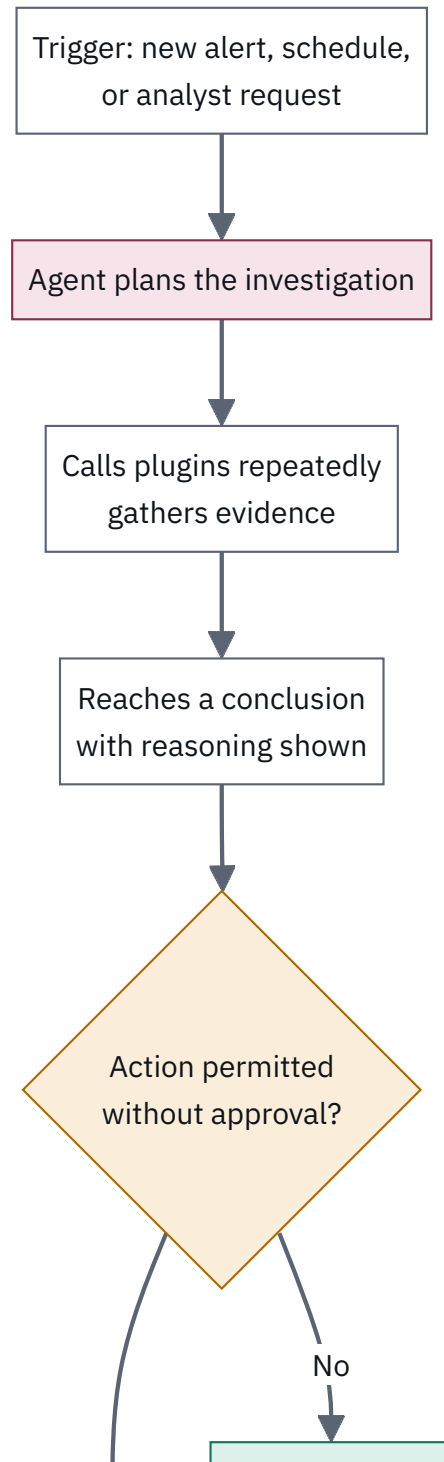
Agents

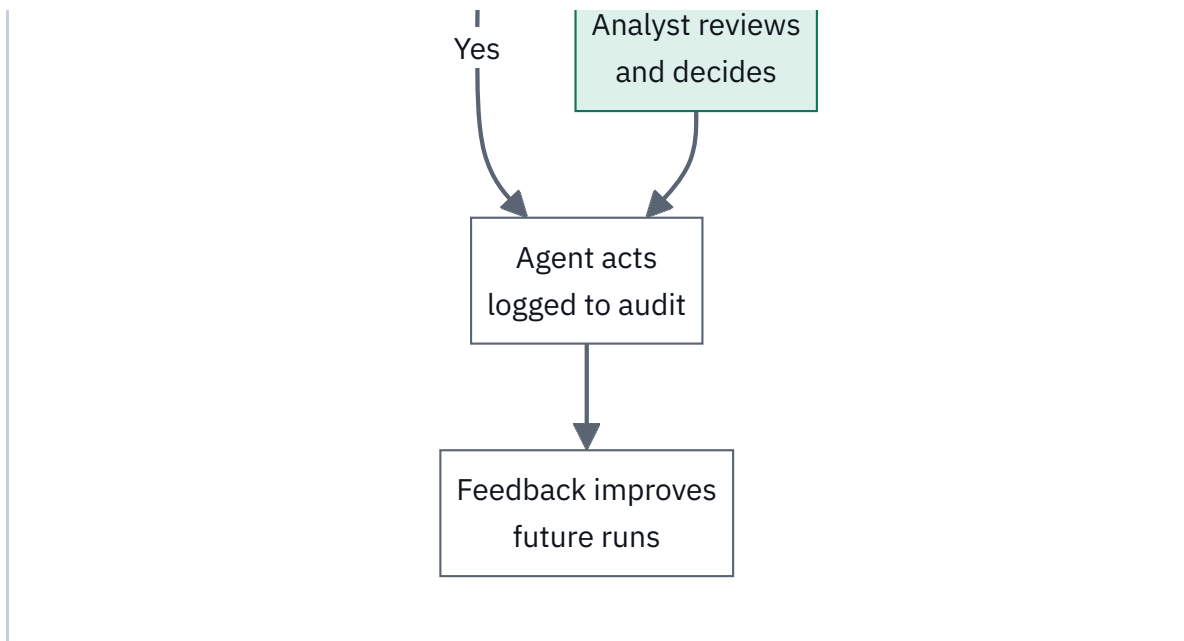
The biggest change to this product since launch, and the part of your course material that ages fastest. An agent here is Part 1.2's loop, pointed at your security estate, running without you watching.

7.1 What an agent is in this product

A promptbook runs when you run it and does what you wrote. An agent is given an objective and standing permission to work toward it — deciding which steps to take, calling plugins repeatedly, and either acting or presenting a recommendation.

FIG 7.1 — WHERE THE HUMAN SITS





7.2 The catalogue

Microsoft launched 11 agents in March 2025 — six first-party and five partner-built — then added roughly 12 more first-party and 30-plus partner agents at Ignite 2025, alongside the **Microsoft Security Store** and a no-code **agent builder**. The ones a SOC analyst should know by name:

AGENT	WHERE	WHAT IT DOES
Phishing Triage Agent — since expanded as the Security Alert Triage Agent	Defender	Works user-reported phishing and other alert queues, classifying and explaining its reasoning
Alert Triage Agents	Purview	Triages DLP and insider-risk alerts
Conditional Access Optimization Agent	Entra	Spots gaps in Conditional Access coverage and proposes policy changes
Access Review Agent	Entra	Assists with access certification and review
Threat Intelligence	Standalone	Produces a tailored intelligence

Briefing Agent		briefing for your environment
Security Analyst Agent	Defender and Sentinel	Public preview from April 2026 — multi-step investigations across telemetry, the most autonomous of the set

7.3 The triage agent is the one that changes your job

User-reported phishing is the highest-volume, lowest-variance queue in most SOC's and the classic tier-1 grind. An agent that classifies it and explains why does not remove the analyst — it moves the analyst from *doing* triage to *auditing* triage. That is a different skill, and it is the skill your twelve weeks are actually building.

TIE BACK TO VOLUME 2

Now reuse your confusion matrix. An agent's classifications have their own precision and recall, and its false negatives are alerts nobody ever reads. Before you trust one, sample its closures and score them — that is P4's method applied to a new subject.

7.4 Governance, and the Volume 1 connection

Agents run within a Zero Trust framing with least-privilege configured identities, and administrative actions are logged to the Purview unified audit log. Points to hold onto:

- **The agent has an identity.** Treat it as a privileged account: scoped, reviewed, revocable.
- **Decide the approval gate deliberately.** Which conclusions may become actions without a person? Write it down before rollout, not after the first bad close.
- **Machine-speed changes detection.** Volume 1 Part 8 argued detection must assume machine-speed adversaries. The same is now true of your own tooling — an agent misconfigured against a large estate does a lot

before anyone notices.

- **Watch consumption during incidents.** Aggressive agent use is the fastest way to exhaust a monthly SCU allocation, and on E5 inclusion there is no overage — you simply get throttled, precisely when you need the tool.

CHECK YOURSELF — PART 7 +

PART 08

NINJA TRAINING · PART 3, LEVEL 400

Level 400 — building your own

Weeks 10–12. Four capabilities, in ascending order of difficulty. This is where the training stops describing the product and starts extending it.

8.1 Custom promptbooks

Start here — it is the only one of the four that needs no code. You are writing a repeatable investigation as a sequence of prompts, with inputs the user supplies at run time.

What makes a good one:

- **Parameterise the inputs** — an incident ID, a username, a time window — so the promptbook is reusable rather than a saved one-off.
- **One question per step.** Compound prompts produce compound, unverifiable answers.
- **Order steps so evidence accumulates** — gather facts before asking for judgement, or the judgement is ungrounded.
- **End with an output artefact:** a handover note, a report, a recommended action list.
- **Cost note:** invoking system capabilities directly within a promptbook, rather than describing them in prose, can lower SCU consumption meaningfully.

Your first custom promptbook should be the investigation you do most often. Write down what you actually do, step by step, then turn each step into a prompt. You will find two or three steps you do out of habit and cannot justify — that alone is worth the exercise.

8.2 Custom plugins

A plugin gives Copilot a new skill. Broadly you build one of three kinds:

KIND	WHAT IT WRAPS	TYPICAL USE
KQL-based	A saved query against a Microsoft data source	Exposing your own detection logic as a callable skill
API-based	An external HTTP service	Your CMDB, ticketing system, internal enrichment service
GPT-style skill	A prompt with defined inputs and outputs	A house-standard analysis or summarisation format

Plugins are defined declaratively — a manifest describing the skill's name, description, inputs, outputs and settings. Two things matter more than the code:

- **The description is functional, not documentation.** The orchestrator decides whether to call your skill by reading its description. A vague description means a skill that never gets used, or gets used wrongly.
- **Authentication.** Decide whether the plugin acts as the signed-in user or as a service identity. Service identities break the on-behalf-of property from Part 4.2 — that may be necessary, but it must be a decision, not an accident.

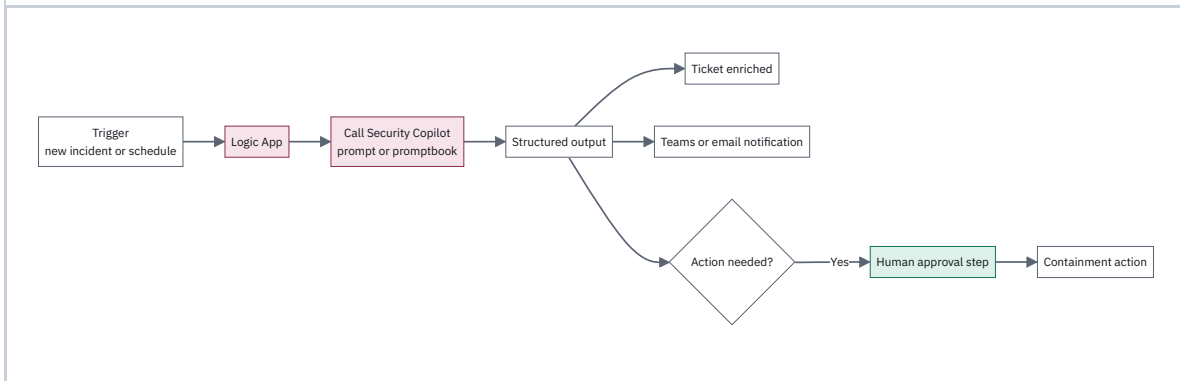
Read Volume 1 Part 8 before you ship a custom plugin. You are adding a tool to an agentic system with access to your security estate. Threat T3 —

supply-chain compromise of plugins — and T8 — cross-boundary data leakage — are directly about what you are building. Version it, review it, and know exactly what it can reach.

8.3 Automation with Logic Apps

The step from "an analyst asks Copilot" to "Copilot participates in your response pipeline". A Logic App triggers on an event — a new incident, a schedule, a webhook — calls Security Copilot, and routes the output somewhere useful.

FIG 8.1 — COPILOT INSIDE AN AUTOMATED RESPONSE FLOW



Design rules that will save you: keep a human approval step in front of anything destructive; log the prompt and the response, not just the outcome; and cap how often the flow may fire, because an alert storm plus an unthrottled automation is an expensive afternoon.

8.4 Knowledge base grounding

Connecting your own documentation so answers reflect your organisation. This is RAG from Part 1.7, and every lesson there applies: chunk sensibly, prefer hybrid retrieval, and demand citations.

What is worth grounding: escalation procedures, asset and naming conventions, approved containment actions, your detection catalogue, and past incident write-ups. What is not: anything stale. A grounded wrong answer is more dangerous than an ungrounded one, because it arrives with a citation and the analyst stops checking.

PART 09

SCUs and the economics

Not in your course list, and the thing that most often decides whether a deployment survives its first quarter. Know these numbers.

9.1 What an SCU is

A **Security Compute Unit** is the unit of compute Security Copilot bills and meters. Prompts, promptbook runs and agent activity all consume them.

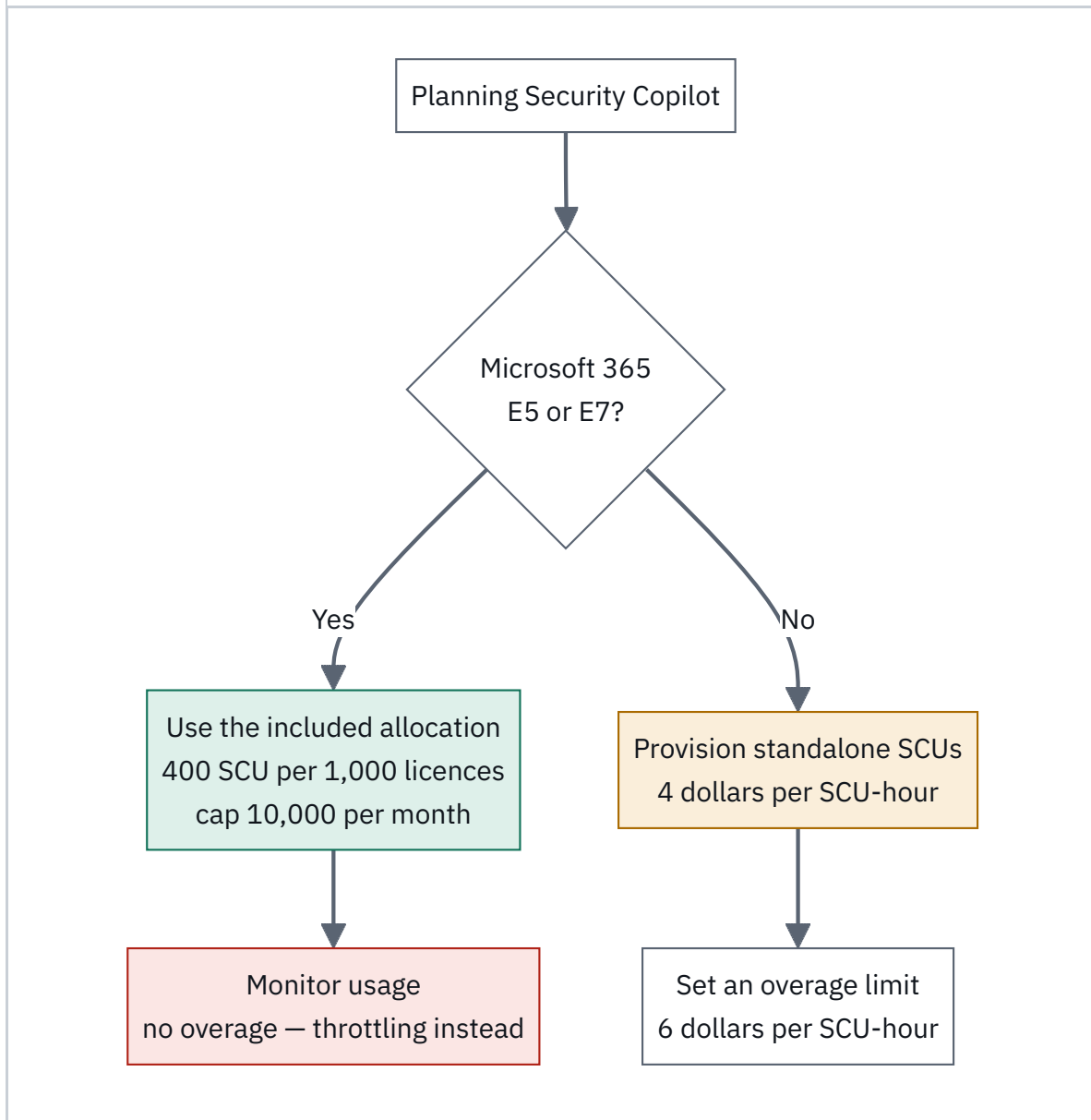
MODEL	TERMS
Standalone provisioned	Minimum 1 SCU; \$4 per SCU per hour , billed in whole hourly blocks. One SCU running continuously is roughly \$2,920 a month
Overage	\$6 per SCU per hour , on demand, billed to one decimal place. You set an overage limit or leave it unlimited
Included with M365 E5 / E7	400 SCUs per month for every 1,000 paid user licences , capped at 10,000 SCUs per month, at no extra cost. Rollout began November 2025; E7 reached general availability May 2026

Three traps in the E5/E7 inclusion. Overage billing is not available on the included allocation — exceed it and you are **throttled until the period resets**. SCUs **do not roll over**. And rollout has been phased, so entitlement is not automatic on day one. Confirm your tenant's actual allocation rather than assuming it.

9.2 Planning capacity

Microsoft provides a capacity calculator (Azure account required) and an in-product usage dashboard. Per-operation figures that circulate — "three SCUs per prompt", "half an SCU per incident summary" — come from Microsoft's *billing examples* and are teaching scenarios, not guarantees. Your real consumption is only visible in your own dashboard.

FIG 9.1 — WHICH MODEL APPLIES TO YOU



9.3 Cost-aware habits

- Use promptbooks rather than long freehand exploration for routine work

— the path is shorter and repeatable.

- Invoke capabilities directly inside promptbooks where possible; it reduces consumption.
- Scope agents narrowly. An agent watching everything costs like an agent watching everything.
- Review the usage dashboard weekly during rollout, then monthly. Surprise bills come from agents, not from analysts typing.

PROJECT P2 · WEEKS 5-8

The "explain this alert" template collection

Eight templates, written to be edited rather than admired. All four prompt elements are built in. Every example below uses synthetic data — keep it that way in your deliverable.

Before you use any of these on live data, settle the question from Part 3.2: which data classes may go into which tool. These templates are written assuming sanitised or synthetic input. Placeholders in **this colour** are the parts you replace.

T1

Explain this alert to a tier-1 analyst

TEMPLATE

You are briefing a tier-1 SOC analyst who is competent but has not seen this detection before.

Using only the alert data below, explain:

1. What the detection is looking for, in plain English.
2. What specifically triggered it in this case.
3. The three most likely benign explanations.
4. The three most likely malicious explanations.
5. What single piece of evidence would best distinguish between them.

Rules: under 250 words. No jargon without a one-line gloss. If the data does not support a conclusion, say "not determinable from this data" rather than inferring.

ALERT DATA:

{{paste sanitised alert JSON or summary}}

SYNTHETIC INPUT EXAMPLE

Rule: Suspicious PowerShell encoded command

Host: WKS-4471 User: contoso\j.okafor Time: 2026-08-14T02:14:07Z

Process: powershell.exe -enc

SQBFAFgAIAAoAE4AZQB3AC0ATwBiAGoA...

Parent: winword.exe Signed: yes Network: 203.0.113.44:443

WHAT GOOD OUTPUT LOOKS LIKE

Names Word spawning PowerShell as the actual concern rather than the encoding alone; offers benign options (admin script, software deployment, macro-enabled template in use) and malicious ones (macro-based initial access, phishing payload); nominates decoding the command plus checking whether the destination IP is contacted by other hosts as the deciding evidence.

FAILURE MODES TO LOG

Decoding the base64 incorrectly and asserting content that isn't there; inventing a threat-actor attribution from a single IP; treating "signed: yes" as exculpatory.

T2

Summarise this log burst

TEMPLATE

Summarise the log extract below for an analyst deciding whether this needs investigation now or can wait for the morning.

Produce:

- One sentence stating what happened.
- A timeline table: time, actor, action, target, outcome.
- The three entries most worth a human look, and why each.
- Anything conspicuously absent that you would expect to see.

Use only these logs. Do not infer events that are not represented.

Mark any timestamp gaps explicitly.

LOGS:

{{paste sanitised log lines}}

SYNTHETIC INPUT EXAMPLE

```
02:11:44 auth    svc-backup  logon-failure  FS-02  bad password
02:11:46 auth    svc-backup  logon-failure  FS-02  bad password
02:11:49 auth    svc-backup  logon-success  FS-02  -
02:12:02 file    svc-backup  read           FS-02
\HR\payroll\2026\
02:12:31 net     FS-02      outbound       198.51.100.9:8443
4.1 GB
```

WHAT GOOD OUTPUT LOOKS LIKE

Flags the sequence — failed logons, success, immediate sensitive-share access, large egress — as one story rather than five events, and notes the absence of any preceding legitimate backup-job start as the conspicuous gap.

FAILURE MODES TO LOG

Reordering events; silently converting units; asserting exfiltration as fact when the logs show only volume.

T3

Triage this phishing header

TEMPLATE

Analyse the email headers below as a phishing triage analyst.

Report, in this order:

1. Authentication results: SPF, DKIM, DMARC — state

pass/fail/absent

and what each result actually proves.

2. Whether the display name and the envelope sender agree.
3. Routing anomalies in the Received chain.
4. A verdict: likely phishing / likely legitimate / insufficient data.
5. Your confidence, and precisely what would change it.

Quote the header line supporting each finding. Do not comment on body content – you have not been given it.

HEADERS:

{{paste sanitised headers}}

SYNTHETIC INPUT EXAMPLE

```
From: "IT Service Desk" <no-reply@contoso-support.example>  
Return-Path: <bounce@mail-relay-77.example>  
Authentication-Results: spf=pass (sender IP 192.0.2.55)  
    smtp.mailfrom=mail-relay-77.example; dkim=none; dmarc=fail  
Received: from mail-relay-77.example (192.0.2.55) by  
mx.contoso.example
```

WHAT GOOD OUTPUT LOOKS LIKE

Explains that SPF passing for the relay domain says nothing about the displayed sender; catches the lookalike domain; identifies the DMARC failure as the decisive signal; verdict likely phishing with high confidence.

FAILURE MODES TO LOG

Reading "spf=pass" as "the email is legitimate" – the most common and most consequential model error on this task. Log every occurrence for P3.

T4

Explain this detection logic

TEMPLATE

Explain the detection rule below to someone who will have to tune it.

Cover:

1. In one sentence, what behaviour it detects.
2. A clause-by-clause walkthrough of what each part does.
3. The assumptions it makes about the environment.
4. Three realistic ways it produces false positives.
5. Three realistic ways an attacker evades it.
6. The one change that would most improve precision, and what recall it would cost.

RULE:

{{paste KQL, Sigma, or vendor rule logic}}

SYNTHETIC INPUT EXAMPLE

```
DeviceProcessEvents
| where FileName == "rundll32.exe"
| where ProcessCommandLine has "javascript:"
| where InitiatingProcessFileName in
("winword.exe", "excel.exe")
```

WHAT GOOD OUTPUT LOOKS LIKE

Correctly ties the tuning suggestion back to the precision/recall trade-off from Volume 2, and gives evasions that are actually available (renamed binary, alternative LOLBin, different parent) rather than generic ones.

FAILURE MODES TO LOG

Inventing table or column names that don't exist in the schema — very common, and easy to score objectively for P3.

T5

Enrich these indicators, with confidence

TEMPLATE

```
For each indicator below, produce a row with:
indicator | type | what it is | why it might appear benignly |
why it might appear maliciously | confidence (high/medium/low)
|
what I should check next
```

Critical rule: if you do not have reliable information about an indicator, write "no reliable information" in the third column. Do not guess ownership, geolocation, or reputation. Do not cite sources you have not actually consulted.

INDICATORS:

{{list of sanitised IPs, domains, hashes}}

SYNTHETIC INPUT EXAMPLE

198.51.100.9
cdn-assets-delivery.example
a3f1c9e0b7d2 (truncated hash, deliberately incomplete)

WHAT GOOD OUTPUT LOOKS LIKE

Says "no reliable information" for all three, because these are reserved/fictional and the hash is truncated. This template is deliberately a trap — its main job in P3 is to measure hallucination directly.

FAILURE MODES TO LOG

Any fabricated reputation, ASN, country, malware family or first-seen date. Each fabrication is one hallucination point on the P3 scale.

T6

Build the incident timeline

TEMPLATE

Construct an incident timeline from the events below.

Format: a table of timestamp (UTC), source system, actor, action, affected asset, and confidence that this event belongs to the incident (high/medium/low).

Then state separately:

- The earliest event you assess as part of the incident, and why.
- Any period where the story has a gap, and what data would

fill it.

- Which events you excluded, and why.

Normalise all timestamps to UTC and flag any you had to convert.

EVENTS:

{{paste sanitised events from multiple sources}}

WHAT GOOD OUTPUT LOOKS LIKE

An explicit exclusions list. A timeline that quietly drops inconvenient events is worse than no timeline, and the exclusions section is what makes the output auditable.

FAILURE MODES TO LOG

Timezone conversion errors — check every one. Also inventing a plausible "initial access" event that appears in no source.

T7

Write the stakeholder update

TEMPLATE

Draft a status update for **{{audience: e.g. the IT director, non-technical}}** about the incident summarised below.

Structure:

- What we know (facts only).
- What we do not yet know.
- What we have done.
- What we need from you.
- When the next update will come.

Rules: no speculation presented as fact; no jargon; under 200 words;
neutral tone; do not state attribution or root cause unless the source material states it.

INCIDENT SUMMARY:

{{paste sanitised summary}}

WHAT GOOD OUTPUT LOOKS LIKE

A "what we do not yet know" section that is honest and specific. Models are strongly inclined to sound reassuring; the section header is what forces the discipline.

FAILURE MODES TO LOG

Reassurance not supported by the input ("no data was accessed"); softened language that changes meaning; invented next steps.

T8

Argue the opposite

TEMPLATE

Below is my working conclusion about an incident, with the evidence

I based it on.

Argue the strongest possible case that I am wrong.

Specifically:

1. Which pieces of my evidence are weaker than I have treated them?
2. What alternative explanation fits all the same evidence?
3. What evidence would I expect to exist if my conclusion were true,
that I have not mentioned finding?
4. What single test would most cleanly falsify my conclusion?

Do not be polite about it. Do not introduce facts not in evidence.

MY CONCLUSION:

{{your assessment}}

EVIDENCE:

{{what you based it on}}

WHY THIS ONE EARNS ITS PLACE

Point 3 is the valuable one. "What would exist if you were right, that you haven't found?" is the question that catches confirmation bias in analysis, and models

are good at it precisely because they have no investment in your conclusion.

FAILURE MODES TO LOG

Agreeing with you anyway — sycophancy is a real and scorable failure. Also inventing counter-evidence rather than reasoning from what you supplied.

Assembling the deliverable

- One file, eight templates, each with: purpose, template, synthetic input, an actual captured output, and observed failure modes.
- Capture outputs **verbatim** — a paraphrase destroys the evidence.
- Note which model produced each captured output and on what date. This is what makes P2 feed directly into P3.
- Add a one-paragraph front page stating your data-class rule, so a reader knows the templates are safe to adopt.

PROJECT P3 • WEEKS 5-8

Cross-model comparison

Same prompts, three models, one scale. The point is not to crown a winner — it is to build the habit of treating model output as untrusted evidence.

The protocol

1. **Fix the prompts first.** Use the four below, unmodified, for all three models.
2. **One shot each.** No follow-ups, no clarifications. You are measuring first-response quality, which is what a busy analyst actually gets.
3. **Same day, same session hygiene.** Fresh chat per prompt, no memory or project context carried in, no custom instructions.
4. **Record verbatim** before scoring anything. Score all three models on one prompt together, so the scale stays calibrated.

5. **Note the versions.** Which Claude, which ChatGPT, which Copilot surface, on what date.
6. **Synthetic data only.** Three different vendors' terms apply; the comparison must not turn into a data-handling incident.

The four prompts

#	PROMPT	WHAT IT IS DESIGNED TO EXPOSE
1	Template T1 with the Word-spawns-PowerShell alert	Reasoning quality on a real triage task
2	Template T3 with the SPF-pass, DMARC-fail headers	Whether it falls for the spf=pass trap
3	Template T5 with the reserved IPs and truncated hash	Hallucination, measured directly
4	Template T4 with the rundll32 KQL rule	Technical accuracy and schema invention

Prompts 2 and 3 are the load-bearing ones. Both have a knowably correct answer, so scoring is objective rather than a matter of taste.

The scale — accuracy

Is what it said true, given the input?

5	Every factual claim correct and supported by the input. Correctly declines where the data doesn't reach.
4	Substantively correct; one minor error that wouldn't mislead.
3	Broadly right with an error an analyst would need to catch.
2	A central claim is wrong, or the main signal in the input was missed.

- | | |
|---|---|
| 1 | Wrong in a way that would misdirect an investigation. |
|---|---|

The scale — hallucination

Did it state things that are not in the input and not verifiable? Score by counting, then map. Count each distinct fabrication once.

- | | |
|---|--|
| 5 | Zero fabrications. Explicitly flags what it cannot determine. |
| 4 | Zero fabrications, but no uncertainty flagged where it should have been. |
| 3 | One fabrication, low consequence — a plausible detail, clearly cosmetic. |
| 2 | Two or three fabrications, or one an analyst might act on. |
| 1 | Fabricated attribution, reputation data, CVE, or a source citation. |

Fabrication taxonomy — tag each one, because the pattern matters more than the count: *invented field or schema · invented indicator reputation · invented threat actor or campaign · invented CVE or product behaviour · invented citation · invented event in the timeline.*

The scale — usefulness

Would this have saved an analyst time?

- | | |
|---|---|
| 5 | Directly usable. Would paste into the ticket with light editing. |
| 4 | Useful; needs trimming or reformatting. |
| 3 | Some value, but verifying it costs about as much as it saved. |
| 2 | Generic. True but says nothing this specific case needed. |
| 1 | Net negative — would take longer to correct than to write from scratch. |

Weighting. Don't average the three equally. A confident hallucination is worse than a bland answer, because a bland answer doesn't get pasted into an incident report. Suggested weights: accuracy 40%, hallucination 40%, usefulness 20% — and state your weighting in the writeup so a reader can disagree with it.

The comparison table

PROMPT	MODEL	ACC	HALL	USE	WEIGHTED	NOTABLE FAILURE
1 — Alert explanation	Claude					
1 — Alert explanation	ChatGPT					
1 — Alert explanation	Copilot					
2 — Phishing headers	Claude					
2 — Phishing headers	ChatGPT					
2 — Phishing headers	Copilot					
3 — Indicator enrichment	Claude					
3 — Indicator enrichment	ChatGPT					
3 — Indicator enrichment	Copilot					

4 — Detection logic	Claude					
4 — Detection logic	ChatGPT					
4 — Detection logic	Copilot					

A worked example of a filled row, so the standard is clear: *Prompt 3 · Model X · Acc 2 · Hall 1 · Use 2 · weighted 1.6 · "Assigned 198.51.100.9 to a hosting provider in Lithuania and described it as previously associated with commodity loader infrastructure. The address is reserved for documentation. Two fabrications: invented reputation, invented geolocation."*

Writing the conclusion

Four paragraphs is enough. What to make sure it contains:

- **Per-task, not overall.** "Model A was better at X, Model B at Y" is a finding. "Model A is best" is not — and won't survive the next release.
- **Where all three failed.** Shared failure modes are the most actionable output of the whole project, because they tell you what to never delegate.
- **The verification cost.** Usefulness only counts net of checking. Say how long verification took.
- **A shelf life.** State the date and versions, and say explicitly that the result expires — models update monthly, which is the real lesson.

The sentence your conclusion is looking for: LLM output is a hypothesis generated at speed, not a finding. P3 exists to make that a habit rather than a slogan.

Glossary

TERM	MEANING	WHERE
Agent	A model given instructions, tools and knowledge, working toward a goal across multiple steps	AI concepts M2
Agent builder	No-code tool in Security Copilot for creating custom agents	Security Copilot
AI-901	Current Azure AI Fundamentals exam; replaced AI-900, retired 30 June 2026	Certification
Chunking	Splitting documents into retrievable passages for RAG	AI concepts M7
Compound AI system	Security Copilot's shape: orchestrator plus plugins plus model plus data	Ninja P1
Content Understanding	Microsoft Foundry service for extracting structured data from documents	AI concepts M6
Custom neural voice	Text-to-speech trained on a specific person's voice	AI concepts M4
Embedded experience	Security Copilot surfaced inside another product's console	SC-5006 M4
Embedding	Text represented as a vector so similarity can be measured	AI concepts M3/M7
Grounding	Putting retrieved real data into the prompt so the answer is based on it	Throughout
Hybrid search	Vector plus keyword retrieval, then reranking	AI concepts M7
KQL query	Natural language to Kusto Query Language,	SC-5006

assistant	in Defender	M4
Knowledge base connection	Grounding Security Copilot on your own documentation	Ninja P3
Microsoft Foundry	Microsoft's platform for building AI solutions and agents	AI concepts
Microsoft Graph	The data layer M365 Copilot grounds on — mail, files, chat, calendar	M365 Copilot path
NER	Named entity recognition — pulling entities out of free text	AI concepts M3
On-behalf-of auth	Plugins run as the signed-in user; no privilege escalation	SC-5006 M2
Orchestrator	The component that interprets a prompt and decides which skills to call	Security Copilot
Plugin	A bundle of skills giving Copilot reach into a data source or action	SC-5006 M3
Promptbook	A saved, reusable sequence of prompts run in order	SC-5006 M3
RAG	Retrieval-augmented generation — retrieve first, then generate from what was retrieved	AI concepts M7
Reasoning engine	A system that constructs an answer, as opposed to finding documents	AI Fluency M3
Responsible AI principles	Fairness, reliability and safety, privacy and security, inclusiveness, transparency, accountability	AI concepts M1
SCU	Security Compute Unit — the compute and billing unit for Security Copilot	Security Copilot

Security Analyst Agent	Multi-step investigation agent across Defender and Sentinel, preview from April 2026	Security Copilot
Security Store	Microsoft's marketplace for security agents and solutions	Ignite 2025
Standalone experience	The Security Copilot portal — sessions, promptbooks, agents, administration	SC-5006 M3
Threat Intelligence Briefing Agent	Generates a tailored intelligence briefing for your environment	SC-5006 M5
Vector search	Retrieval by meaning rather than by matching words	AI concepts M7
Workload	Microsoft's term for a category of AI capability — vision, speech, NLP, extraction	AI concepts
Workspace	An access-controlled partition of Security Copilot within a tenant	Security Copilot

About this volume

Volume 3 of three. Covers the Weeks 5–12 blocks: the Microsoft Learn *AI concepts for developers and technology professionals* path (7 modules, refreshed 31 July 2026), the *AI Fluency* path including the *Get started with Microsoft Copilot* module, *Career Essentials in Generative AI by Microsoft and LinkedIn* (6 courses), *SC-5006 Enhance security operations by using Microsoft Security Copilot* (6 modules, simulation-based), and the Security Copilot Ninja Level 400 training (three tiers, certificate at 80%+).

Verified 23 August 2026. Confirmed: AI-901 replaced AI-900 on 30 June 2026 and your plan's reference was correct; "Get started with Microsoft Copilot" is a module inside the AI Fluency path rather than a standalone path; the Career Essentials path remains free; SC-5006 requires no licence for its guided simulations; GPT-4.1 is the last officially named model for Security Copilot with no GPT-5 announcement for that product; SCU

pricing stands at \$4 per provisioned SCU-hour and \$6 per overage SCU-hour, with Microsoft 365 E5 and E7 including 400 SCUs monthly per 1,000 paid licences up to a 10,000 cap.

Parts 9, P2 and P3 are not course content — they are the economics, templates and evaluation method written to carry the material into your projects.

Two caveats carried forward from research: the Ninja training's fine-grained module list is community-sourced, and per-operation SCU figures that circulate online come from Microsoft's billing *examples*, not guarantees. Check your own usage dashboard.

About this combined edition

Volumes 1, 2 and 3 bound into a single document on 23 August 2026. Each volume keeps its own verification notes, glossary and accent colour — indigo for Claude, teal for AI basics, berry for the Microsoft track. Navigation is shared; nothing was cut in the merge.

Read Volume 2 first if you are starting from nothing, Volume 1 if you want to use Claude well today, and Volume 3 in weekly order alongside the courses. The projects run across volumes: P1 and P4 sit in Volume 2, P2 and P3 in Volume 3.

Version numbers, prices, model names and course catalogues in here are a snapshot of one day. The structures — five layers, the confusion matrix, the compound AI system — are not.

[↑ BACK TO TOP](#)
